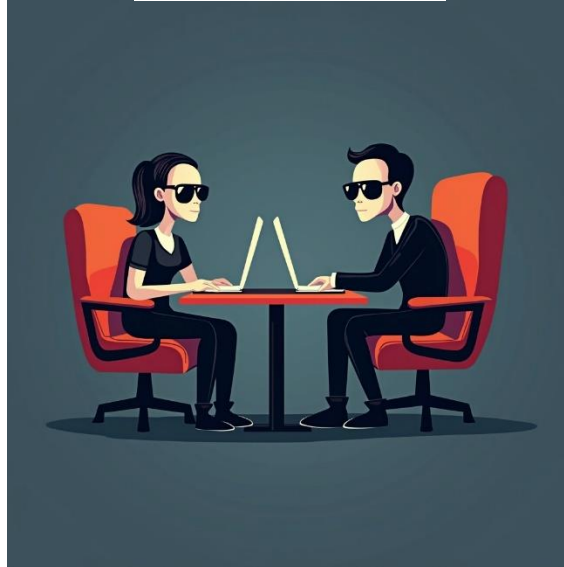




RandChat

שם העבודה: RandChat

שם התלמיד: עמית דור

ת"ז התלמיד: 216239541

שם הבית ספר: תיכון אלון רמת השרון

שם החלופה: הגנת סייבר

שם המנחה: עידן פלדברג

תאריך הגשה: 23/5/2025

תוכן עניינים

4	מבוא
4	ייזום
4	תיאור ראשוני של המערכת
4	הגדרת לקוח
4	הגדרת יעדים
5	בעיות תועלות וחסכונות
5	סקירת פתרונות קיימים
5	סקירת טכנולוגית הפרויקט
5	תיחום הפרויקט
6	פירוט תיאור המערכת
7	פירוט יכולות
7	פירוט בדיקות
8	תכנון זמנים
9	ניהול סיכונים
10	תיאור תחום הידע-פרק מילולי
10	פירוט היכולות בצד השרת
12	פירוט היכולות בצד הלקוח
15	מבנה/ארכיטקטורה של הפרויקט
15	תיאור הארכיטקטורה של המערכת המוצעת ותיאור החומרה
16	תיאור הטכנולוגיה הרלוונטית
17	תרשימי זרימה
21	תיאור האלגוריתמים מרכזיים בפרויקט
24	תיאור סביבת הפיתוח
24	תיאור פרוטוקול התקשורת
24	תיאור מילולי
25	תיאור כלל ההודעות הזורמות במערכת
31	תיאור מסכי המערכת
40	תיאור מבני הנתונים
41	סקירת חולשות ואיומים
42	מימוש הפרויקט

42.....	מודלים/מחלקות מיובאים
43.....	מודלים/מחלקות שאני מפתח
52.....	קטעי קוד האלגוריתמים המרכזיים
58.....	מסמך בדיקות מלא:
60.....	מדריך למשתמש
60.....	עץ קבצים
60.....	התקנת במערכת
60.....	פירוט הסביבה הנדרשת
60.....	פירוט הכלים הנדרשים
61.....	מיקומי הקבצים
61.....	נתונים התחלתיים
63.....	רשת
63.....	ארכיטקטורה מינימלית
63.....	משתמשי המערכת
73.....	רפלקציה אישית
75.....	ביבליוגרפיה
76.....	נספחים

מבוא

ייזום

תיאור ראשוני של המערכת

אפליקציית צ'אט שבא ניתן להכיר ולשוחח עם אנשים באנונימיות וברנדומליות בהתאם לשפות שהשתמש מכיר. המוצר המוגמר צריך להיות עם יכולת להתחבר למשתמש לשמור את המידע ולפתוח צ'אט לשיחה עם חבר רנדומלי ואנונימי. בחרתי בפרויקט זה מכיוון שהוא יכול מאוד משמעותי עבורי ועבור תהליך הלמידה שלי וכמובן יתרום לאחרים להכיר אחד את השני ויעזור להם להתגבר על קשיי שיחה או לחץ, פרויקט זה יכול להוות להם מקום שהם יכולים לבטא את עצמם ולהרגיש יותר בטוחים. במהלך הכנת הפרויקט אני צופה להיתקל בהרבה אתגרים, כמו החיבור של המשתמשים עם השרת ביחד עם אפקט הרנדומליות וההחלפה המהירה של שיחות בין משתמשים, טיפול באפשרות שרק אחד מן המשתמשים מדבר בשיחה והאחר אינו רושם כלום, ובעיות בבניית מסד הנתונים.

הגדרת לקוח

המערכת מיועדת לכל אדם אשר שרוצה לדבר עם אדם אחר, להכיר אחרים ולרכוש חברים, לתרגל כתיבה ושיחה בשפות שונות עם אנשים שונים או סתם להעביר את הזמן בשיחה בהתכתבות.

הגדרת יעדים

המטרה של הפרויקט היא לתרום לאחרים להתמודד עם קשיים חברתיים, לחץ או חרדה הנגרמים מדיבור עם אנשים אחרים, ועצם זה שהפרויקט רנדומלי ואנונימי הוא מהווה מקום שבו אנשים יכולים לתקשר ללא פחד ולבטא את עצמם ביותר נוחות וביותר ביטחון. ובנוסף הפרויקט גם מהווה מקום לאנשים אשר לומדים שפה ורוצים לתרגל אותה בשיחה עם אחרים.

בעיות תועלות וחסינות

בדור של היום קיימת בעיה של רבים בתקשורת עם אחרים, בעיה זו נגרמה משלל סיבות, עם ההתקדמות של העידן הטכנולוגי, בגלל הסגרים בקורונה והמצב במלחמה. ובמיוחד במצב כזה חשוב לתקשר אחרים, להשתפר בתקשורת ולשפר את מצבינו. הפרויקט תיתן מקום פרטי לשוחח עם אחרים למרות הפחדים והלחצים ולתרום לאנשים שיותר מתקשים.

סקירת פתרונות קיימים

פתרון שידוע שקיים הוא אתר בשם omegle שבוא ניתן לשוחח עם אחרים ברנדומליות אך לא באנונימיות, באתר הזה התקשורת מתבצעת באמצעות שיחת וידאו שבה ניתן לדבר ולשוחח ברנדומליות עם אפשרות להחליף בין אנשים וההבדלים המשמעותיים בפרויקט שלי הוא שבו השיחה והתקשורת אנונימיים וגם השיחה מתבצעת בצ'אט במקום עם מיקרופון ומצלמה.

סקירת טכנולוגיית הפרויקט

הטכנולוגיה בה השתמשתי בפרויקט זה אינה חדשה וכמעט ואינה חורגת מגבול החומר שנלמד בבית הספר, אבל השתמשתי בחלק מהפרוטוקולים ידנית ולא דרך המודולים הכתובים.

תיחום הפרויקט

הפרויקט משלב מספר תחומים כגון: בסיס נתונים, תקשורת שרת-לקוח, הצפנות, תהליכונים, ותחומי עניין נוספים מתוך מדעי המחשב, רשתות ועולם הסייבר.

רשתות – בפרויקט זה התקשורת בין השרת ללקוחות מתבצעת באמצעות ספריית socket של פייתון עם פרוטוקול TCP. המערכת פועלת בשיטת שרת מרובה משתמשים, כך שכל פעולה בין משתמשי קצה עוברת דרך השרת. הלוגיקה ברובה מתבצעת בצד השרת. התקשורת במערכת תלויה בתקשורת בין השרת המרכזי לבין הלקוחות.

מערכת הפעלה – מערכת הפרויקט הינה מערכת גנרית שיכולה להתאים למערכות שונות ומשתמש בתהליכונים (multi-threading). עם זאת, בשלב זה המערכת פועלת רק במחשבים נייחים וניידים עם מערכת הפעלה של windows וטרם נוסתה על מערכות אחרות, ובנוסף לא נבנתה עדיין אפליקציה למובייל.

הצפנה - על מנת לאבטח את המידע של משתמשי מערכת הפרויקט, קיימים אלגוריתמים שונים מתחום ההצפנה, כגון:

- הצפנת RSA בתקשורת בין השרת למשתמש
- והצפנת Hashing למד נתונים.

הגנת סייבר - מערכת הפרויקט כוללת הגנה מפני מתקפות סייבר כגון:

- Sql Injection
- זיהוי והגנה מיטבית מפני man in the middle.
- DDoS – Distributed Denial of Service

ממסדי נתונים - מערכת זו משתמשת בבסיס נתונים מסוג SQLite לאחסון המידע.

שפות תכנות - שפת התכנות העיקרית בפרויקט היא Python וקיים גם שימוש בשאילתות משפת SQL בפניה למסד הנתונים.

ממשק משתמש גרפי (GUI) - מרכיב זה בא לידי יישום בפרויקט בשימוש בממשק customtkinter.

מערכת זו אינה עוסקת באתרי אינטרנט. WEB (שימוש בשפות HTML, CSS, JavaScript ועוד).

פירוט תיאור המערכת

כאמור RandChat היא מערכת דיגיטלית שבה ניתן לשוחח ברנדומליות ובאנונימיות עם משתמשים אחרים.

ממשק המשתמש הגרפי שנוצר באמצעות customtkinter, בנוי כך שבפתיחת התוכנה המשתמש יוכל לעשות אחת מן הפעולות הבאות:

1. להיכנס לחשבון קיים
2. ליצור משתמש חדש
3. להיכנס למשתמש Admin.

בתהליכי הכניסה לחשבון קיים וביצירת משתמש חדש המשתמש ידרש להזין שם משתמש וסיסמה שתקינותם תיבדק אל מול מסד הנתונים בשרת. לאחר מכן בתהליך היצירת משתמש חדש יפתח מסך נוסף בו יצטרך לבחור ממספר שפות שמופיעות את השפות שהוא מעוניין לדבר בהן. לאחר שהסתיימו פעולות ההרשה או הכניסה למערכת יפתח מסך הבית.

במסך הבית המשתמשים יכולים לבצע 2 פעולות:

1. ללכת לשוחח עם משתמשים מחוברים
2. לשנות את השפות שהם מעוניינים לשוחח עימם

לאחר שמשתמש לוחץ על כפתור ומציין שרוצה ללכת לשוחח נפתח לו מסך המתנה ובוא הוא מחכה עד אשר השרת מוצא לו משתמש לשוחח איתו בהתאם לשפה שלו ובתנאי שלא דיבר איתו פעם אחרונה או

לחוץ על X כדי לחזור למסך הבית ולהפסיק לחכות. לאחר שהשרת שיבץ אותו עם משתמש נוסף נפתח מסך חדש בו יש למשתמש כמה אפשרות:

1. ללחוץ על X ולחזור למסך הבית
2. להתכתב עם המשתמש שהשרת חיבר אותו עימו.
3. ללחוץ על כפתור random שלאחר לחיצתו הוא יוחזר למסך ההמתנה ויחזור אותו תהליך שצוין מקודם על מסך ההמתנה.

פירוט יכולות

יכולות משתמש:

- הרשמה והתחברות ומערכת
- קביעת ושינוי שפות
- ניהול צ'אט אחד על אחד עם משתמש רנדומלי ואנונימי
- שינוי משתמש שאיתו משוחחים לאחד אחר

יכולות Admin:

- לראות את המשתמשים המחוברים
- לראות את המשתמשים המדברים
- לראות את השפות שהמשתמשים מדברים בהן

פירוט בדיקות

בדיקה 1: האם המידע עובר כמו שצריך בין צדדי התקשורת?
כדי לבדוק זאת הדפסתי את המידע בצד השולח ובצד המקבל ווידאתי שתוכל שתי ההדפסות זהה.

בדיקה 2: תקינות מנגנון ההצפנה והפענוח – וידאתי זאת האמצעות הדפסת המידע לפני ההצפנה, והדפסת המידע לאחר ההצפנה ופענוח.

בדיקה 3: תפקוד מסד הנתונים. בדיקה זו מתחלקת לשני חלקים:

1. האם בסיס הנתונים מתעדכן בהתאם לפעולות של המשתמשים במסגרת הפיצ'רים המוצעים במערכת?

כדי לבדוק זאת, ערכתי סימולציה שבא אני משתמש הקצה, בה בחנתי את הפיצ'רים ולאחר מכן בדקתי אם טבלאות של בסיס הנתונים שנמצאות בDB Browser SQLite התעדכנו. חלק זה של הבדיקה הוא רלוונטי לפיצ'רים כמו הרשמה ועדכון של המשתמש במסדר נתונים ועוד.

2. האם הפיצ'רים המוצעים במערכת פועלים בהתאם למידע שבמסד הנתונים?

בדקתי בכך שהכנסתי למסד הנתונים משתמש ישירות ובדקתי אם כשניסיתי להתחבר אליו הוא קרא את המידע שצורה נכונה וברורה.

בכל פעולה שרשמתי מול המסד נתונים ביצעתי בדיקה שאכן המידע עובד כמו שצריך וגם הפעולה כנדרש.

בדיקה 4: בדיקת הצגת המידע במסכי המשתמשים – לוודא שכל המידע הרלוונטי ורק המידע הרלוונטי מוצג בממשק המשתמש הגרפי – בתצוגת הצ'אט, בתצוגת השפות, ככל המסכים ועוד. כדי לבדוק זאת השוואתי בין המידע המופיע על המסך לבית המידע שנשלח או שנמצא במסד נתונים.

בדיקה 5: בדיקת מקרי התנתקות של המשתמש מהשרת וההפך – לוודא שכאשר המשתמש מתנתק מהשרת בזמן שיחה זה מעיף אותו מהנתונים המתאימים וגם מודיע למשתמש אחר שהמשתמש שאיתו דיבר התנתק ולעדכן את מסד הנתונים אם משתמש מתנתק בזמן ההרשמה. בדקתי זאת בכך שהדפסתי וראיתי את הנתונים וגם שהמשתמש נמחק מהמסד נתונים אם המשתמש התנתק באמצע ההרשמה. לבדיקה אם השרת נסגר או קרס, הוספתי שכל 5 שניות הלקוח שולח הודעה ping לשרת ובודק אם החיבור עדיין, אם כן אז יופי, אם לא אז הוא שולח הודעה לשרת ומנתק אותו.

תכנון זמנים

משימה	פירוט המשימה	זמן מתכונן	זמן בפועל
בחירת נושא פרויקט ותכנון תצורת הפרויקט	בחירת הנושא בוצעה לאחר חשיבה ובדיקת כל תנאי הפרויקט הנדרשים. התכנון כלל חשיבה על תוצר שמתאים ומטרותיו, ותכנון כל המסכים ופונקציות חשובות בפרויקט.	שבועיים	שבוע וחצי
ממש משתמש גרפי customtkinter	למידת תכונות, ויכולות של הממשק ופיתור הממשק ובדיקות לעבודתו	שבוע - שבועיים	ביצעתי את משימה זו לכל אורך הפרויקט ושיניתי את הממשק מספר פעמים
בניית מסד הנתונים	מסד הנתונים הוא מסוג SQLite וכולל מספר טבלאות.	2 – 5 ימים	4 ימים (אך עשיתי קצת שינויים במהלך עשיית הפרויקט בשביל להתאים אותו לקוד)
בניית הפעולות שעובדות מול מסד הנתונים	פנייה למסד הנתונים באמצעות שאליות וקוד/פרמטרים בשפת SQL	שבוע	שלב זה התבצע לאורך הפרויקט עם לא מעט שינויים לאורכו אך

בסופו של דבר כ-5 ימים.			
יומיים	שבוע וחצי	יצירת מנגנון Hashing להצפנת הסימאות של המשתמשים	הצפנה למסד הנתונים
3 שבועות	שבועיים	בניית מערכת של תקשורת מרובת משתמשים	יצירת שרת מרובה משתמשים שרץ ממחשבים שונים עם תהליכונים
5 ימים	שבוע	הצפנה א-סימטרית	הצפנת RSA בהודעות בין המשתמש לשרת
שבועיים וחצי	שבועיים	טיפול במקרי קצה, תיקון באגים ושגיאות. לוודא עמידה בכל הקריטריונים והוספת דרישות חסרות	ביצוע בדיקות סופיות
רוב הספר נעשה בסוף הפרויקט, ולקח כשבועיים	3 שבועות	כתיבת הספר פרויקט וההכנות לקראת כתיבתו	הכנת הספר פרויקט

ניהול סיכונים

הזמן המוגבל שקיים להכנת הפרוייקט העלה הרבה ספקות ולקיחת סיכונים של איזה חלק יותר להתמקד בו בזמן הקצוב שיש. התמודדתי בכך שתכננתי סוג של טבלה ב Jira כדי לסדר את כל המטלות והמשימות שחשוב שיהיה פרויקט וכל הדברים שאני עובד עליהם כרגע, ועבדתי על הדברים החשובים יותר ובסוף הוספתי קצת פיצ'רים ופונקציות נוספות בסוף. (ובנוסף גם נבחנתי לפסיכומטרי באפריל מה שהקשה עוד יותר על סידור וניהול הזמנים)

תיאור תחום הידע-פרק מילולי

פירוט היכולות בצד השרת

1. התחברות משתמש – Login

- מהות: לאמת את זהות המשתמש כשהוא מתחבר למערכת.
- יכולות נדרשות:
 - קליטת נתונים מהלקוח (שם משתמש וסיסמה)
 - אימות הנתונים מול מסד הנתונים
 - החזרת תגובה ללקוח אם התחברותו הצליחה או כשלה.
- אובייקטים נחוצים:
 - מסד נתונים, תקשורת, הצפנה בתקשורת (RSA), הצפנה למסד הנתונים (hashlib)

2. הרשמה למערכת – Signup

- מהות: ליצור חשבון חדש שיוכל להשתמש במערכת
- יכולות נדרשות:
 - קליטת נתונים מהלקוח (שם משתמש, סיסמה ושפות)
 - הוספת המשתמש למסד הנתונים אם משתמש חדש
 - החזרת תגובה ללקוח אם התחברותו הצליחה או כשלה.
- אובייקטים נחוצים:
 - מסד נתונים, תקשורת, הצפנה בתקשורת (RSA), הצפנה למסד הנתונים (hashlib)

3. קביעת ושינוי שפות

- מהות: יצירת אפשרות סינון בחיבור המשתמשים לשיחות
- פעולות נדרשות:
 - קליטת שפות מהמשתמש
 - הוספה/שינוי השפות במסד הנתונים
 - החזרת תשובת ללקוח אם פעולתו הצליחה
- אובייקטים נחוצים:
 - מסד נתונים, תקשורת, הצפנה בתקשורת (RSA), הצפנה למסד הנתונים (hashlib)

4. טיפול במשתמשים ממתינים

- מהות: צירוף של זוגות משתמשים לשיחה אחד על אחד
- פעולות נדרשות:
 - בדיקת שפות משותפות בין משתמשים
 - סינון על ידי השיחה אחרונה
 - בדיקה של המשתמשים הממתינים

- החזרת תשובה ללקוח שסיים לחכות
- אובייקטים נחוצים:
- מסד נתונים, תקשורת, הצפנה בתקשורת (RSA)

5. ניהול שיחות בין זוגות משתמשים

- מהות: העברת הודעות בין זוג משתמשים בשיחה
- פעולות נדרשות:
- זיהוי והבדלה בין כל זוג משתמשים בשיחה
- קבלת הודעות מהשולח והעברתן ליעד
- אובייקטים נחוצים:
- תקשורת, הצפנה בתקשורת (RSA)
- 6. טיפול בהתנתקויות והחלפות בשיחות
- מהות: ניהול החלפות בין זוגות לאפשרות של שיחות מהירות ורנדומליות
- פעולות נדרשות:
- שליחת הודעת התנתקות למשתמש שעזבו ממנו את השיחה
- העברת המשתמש שהתנתק למקום בחירתו (מסך הבית או החלפת משתמש לשיחה והחזרתו למסך המתנה)
- טיפול במסד הנתונים אם המשתמש התנתק במהלך ההרשמה
- אובייקטים נחוצים:
- מסד נתונים, תקשורת, הצפנה בתקשורת (RSA), הצפנה למסד הנתונים (hashlib)

7. טיפול בבקשות של Admin

- מהות: העברת מידע שאין למשתמש רגיל לאדמין שהוא יוכל לראות
- פעולות נדרשות:
- קליטת הודעה מהמשתמש
- העברת מידע נדרש לפי בקשת האדמין (משתמשים מחוברים, שפות של משתמשים מחוברים, ומשתמשים בשיחה)
- גישה למסד הנתונים לבדיקת ועדכון המידע להעברה
- אובייקטים נחוצים:
- מסד נתונים, תקשורת, הצפנה בתקשורת (RSA)

פירוט היכולות בצד הלקוח

1. התחברות משתמש – Login

- מהות: התחברות של משתמש קיים למערכת
- פעולות נדרשות:
 - מסך התחברות – GUI – ממשק משתמש
 - קליטת נתוני התחברות
 - שליחה פרטי ההתחברות לשרת
 - קבלת תשובה מהשרת ופענוח
 - הצגת תשובה למשתמש
- אובייקטים נחוצים:
 - ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

2. הרשמה למערכת – Signup

- מהות: יצירת משתמש חדש לשימוש במערכת
- פעולות נדרשות:
 - מסך הרשמה – GUI – ממשק משתמש
 - קליטת נתוני ההרשמה
 - שליחה פרטי ההרשמה לשרת
 - קבלת תשובה מהשרת ופענוח
 - הצגת תשובה למשתמש
- אובייקטים נחוצים:
 - ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

3. עדכון וקביעת שפות

- מהות: שינוי השפות שהמשתמש מעוניין לדבר בהן
- פעולות נדרשות:
 - מסך הרשמה – GUI – ממשק משתמש
 - קליטת השפות שנבחרו
 - שליחת השפות שנבחרו לשרת
 - קבלת תשובה מהשרת ופענוח
 - הצגת תשובה למשתמש
- אובייקטים נחוצים:
 - ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

4. שליחת הודעה בשיחה (צ'אט)

- מהות: התכתבות בשיחה אחד על אחד עם משתמש אחר
- פעולות נדרשות:
 - מסך צ'אט עם הודעות שכל אחד מהמשתמשים שלחו בשיחה
 - קבלת הודעות מהמשתמש האחר בשיחה דרך השרת
 - שליחת הודעות למשתמש האחר בשיחה דרך השרת
 - שליחת בקשת התחברות לשיחה לשרת ולחכות לתשובה
 - השרת מוסיף את המשתמש למילון הצ'אטים הפעילים וגם את המשתמש שהוא מדבר עימו
- אובייקטים נחוצים:
 - ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

5. החלפת משתמש שיחה למשתמש אחר רנדומלי

- מהות: מעבר של משתמש לשיחה, עם משתמש אחר
- פעולות נדרשות:
 - שליחת הודעה לשרת להחליף משתמש
 - קבלת תשובה מהשרת ופענוח
 - מעבר של מסך הממשק גרפי למסך ההמתנה ולאחר מכן מעבר לשיחה החדשה
 - קבלת הודעה מהשרת לכך שהמשתמש האחר התנתק (אם התנתקו ממני)
- אובייקטים נחוצים:
 - ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

6. עזיבת שיחה עם משתמש

- מהות: עזיבת השיחה להפסקה או שינוי שפות
- פעולות נדרשות:
 - שליחת הודעה לשרת לצאת מהשיחה
 - קבלת תשובה מהשרת ופענוח
 - מעבר של מסך הממשק גרפי למסך הבית, בו יש אפשרות לשנות שפה או לחזור לחפש שיחה
 - קבלת הודעה מהשרת לכך שהמשתמש האחר התנתק (אם התנתקו ממני)
- אובייקטים נחוצים:
 - ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

7. מידע על משתמשים מחוברים – Admin

- מהות: צפייה ב־id של כל המשתמשים המחוברים
- פעולות נדרשות:
 - שליחת בקשה של לצפות במשתתפים המחוברים לשרת

- קבלת תשובה, מידע ופענוח
- שינוי מסך בממשק הגרפי למסך שמראה את המידע
- אובייקטים נחוצים:
- ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

8. מידע על משתמשים בשיחה - Admin

- מהות: צפייה ב-id של כל המשתמשים שנמצאים כרגע בשיחה ומי נמצא באיזו שיחה
- פעולות נדרשות:
- שליחת בקשה של לצפות במשתתפים שנמצאים כרגע בשיחה
- קבלת תשובה, מידע ופענוח
- שינוי מסך בממשק הגרפי למסך שמראה את המידע
- אובייקטים נחוצים:
- ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

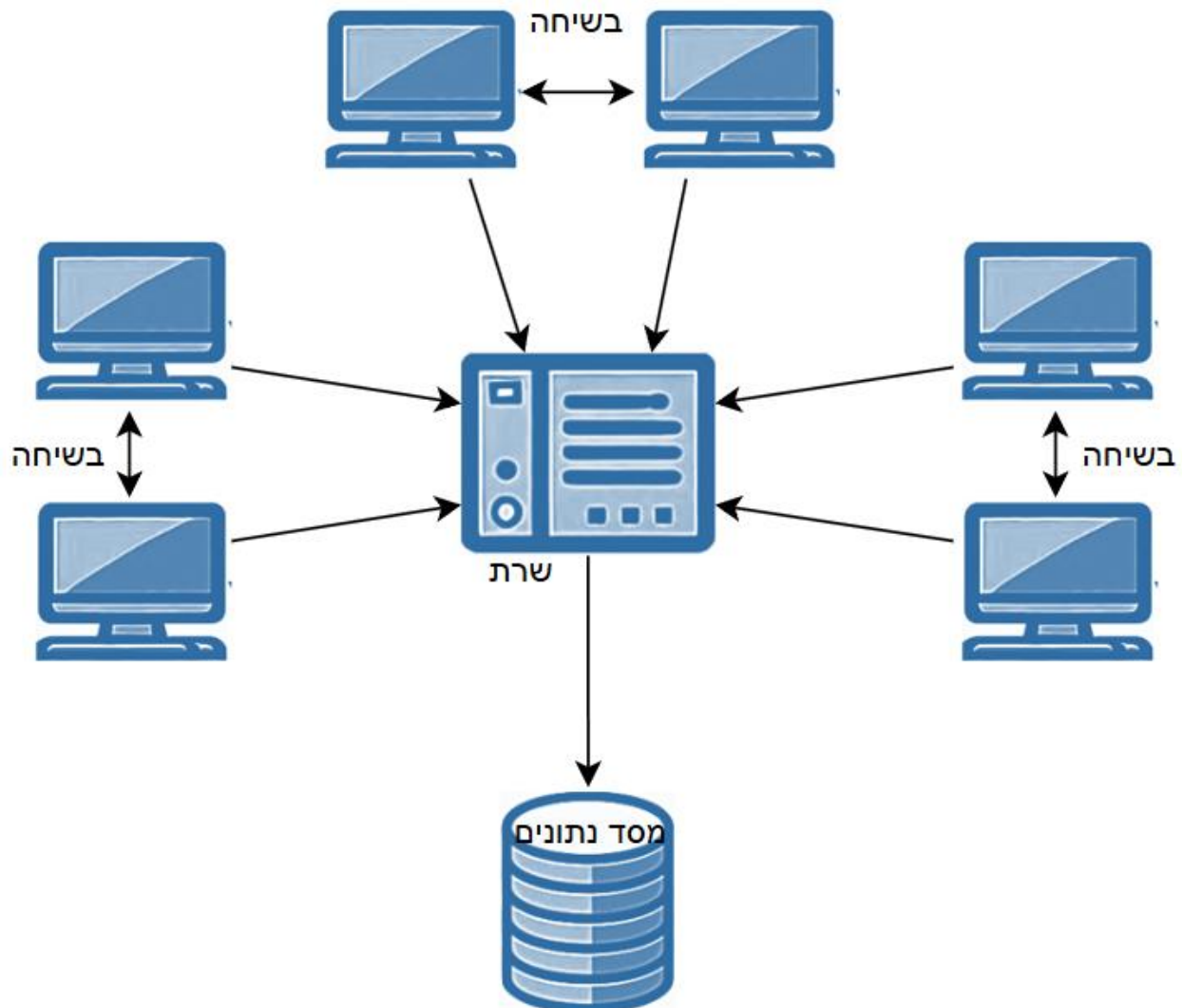
9. מידע על שפות של המשתמשים המחוברים – Admin

- מהות: צפייה בכל שפה ובמספר האנשים המחוברים שיודעים לדבר אותה
- פעולות נדרשות:
- שליחת בקשה לשרת
- קבלת תשובה, מידע ופענוח
- שינוי מסך בממשק הגרפי למסך שמראה את המידע
- אובייקטים נחוצים:
- ממשק משתמש, תקשורת, הצפנה בתקשורת (RSA)

מבנה/ארכיטקטורה של הפרויקט

תיאור הארכיטקטורה של המערכת המוצעת ותיאור החומרה

הפרויקט RandChat מבוסס על תקשורת מרובת משתמשים שפועלת על ספריית socket בשפת Python, כך שהשרת מופעל על ידי מנהל המערכת (שרק לו יש גישה למשתמש Admin) ומשתמשי הקצה הם הלקוחות של המערכת. השרת מבצע ועונה על פעולות לפי בקשת הלקוח. לשם הרצת המערכת צריך לפחות 2 מחשבים, מחשב אחד שיריצ את השרת, יאזין למשתמשים שמתחברים, ומחשב נוסף לכל משתמש שרוצה להשתמש במערכת. בנוסף בפרויקט זה יש שימוש במסד נתונים שמופעל בעזרת הספרייה sqlite3, לכן המחשב שמפעיל את השרת יכול גם את מסד הנתונים שנוצר ומשתנה.

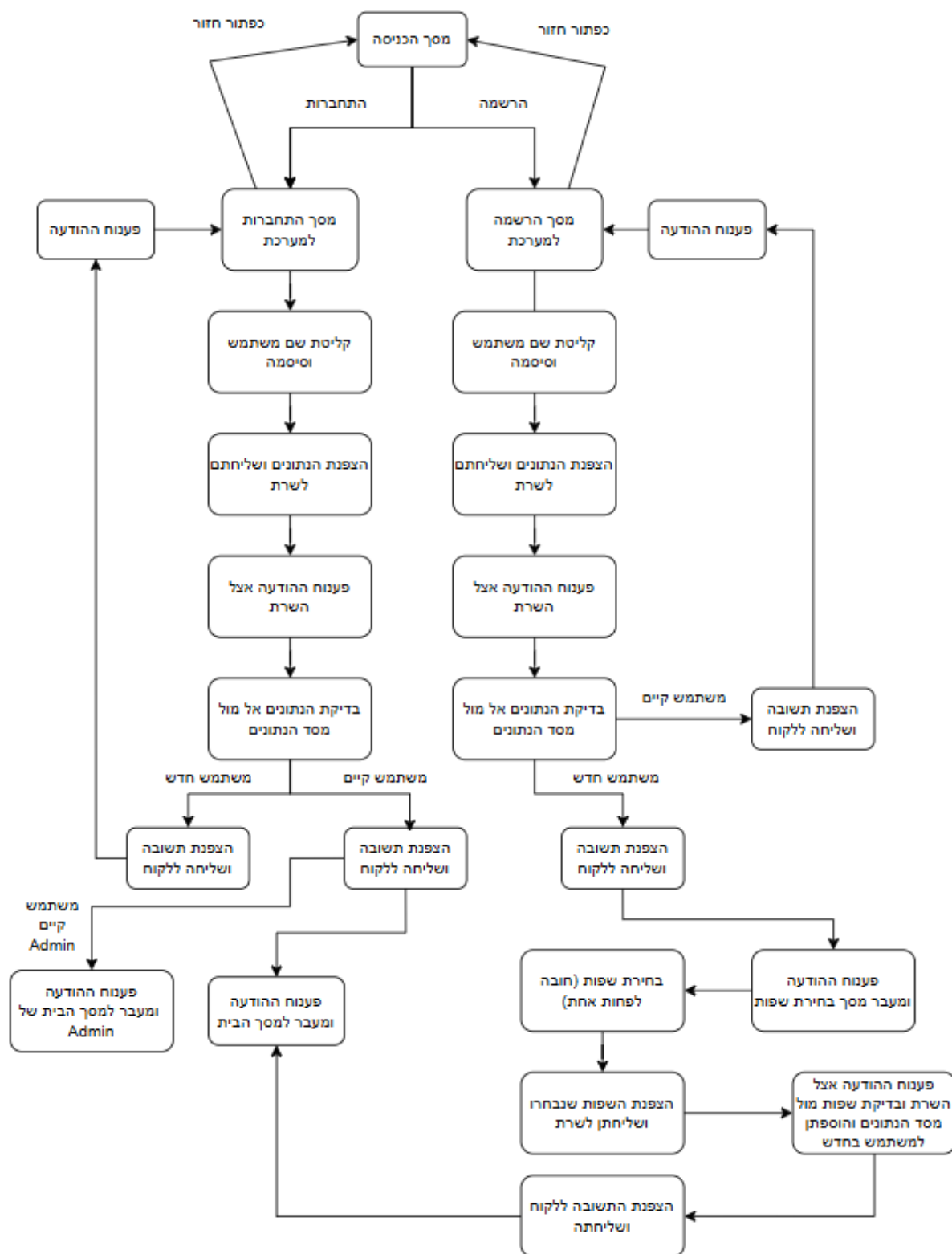


תיאור הטכנולוגיה הרלוונטית

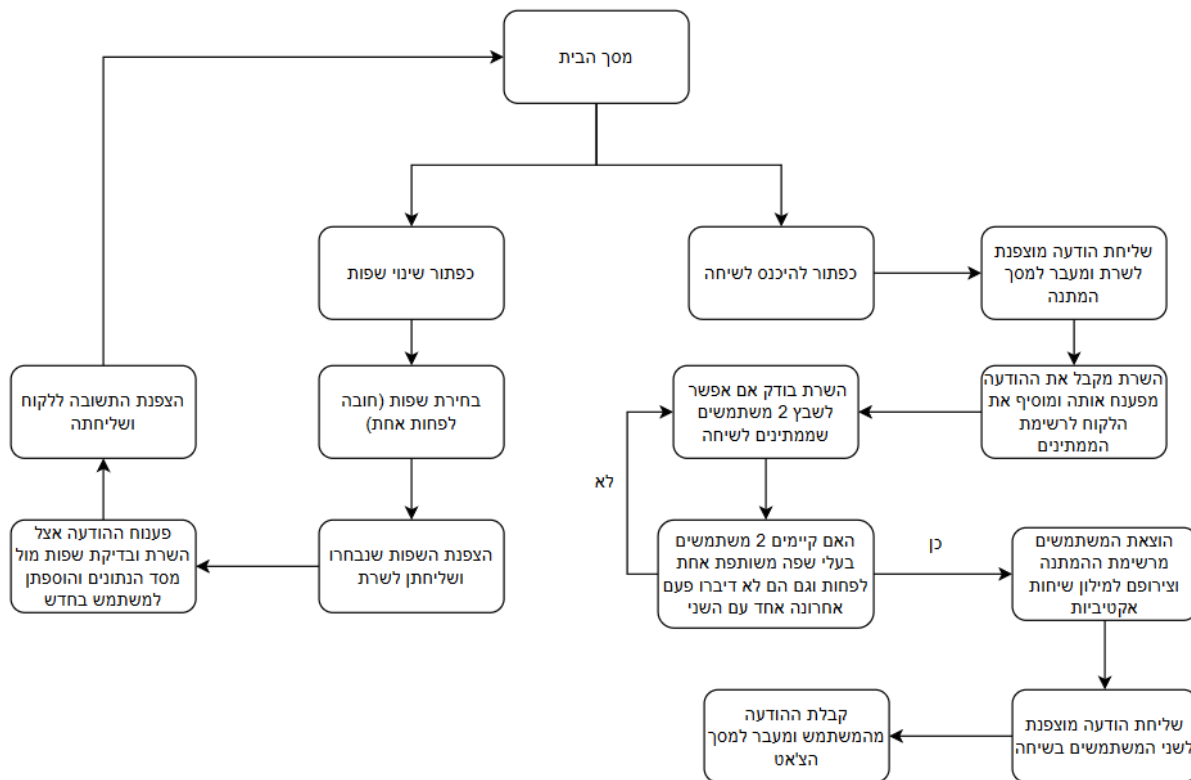
רוב הקוד נכתב בשפת תכנות Python. בנוסף נעשה גם שימוש ב-SQL לצורך שליפה ועדכון נתונים ומידע במסד הנתונים הפרויקט נועד להרצה על מערכת הפעלה מסוג Windows והוא מכיל אלמנטים מתחומי: רשתות, אבטחת מידע, הגנת סייבר, מחלקות, threading, ו-gui.

תרשימי זרימה

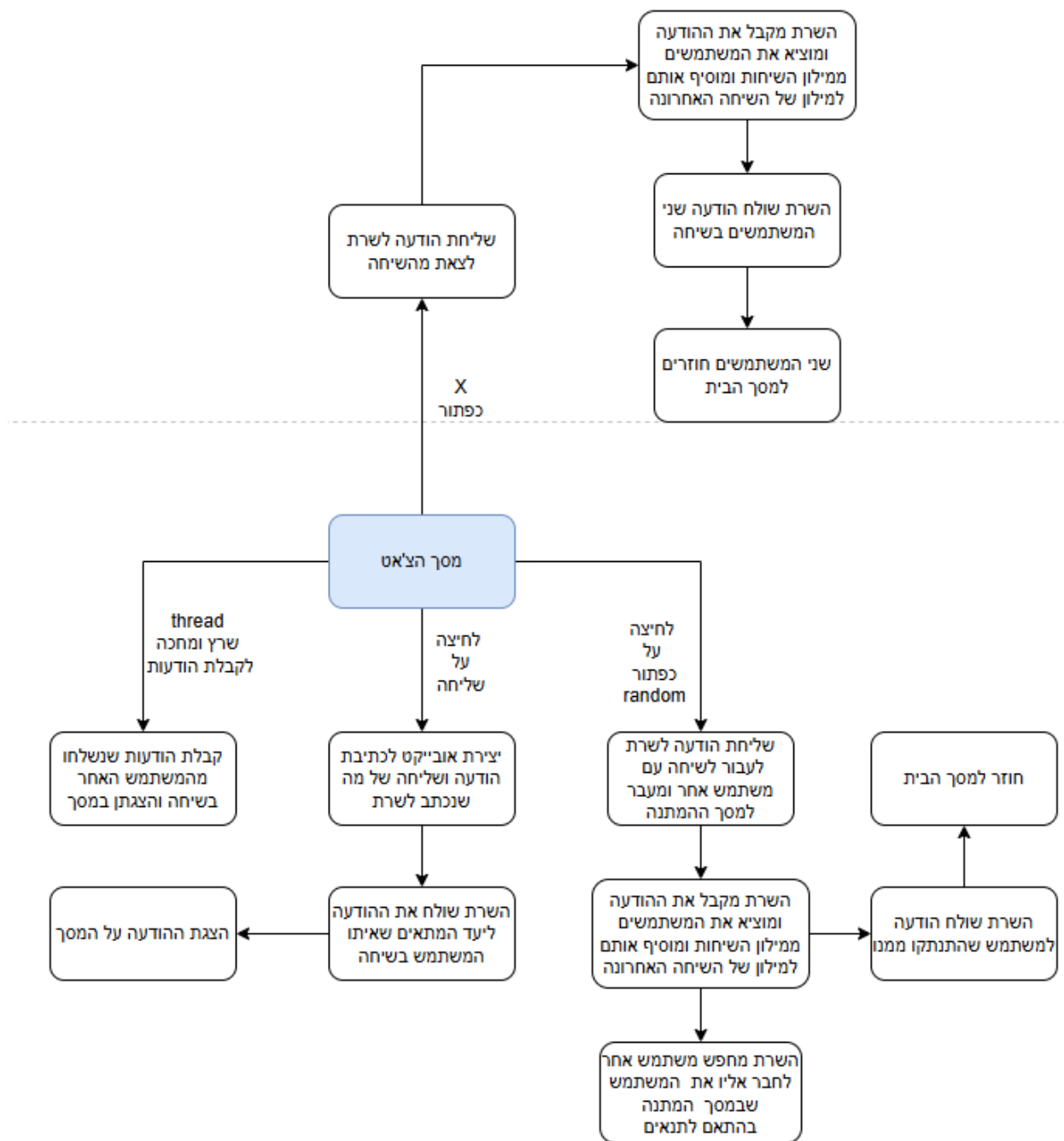
הרשמה והתחברות למערכת



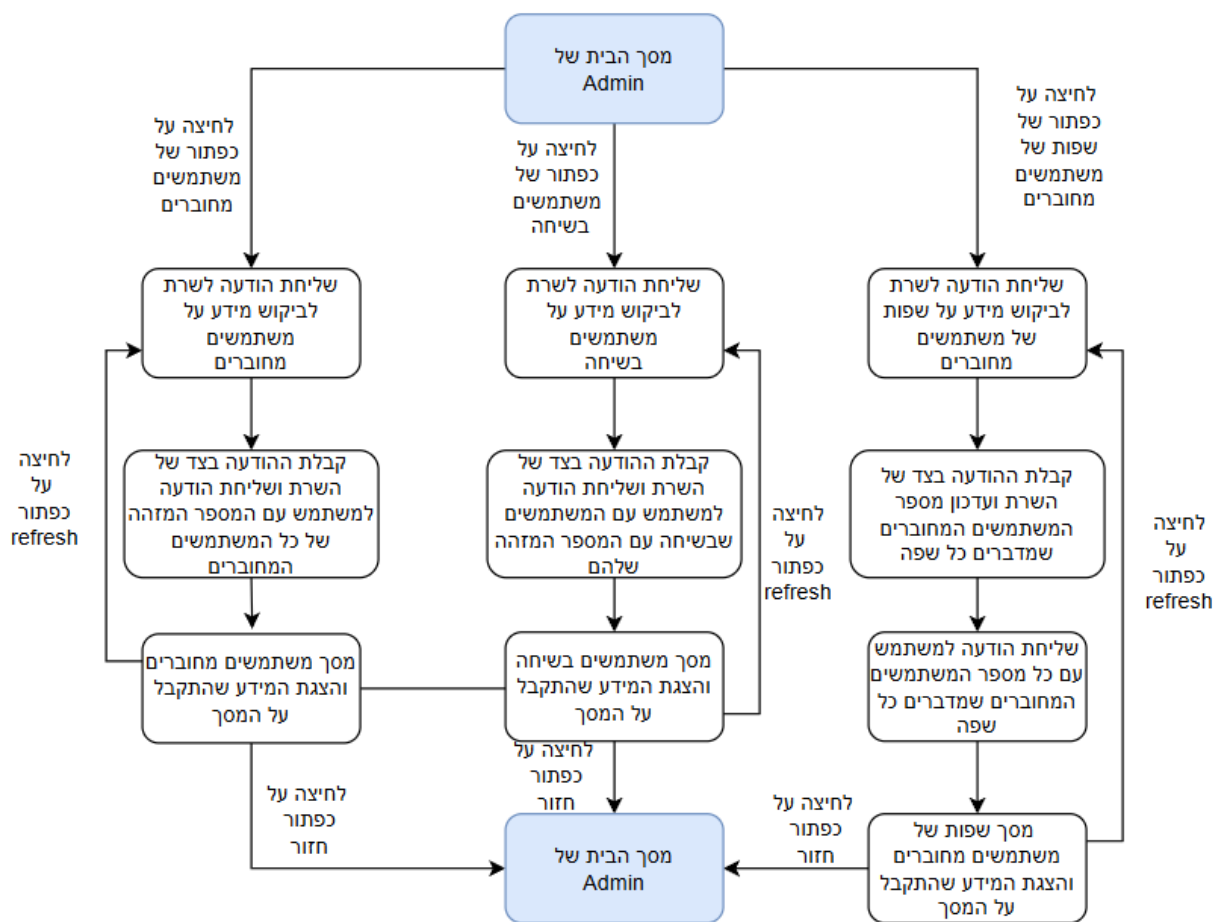
מסך הבית, שינוי שפות ומעבר לשיחה – משתמש רגיל



צ'אט בין זוג לקוחות



מסך בית של Admin



תיאור האלגוריתמים מרכזיים בפרויקט

התחברות והרשמה למערכת

ללקוח יש צורך להיכנס למשתמש שלו, אך קיימת בעיה, ללקוח יש את הממש הגרפי בו הוא רושם את שם המשתמש והסיסמה אך הצד של הלקוח אינו יכול לשמור את נתונים אלו מפני שיש צורך לשמור על סודיות וגם הצד של השרת אינו יכול לשמור על נתונים אילו מאותה סיבה. לכן כדי למנוע פריצה למשתמש השתמשתי ב2 פתרונות

1. הסגנון הצפנה מסד הנתונים – hashing

שהוא אלגוריתם הצפנה שלוקח עצם וממיר אותו למידע מוצפן שמאוד מקשה לפענוח. ומאפשר רק השוואה של עצמים שכבר הוצפנו כדי לראות אם שווים. אלגוריתם זה מצוין לשמירת סיסמאות – ניתן לשמור סיסמה של משתמש במסד הנתונים לאחר הצפנה זו והסיסמה תישאר סודית וקשה לפענוח וכך מפחיתה משמעותית את הסיכוי לפריצה. וכך כשמשתמש בה להירשם יש לבדוק אם שם המשתמש והסיסמה כבר נמצאים במסד הנתונים, ואם הוא אינו קיים ניתן להוסיף אותו בעזרת ההצפנה ולשמור את הסיסמה המוצפנת ביחד עם שם המשתמש במסד הנתונים. וכאשר משתמש רוצה להתחבר זה משווה בין הנתונים שכבר במסד הנתונים לנתונים שנקלטו מהלקוח לאחר שמצפינים את הסיסמה שהלקוח נתן ומשווים עם מסד הנתונים.

2. מתן מספר מזהה לכל משתמש שגם הוא יוכל לשמור

כדי לאפשר לשרת כל פעם לזהות עם איזה משתמש הוא מדבר ולאפשר גישה יותר פשוט למסד הנתונים, לאחר ההתחברות או ההרשמה השרת נותן ללקוח את המספר המזהה שלו לאחר שגם השרת שומר אותו במילון של כל המשתמשים המחוברים כרגע. וכך עוזר לדוגמה לשינוי השפות לאחר שהמשתמש התחבר לזיהוי המשתמש.

ניהול שיחה בין 2 לקוחות

ניסוח וניתוח הבעיה האלגוריתמית:

כששני משתמשים נמצאים בשיחה ומתקשרים אחד עם השני קיימת אופציה בה אחד המשתמשים רושם ומדבר כל הזמן והאחר אינו רושם כלום (מבחירה), או ששניהם מדברים בו זמנים או שאחד מהם רוצה להתנתק או להחליף שיחה למישהו אחר. לכן השתמשתי בmulti-threading ל2 פונקציות, אחת שעוברת על כל הפעולות הלקוח רוצה לעשות (ועל התשובות שלהן) והאחרת (שעובדת רק כשמתקיימת שיחה) בודקת אם נשלחה הודעה מהשרת של ההודעה של המשתמש האחר בשיחה. אך נתקלתי בבעיה כאשר המשתמש שהתנתק (רוצה להחליף לשיחה אחרת או חזר למסך הבית) שלח את ההודעה שלו לשרת וקיבל הודעה חזרה מהשרת (תשובה לפעולה שלו) threadn שעובר על אם נשלחה הודעה להציג, לקח את התשובה המקום threadn (מפני שrecv של socket זה חוסם) האחר ולכן חלק מההודעות מהשרת ללקוח לא הגיעו למקומן המתאים.

תיאור אלגוריתמים קיימים לפתרון הבעיה:

1. שימוש בflag שבעצם אמור להגדיר מתי הלקוח בשיחה כך אפשר לעצור את פעילות threadn

שבדק אם קיבלו הודעה להציג למסך

2. יצירות אפשרות לקבל הודעות ולעשות פעולות גם בthread שמקבל ובודק אם להציג הודעה על המסך
3. להשתמש בselect כך שיהיה יותר בררני לגבי ההודעות שהוא לוקח וכך גם לא יחסום את המשך הפעולות והתקשורת
4. שימוש בlock של threading

סקירת הפתרון הנבחר:

בסופו של דבר ניסיתי כל מיני פתרונות גם עם flag ונמנעתי מלנסות לשנות את הפונקציה שתוכל גם לפעול לפי הודעות של הלקוח כדי למנוע חיכוך ובעיה בין threading ובנוסף ניסיתי עם lock וזה רק יצר עוד בעיות. ולאחר התייעצות עם כמה מן המנחים החלטתי להשתמש גם בflag וגם בselect מכיוון שflag לבד לא עזר לפתור את הבעיה מפני שהפונקציה recv היא חוסמת וכך מונעת מן המקומות המתאימים לקבל את התשובה מן השרת והשימוש בselect פתר את הבעיה ונתן להודעות להגיע למקומן המתאים ללא הפרעה בין שני threads.

שיבוץ הלקוחות לשיחות של אחד על אחד

במערכת שלי, לקוחות יכולים לדבר אחד עם השני (דרך השרת) בשיחה של אחד על אחד בצורה אוניברסלית, בשביל ליצור אלגוריתם כזה במערכת שמרתי על כך שהלקוחות אינם יודעים ואינם שומרים את הסיסמה או שם המשתמש שלהם בנוסף לכך שהם אינם בוחרים עם מי הם רוצים לדבר. בנוסף רציתי שבאלגוריתם לא תהיה אפשרות של שני משתמשים לדבר אחד עם השני בשיחה הבאה ורציתי להוסיף סינון עם שפות, שרק משתמשים שיש להם לפחות שפה אחת משותפת יוכלו לדבר.

תיאור אלגוריתמים אפשריים לפתרון הבעיה

1. השרת יוכל לשבץ כל זוג משתמשים לשיחה אפשר להשתמש ברשימה של טאפלים
2. השרת יכול ליצור מילון של מחוברים ומעוניינים בשיחה
3. לשבץ את הראשונים שפנויים דרך רשימה של משתמשים מחכים ולשמור כל אחד מהם ברשימה

סקירת הפתרון הנבחר:

בשביל הפרויקט שלי בחרתי ליצור רשימה של משתמשים שמחכים להיכנס לשיחה, יצרתי מילון של משתמשים שכרגע בשיחה ומילון של עם מי כל משתמש דיבר בשיחה האחרונה. לא בחרתי ליצור רשימה של טאפלים מכיוון שידעתי איך לעשות זאת כבר בדרך האחרת ולפי שיקול הדעת ותכנון הזמנים לפרויקט היה עדיף לעשות בדרך הידועה. באלגוריתם השרת עובר על הרשימה של האנשים שמעוניינים להיכנס לשיחה ובודק אם הם דיברו עבור כל זוג משתמשים הם אם דיברו לאחרונה ואם יש להם שפות משותפות אם הם לא דיברו ואין להם שפות משותפות אז הוא לא משבץ אותם ביחד ועובר הלאה למשתמש הבא.

העברת מידע בין השרת והלקוחות בצורה מאובטחת

במערכת שלי אני משתמש בsocket לכן העברת ההודעות חייבת להיות על ידי send.client, וזה מאפשר המון דרכים לשלוח הודעות. במהלך מעבר ההודעות בין המקור ליעד, המידע עלול

להגיע לגורמים זדוניים אשר מסניפים את הפאקטות ועלולים להשתמש במידע לרעה, כמו סיסמאות של משתמשים, לכן חשוב מאוד לקיים הצפנה בתקשורת. ויחסית מהר החלטתי להשתמש בתקשורת בין השרת ללקוח בהצפנת RSA. באלגוריתם שלי גם השרת וגם הלקוח יוצרים את ההודעה שהם רוצים לשלוח ורגע לפני השליחה אני מצפין את ההודעה עם המפתח הפרטי ושולח לאחר כדי שיפענח עם המפתח הציבורי של השני. לדוגמה: הלקוח יוצר הודעה ומצפין עם המפתח הפרטי שלו, ואז שלוח את ההודעה המוצפנת לשרת. לאחר מכן השרת מקבל את ההודעה ומפענח אותה לפי המפתח הציבורי של הלקוח וככה המידע מוצפן ללא יכולת להסגת המידע בדרך.

הסבר RSA:

בשביל הצפנת RSA צריך 2 מפתחות. פרטי וציבורי וקיימים שלבים ליצירתן. דבר ראשון מגדלים 2 מספרים ראשוניים p, q . ככל שהם יהיו יותר גדולים כך יהיה יותר קשה לפענח את ההצפנה. כדי למצוא 2 מספרים ראשוניים שרמתי קובץ המכיל כ-500000 המספרים הראשוניים והשתמש ב random בשביל להגדיל אותם. לאחר מציאת המספרים הראשוניים, נחשב את המודולו $n=p*q$. ונשתמש בפונקציית אויילר, פונקציה שלוקחת את n ומחשבת את הפונקציית אויילר שלו. פונקציית

$$\varphi(n) = (p-1)(q-1)$$

אויילר של שניהם היא:

**פונקציית אויילר מוצאת עבור n כמה מספרים זרים יש לו - שהמחלק המשותף המרבי שלהם עם n הוא אחד.

יצירת המפתח הציבורי – את המפתח הציבורי נסמן ב- e . הוא מספר ראשוני בין 3 לתוצאת פונקציית אויילר שחישבנו, לאחר שבחרנו את המפתח הציבורי e נבדוק האם המחלק המשותף המקסימלי שלו עם n של פונקציית אויילר הוא 1 בדיקה זו למעשה מספקת לנו מידע האם המספרים הם ראשוניים ביחס אחד לשני, אם כן נמשיך הלאה, ובמידה ולא יחושב e חדש. את המפתח הציבורי נרשום בצורה הבאה כאשר n הוא תוצאת פונקציית אויילר, ו- e הוא המפתח הציבורי: (e, n) .

יצירת מפתח פרטי - את המפתח הפרטי נסמן ב- d , משום שאנו עובדים עם מספרים ראשוניים נוכל לדעת כי אם d שונה מ- e וראשוני לו, נוכל להשתמש אלגוריתם של אולידיס ולמצוא את d . (d, n)

$$m^{e(mod n)} = c$$

להצפנה:

$$c^{d(mod n)} = m$$

לפענוח:

תיאור סביבת הפיתוח

פירוט כלי הפיתוח הדרושים לפיתוח:

1. שפת תכנות python
2. שימוש ב(DB Browser (sqlite לצורך בדיקת מסד הנתונים ולצפייה במידע שנמצא במסד הנתונים
3. הספריות:
 - Socket
 - Select
 - Sqlite3
 - Threading
 - Customtkinter
 - Json
 - Pillow (PIL)
 - os
4. סביבת עבודה: PyCharm Community Edition

כלים נדרשים לבדיקות:

1. שימוש ב(DB Browser (sqlite לצורך בדיקת מסד הנתונים ולצפייה במידע שנמצא במסד הנתונים.
2. שורת הפקודה CMD - הזנת הפקודה ipconfig לצורך מציאת כתובת ה IP של המחשב שממנו רץ השרת על מנת לאפשר לכמה מכשירים שונים במקביל לרוץ.

תיאור פרוטוקול התקשורת

תיאור מילולי

פרוטוקול התקשורת שאני משתמש בו הוא פרוטוקול TCP (Transmission Control Protocol). פרוטוקול זה נמצא בשכבת התעבורה (השכבה הרביעית), והוא מבטיח העברה אמינה ובטוחה של מידע ונתונים בין 2 תחנות ברשת מחשבים באמצעות יצירת חיבור מקושר. (Oriented Connection)

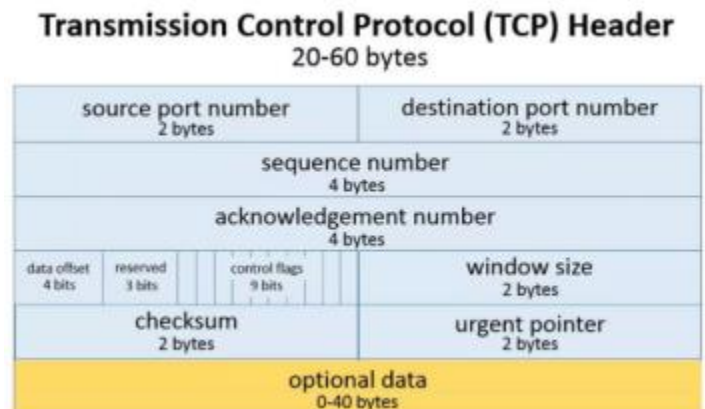
פרוטוקול TCP הוא למעשה צורת התקשרות שכאשר מופעלת על ידי שתי תחנות היא מאפשרת, באמצעות נוספת כמות קטנה של מידע (header בגודל של 20 עד 60 בתים) לכל חבילה, העברת מידע אמינה וללא שגיאות בין שני הצדדים, גם כאשר התווך ביניהם רעוע.

פרוטוקול TCP מאפשר מעבר של נתונים דרך כתובות IP ושיוכן לאפליקציות דרך פורט שמשוך לאפליקציה זאת. בפרויקט שלי פרוטוקול זה נמצא בספרייה socket של python.

מבנה ה-Header של פרוטוקול TCP:

בפרוטוקול TCP שבו אני משתמש לתקשורת בין השרת ללקוח, מחייב את מקור ההודעה בתהליך שנקרא "לחיצת יד משולשת" ותהליך כולל שלושה שלבים:

1. המקור שולח הודעה סנכרון (SYN) כדי להודיע לשרת על רוצנו לפתיחת התקשורת.
2. במידה והשרת זמין ומחובר ויש ביכולתו לפתוח בשיחה, הוא ישלח אל המקור הודעה אישור סנכרון (ACK-SYN) ובכך יודיע למקור על זמינותו.
3. המקור מקבל את הודעת התשובה מהיעד ושולח לו הודעה אישור (ACK) וכעת שני הצדדים מחוברים בשיחה ויכולים לדבר ולהעביר נתונים.



תיאור כלל ההודעות הזורמות במערכת

פרוטוקול התקשורת שלי בהודעות בין השרת למשתמש מסודר כך שכל הודעה מורכבת מפעולה ומידע שרוצים להעביר ואם יש כמה חלקים במידע אז הוא מופרד בDATA_DELIMITER ובין הפעולה למידע יש הפרדה של DELIMITER. כל פעולה מאופיינת עם מחרוזת בעלת 4 ספרות. וכך יוצא שההודעות שמועברות בין השרת למשתמש ובין המשתמש לשרת הן מחרוזות המחילות את הפעולה והמידע שרוצים להעביר.

הערה: במקומות שרשום and, המידע הזה אינו עובר אלא יש שם DATA_DELIMITER ובמקום פסיק יש שם DELIMITER וזו ההודעה שעוברת.

שם ההודעה: התחברות לשרת

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: (ip, port)

שם ההודעה: החלפת מפתחות

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ("0009", public key)

שם ההודעה: החלפת מפתחות

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ("1015", public key)

שם ההודעה: התחברות למערכת

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["0000", username and password]

שם ההודעה: התחברות למערכת נכשלה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1001", ""]

שם ההודעה: התחברות למערכת הצליחה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1002", id and languages]

שם ההודעה: הרשמה למערכת

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["0004", username and password]

שם ההודעה: הרשמה למערכת נכשלה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1004", ""]

שם ההודעה: ההרשמה למערכת הצליחה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1003", id and languages]

שם ההודעה: בחירת שפות - משתמש רגיל

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["0005", id and selected_languages]

שם ההודעה: בחירת שפות נכשלה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1007", ""]

שם ההודעה: בחירת שפות הצליחה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1006", ""]

שם ההודעה: מחכה - משתמש רגיל

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["0007", ""]

שם ההודעה: תפסיק לחכות

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1012", ""]

שם ההודעה: לדבר - משתמש רגיל

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["0003", message]

שם ההודעה: העברת הודעה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1009", message]

שם ההודעה: להפסיק לדבר - משתמש רגיל

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["0008", ""]

שם ההודעה: להחליף שיחה - משתמש רגיל

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["0006", ""]

שם ההודעה: החלפת שיחה בהצלחה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1010", "putting you in waiting list"]

שם ההודעה: משתמש שהיה בשיחה עזב

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1014", "Your friend has left the chat."]]

שם הודעה: שגיאה (ERROR)

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["9999", "Unknown command"]

שם ההודעה: ping

נשלח מ: לקוח

אל: שרת

מבנה ההודעה: ["", "0010"]

שם ההודעה: לראות משתמשים מחוברים - משתמש Admin

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["", "5000"]

שם ההודעה: לראות משתמשים בשיחה - משתמש Admin

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["", "5001"]

שם ההודעה: לראות סכום המשתמשים שיודעים כל שפה - משתמש Admin

נשלח מ: לקוח

אל: שרת

מבנה השדות בהודעה: ["", "5002"]

שם ההודעה: לראות משתמשים מחוברים בהצלחה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["list of id's", "1017"]

שם ההודעה: לראות משתמשים שיחה בהצלחה

נשלח מ: שרת

אל: לקוח

מבנה השדות בהודעה: ["1018", "dictionary of id's"]

שם ההודעה: לראות סכום המשתמשים שיודעים כל שפה בהצלחה

נשלח מ: שרת

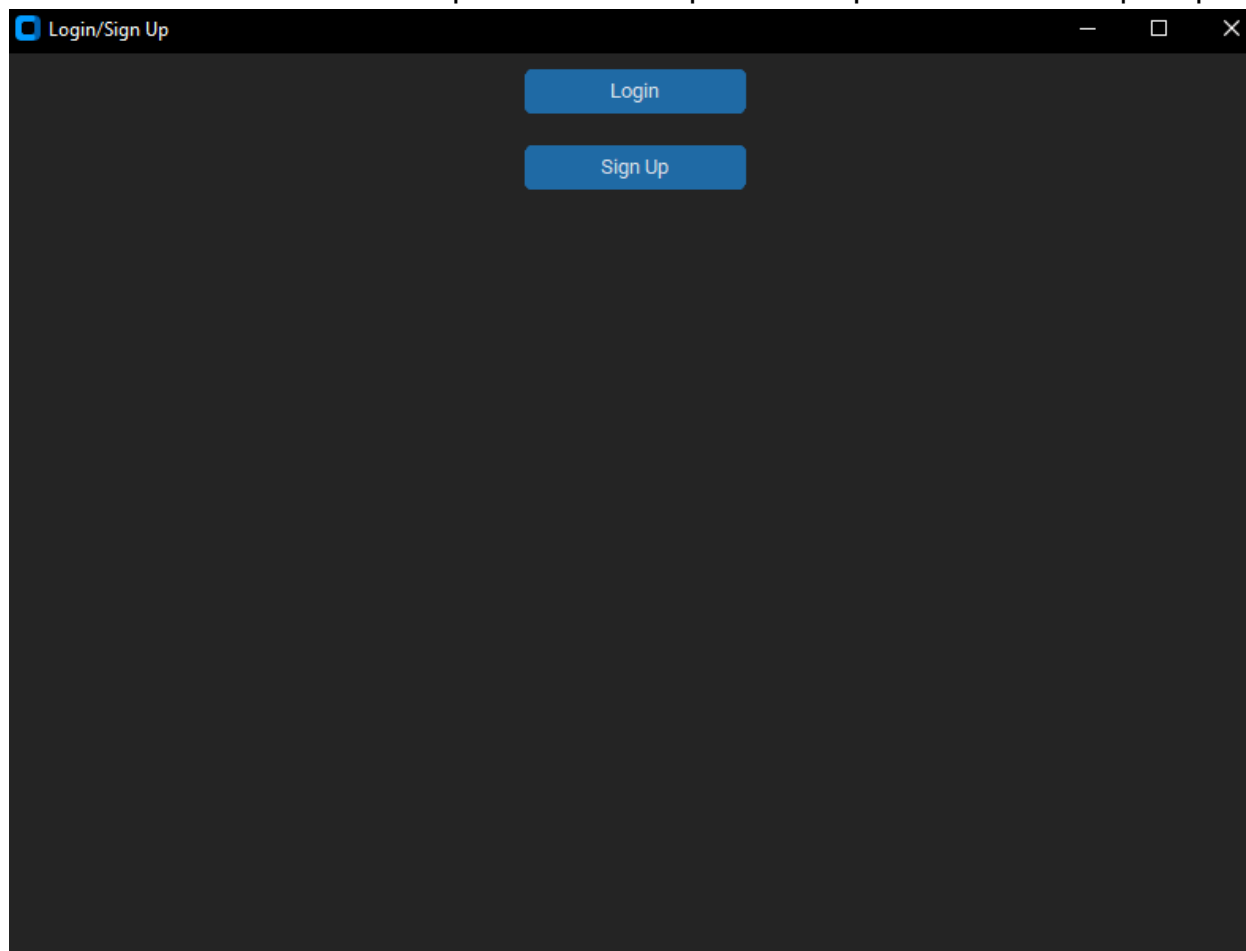
אל: לקוח

מבנה השדות בהודעה: ["1019", "dictionary(with languages and number of users)"]

תיאור מסכי המערכת

מסך הכניסה/פתיחה

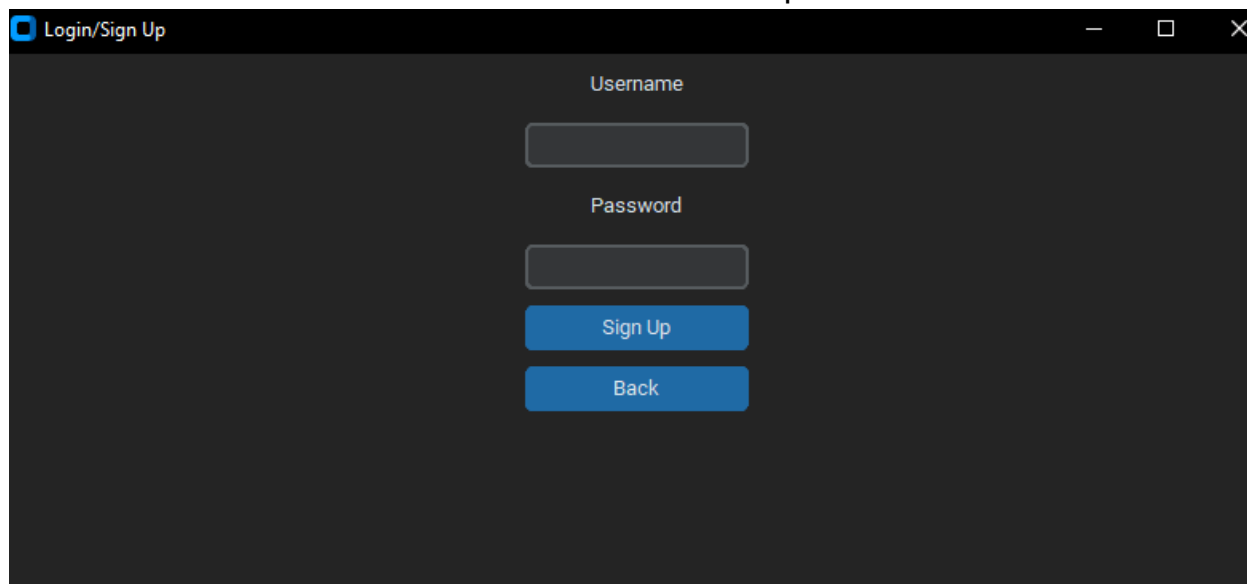
במסך זה קיימים 2 כפתורים וניתן להיכנס למסך ההרשמה או למסך ההתחברות בהתאם למטרה.



מסך ההרשמה

במסך זה קיימים 2 כפתורים וניתן להכניס שם משתמש וסיסמה ולהיכנס עם משתמש חדש למערכת וגם כפתור של חזור כדי לחזור למסך הכניסה, לאחר ההרשמה המשתמש יועבר למסך בו יצטרך לסמן

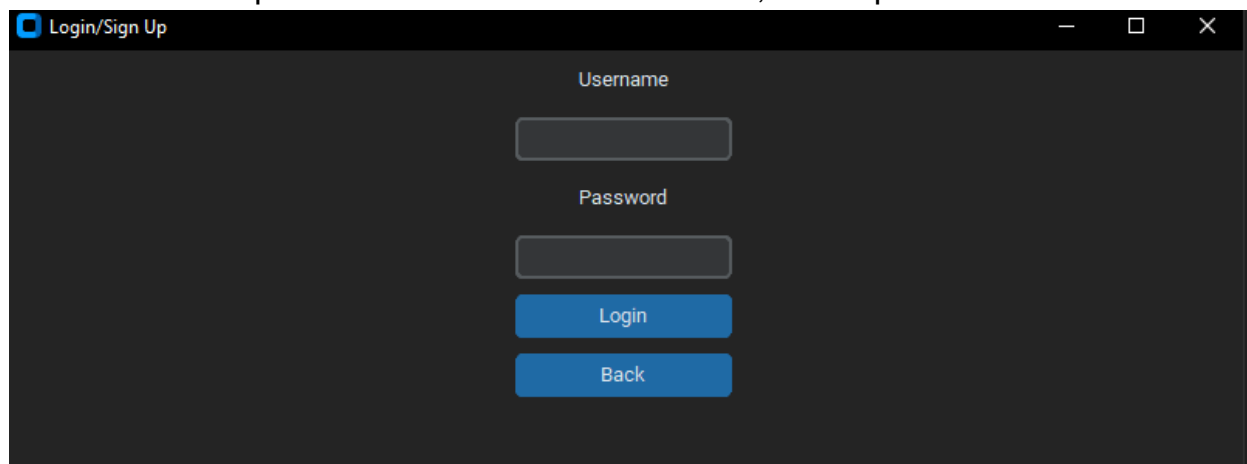
את השפות שהוא יודע/רוצה לדבר איתן.



The screenshot shows a dark-themed window titled "Login/Sign Up". It contains two input fields: "Username" and "Password". Below the "Password" field are two blue buttons: "Sign Up" and "Back".

מסך התחברות

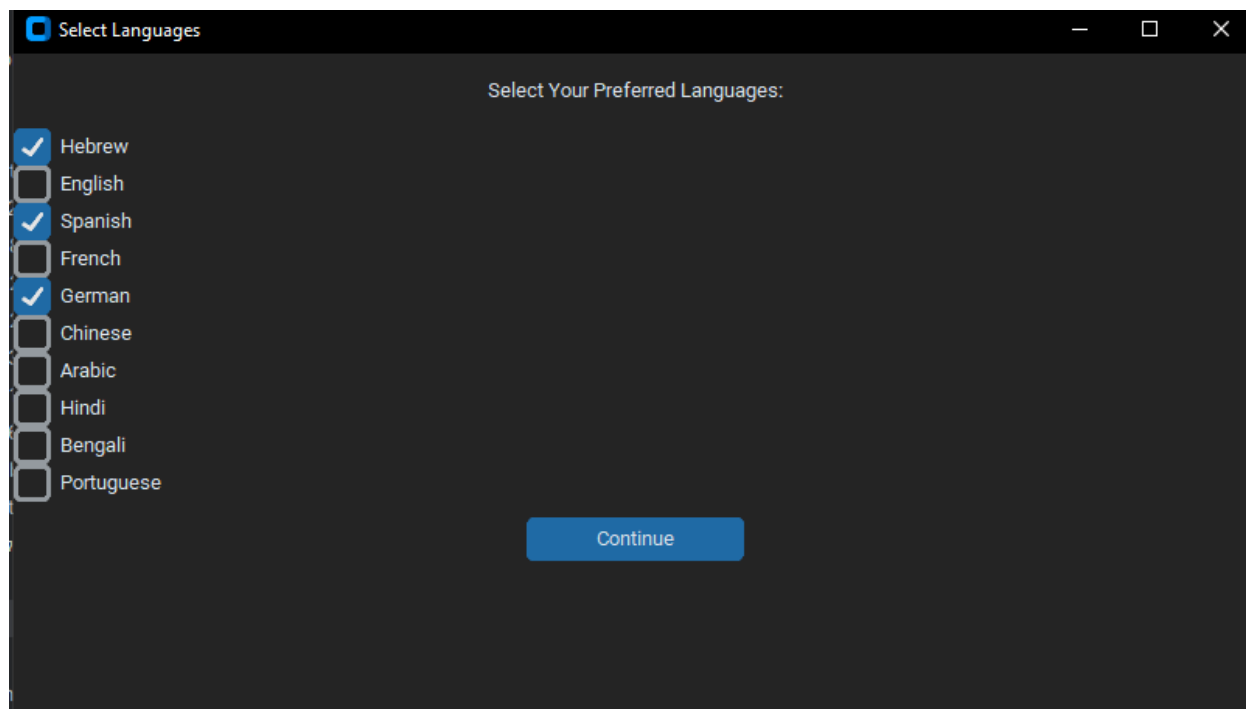
במסך זה קיימים 2 כפתורים וניתן להכניס שם משתמש וסיסמה ולהיכנס עם משתמש קיים למערכת וגם כפתור של חזור כדי לחזור למסך הכניסה, לאחר ההתחברות המשתמש יועבר למסך הבית.



The screenshot shows a dark-themed window titled "Login/Sign Up". It contains two input fields: "Username" and "Password". Below the "Password" field are two blue buttons: "Login" and "Back".

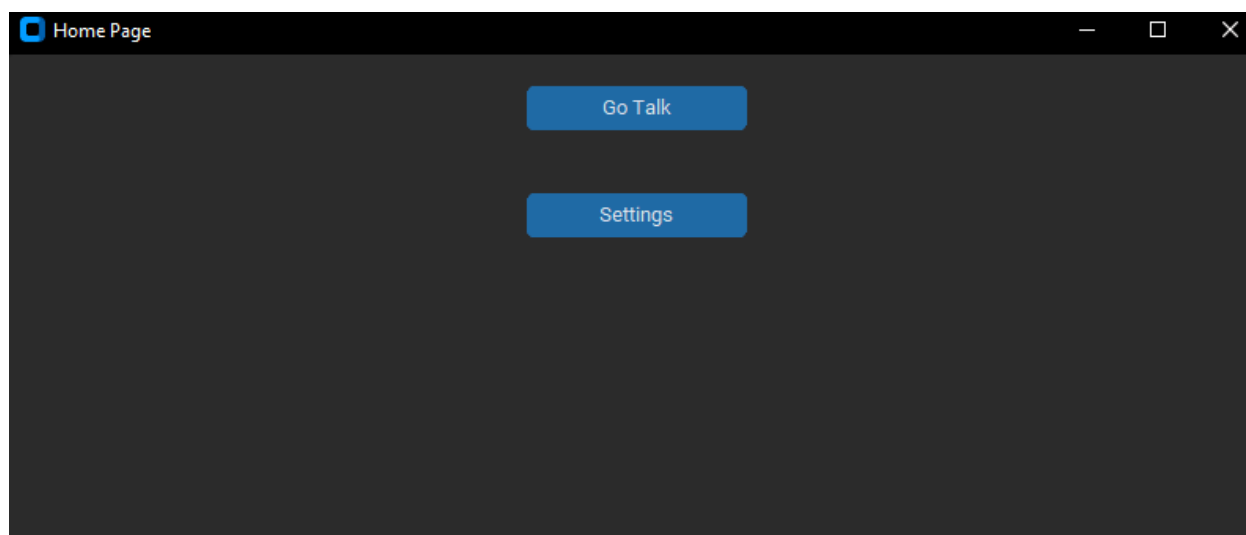
מסך בחירת שפות

במסך זה קיימים 10 תיבות שניתן למלא בהתאם לשפות שהמשתמש רוצה לדבר בהן, וכפתור להמשיך למסך הבית לאחר שמילאו לפחות שפה אחת.



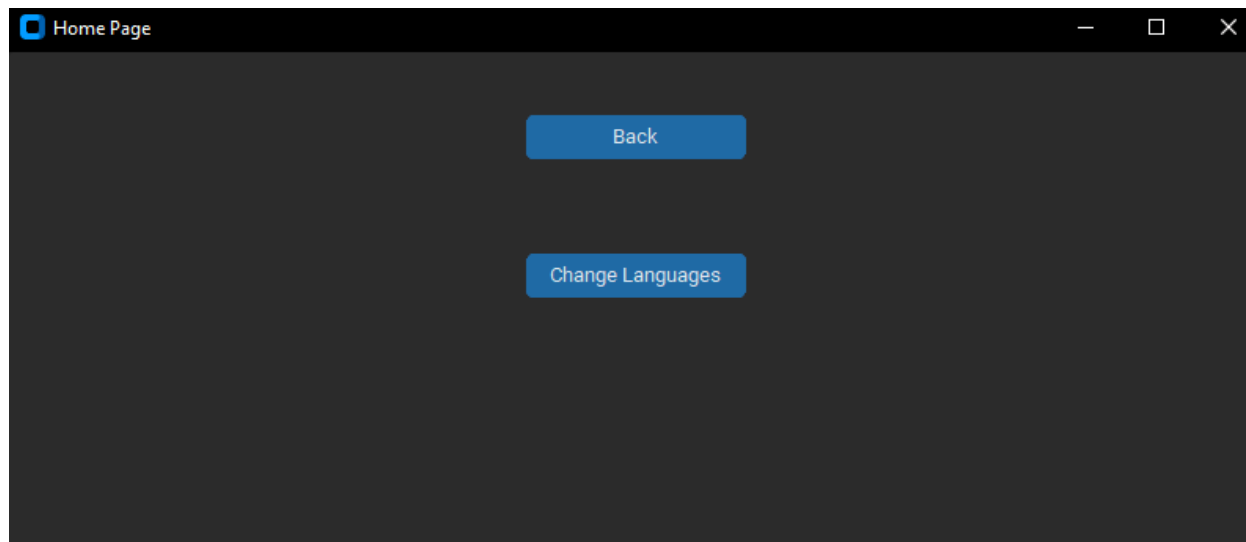
מסך הבית

במסך זה קיימים 2 כפתורים אחד מוביל למסך ההמתנה ולשיחה והאחר פותח את ההגדרות



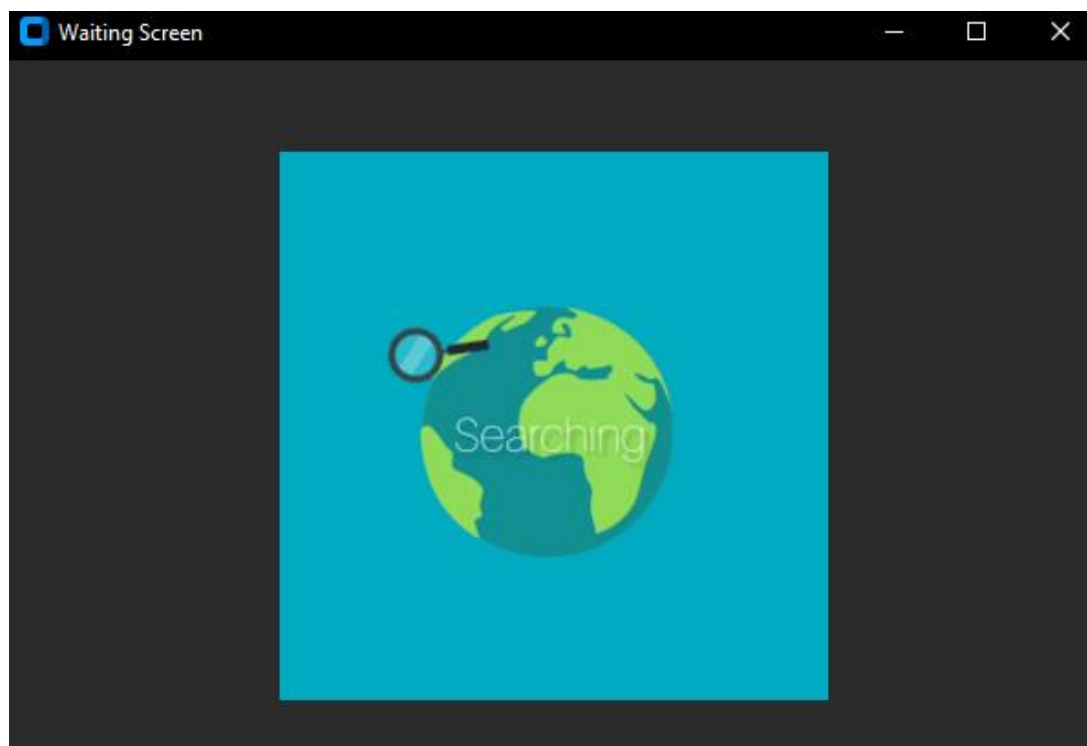
מסך ההגדרות

כרגע במסך ההגדרות קיימים 2 כפתורים, אחד לחזור למסך הבית והאחר לשנות את השפות שהמשתמש יודע.



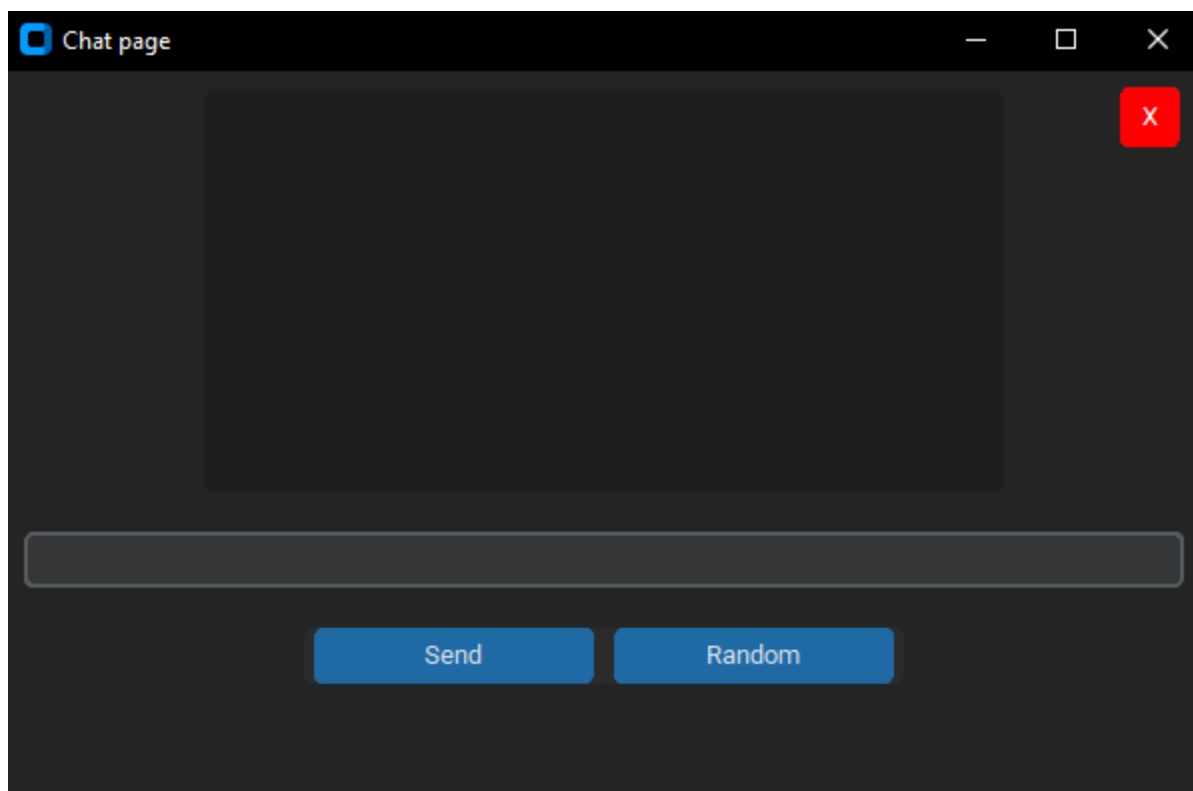
מסך ההמתנה

במסך זה מוצג GIF שמראה למשתמש שהשרת מחפש לו משתמש אחר לדבר איתו, לאחר שהשרת מוצא משתמש אז הוא מועבר למסך הצ'אט

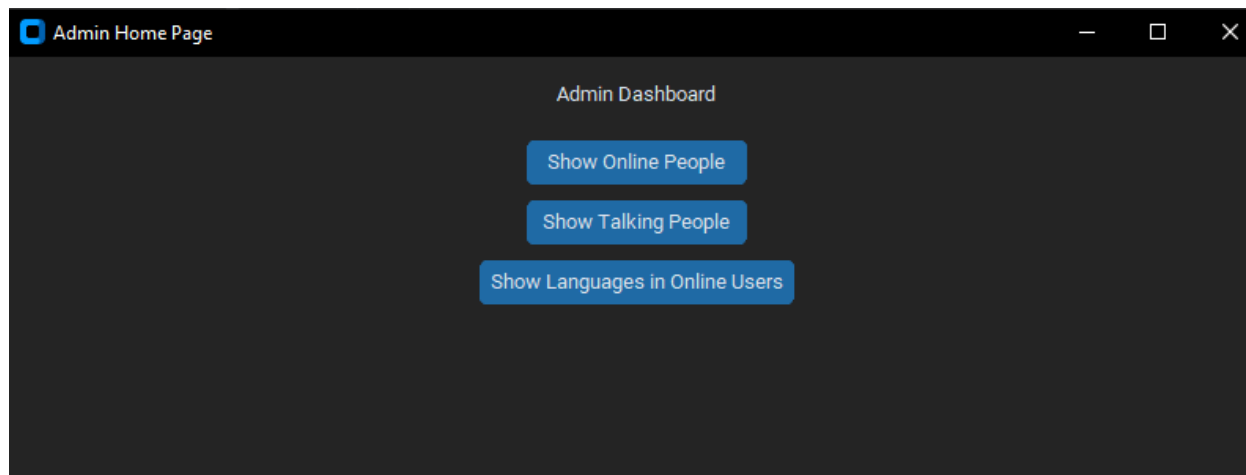


מסך הצ'אט

במסך הצאט יש 3 כפתורים ומקום למלא הודעות ומקום שמראה הודעות. לאחר לחיצה על כפתור הX בימין למעלה המשתמש מתנתק מהשיחה וחוזר למסך הבית (פעולה זו גם מחזירה את המשתמש האחר למסך הבית). לחיצה על כפתור השליחה ששולח את ההודעה לשרת ואז למשתמש האחר. וכפתור random שלאחר לחיצה עליו המשתמש חוזר למסך ההמתנה עד שהשרת יחבר אותו למשתמש אחר שמתאים אליו.

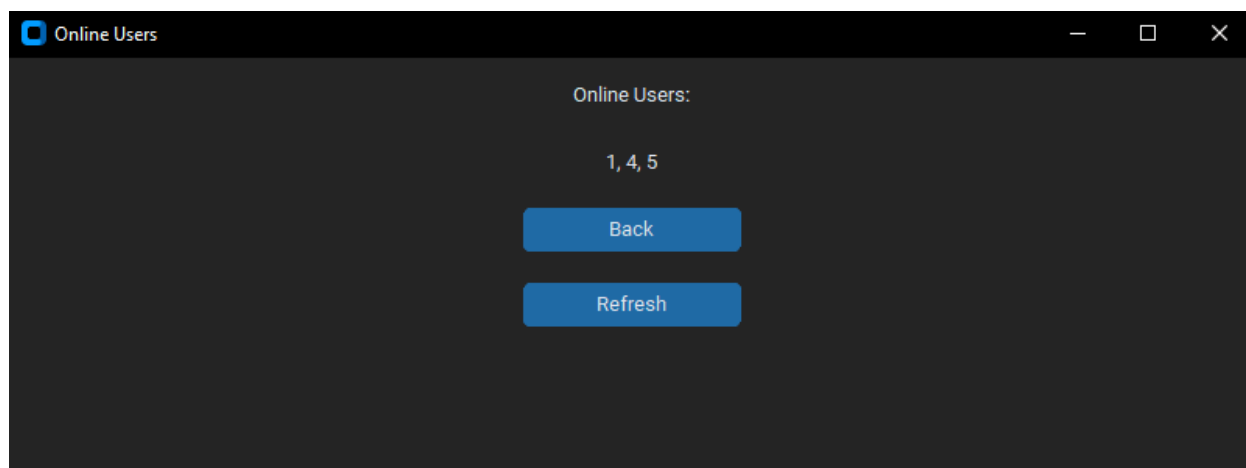


במסך זה יש 3 כפתורים, אחד מעביר למסך שמראה את המשתמשים המחוברים, האחר מראה את המשתמשים שבשיחה, והאחרון מראה את כמות המשתמשים המחוברים שמדברים כל שפה.



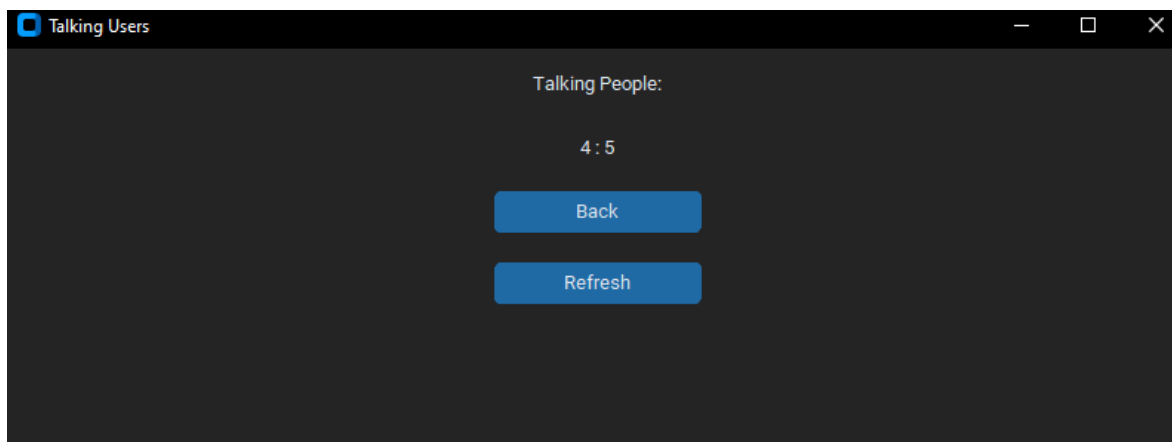
מסך שמראה את המשתמשים המחוברים – Admin

במסך יש 2 כפתורים אחד שמחזיר את הAdmin למסך הבית שלו והאחר שעושה refresh למסך למקרה ויש משתמשים התנתקו או התחברו.

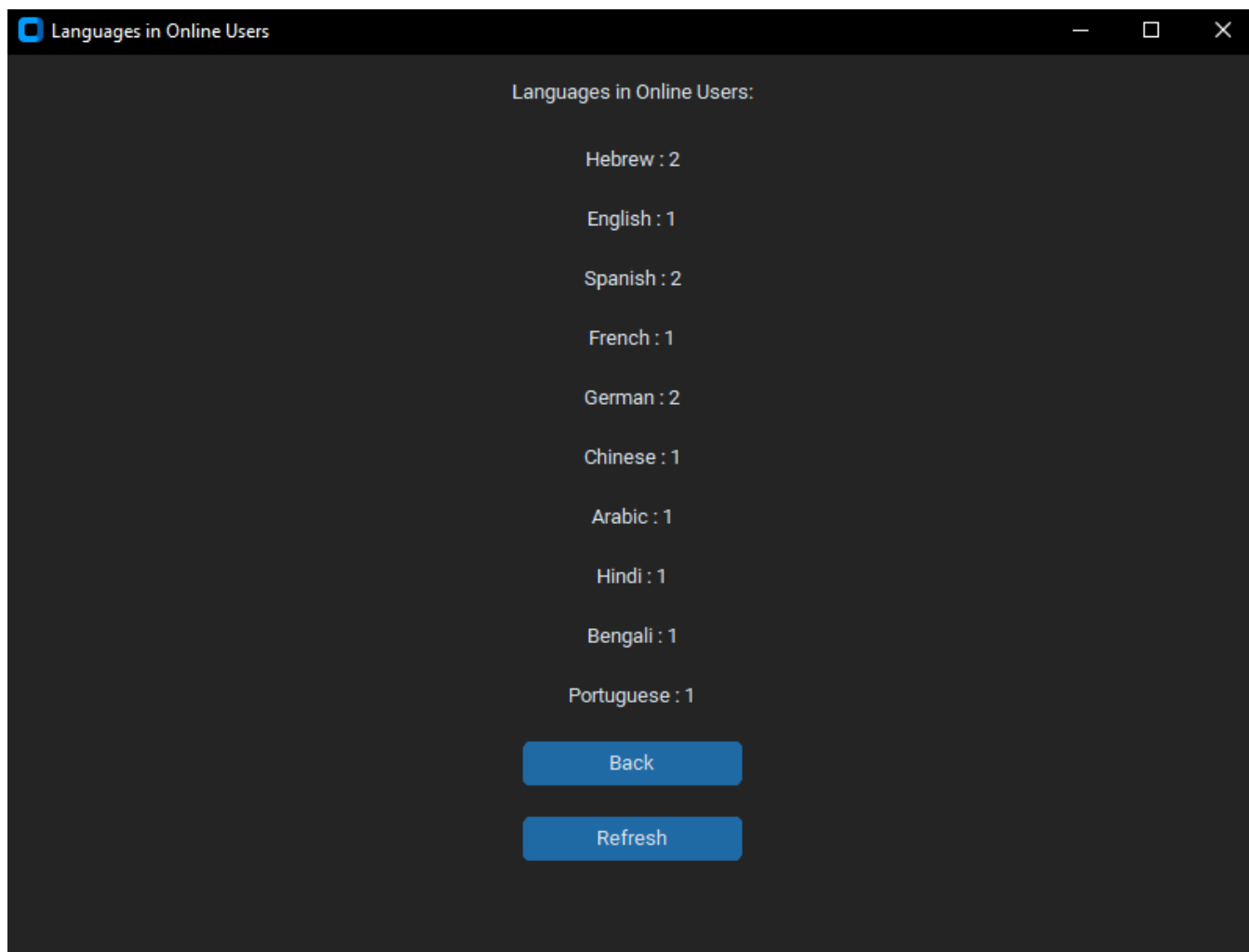


מסך שמראה את המשתמשים שמדברים – Admin

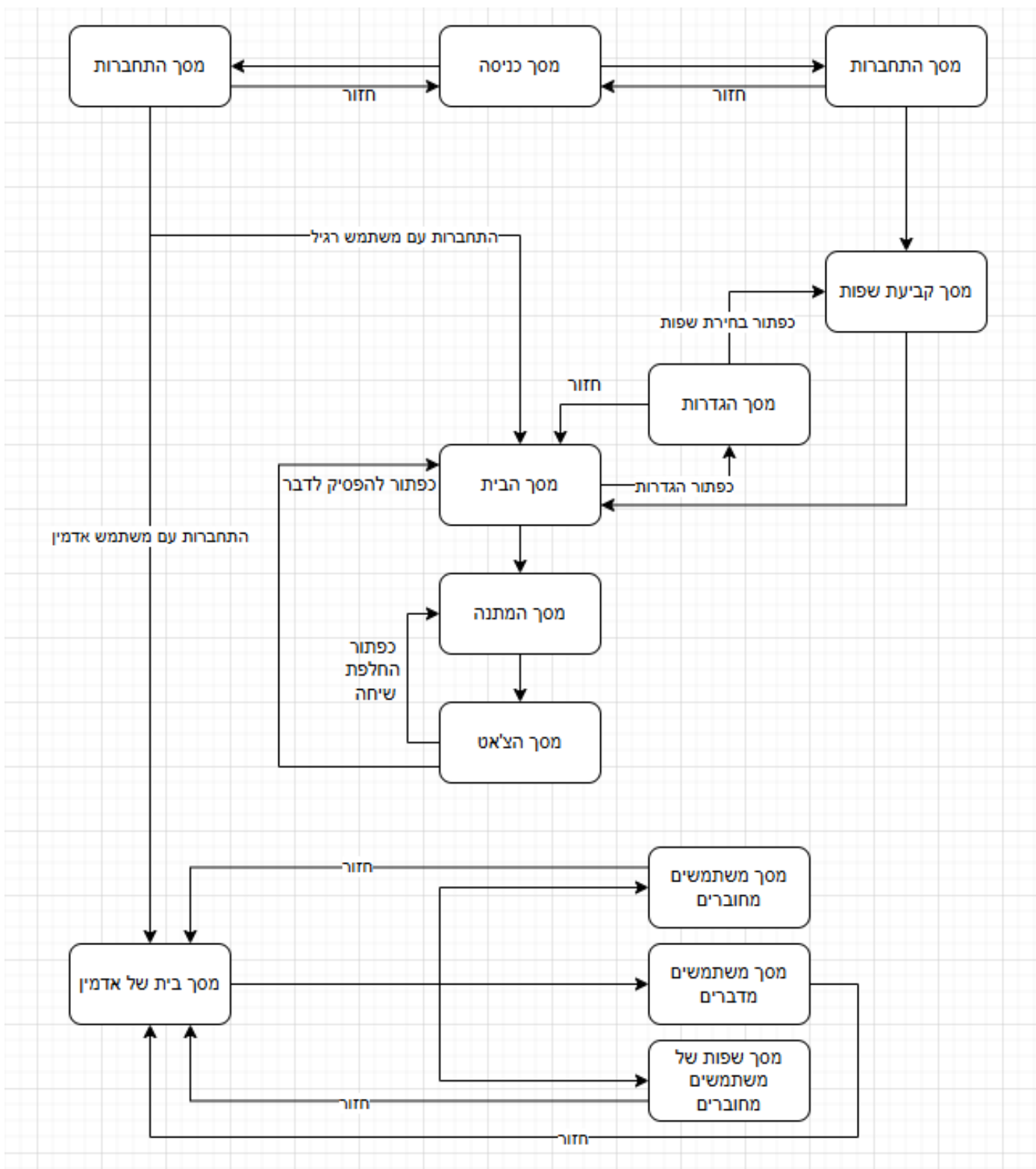
במסך יש 2 כפתורים אחד שמחזיר את הAdmin למסך הבית שלו והאחר שעושה refresh למסך למקרה ויש משתמשים שסיימו לדבר או אחרים שגם מדברים.



מסך ש מראה את כמות המשתמשים המחוברים שמדברים כל שפה – Admin
 במסך יש 2 כפתורים אחד שמחזיר את הAdmin למסך הבית שלו והאחר שעושה refresh למסך
 למקרה ויש משתמשים התנתקו או התחברו או שינו שפות.



תרשים של כל המסכים



תיאור מבני הנתונים:

קבצים:

“client.py”, “server.py”, “server_comm.py”, “client_comm.py”, “db_manager”, “gui.py”,
“primes.txt”, “protocol.py”, “rsa.py”, “user.db”, “searching.gif”

כמעט כל הקבצים הם קבצי קוד של פייטון חוץ מuser.db שזה מסד הנתונים שמכיל 3 טבלאות
searching.gif שזה GIF למסך ההמתנה.

מסד הנתונים (database)

שימוש בSQLite בDB Browser

שם מסד הנתונים: RandChat

טבלאות:

טבלה 1 – users: הטבלה שומרת על פרטי המשתמשים. שם משתמש, סיסמה ושפות שהמשתמש
יודע והמפתח הראשי הוא ID.

שם השדה	טיפוס השדה	תיאור	מאפיינים	דוגמאות
ID	INTEGER	המספר המזהה של כל משתמש	PRIMARY KEY NOT NULL	3
USERNAME	TEXT	שם המשתמש	UNIQUE NOT NULL	"Shlomo"
PASSWORD	TEXT	סיסמת המשתמש	NOT NULL	"d4735e3a265e16eee03f59718b9b5" "d03019c07d8b6c51f90da3a666eec13ab35"
LANGUAGES	TEXT	השפות שהמשתמש יכול לדבר איתם		"Hebrew,English,Spanish"

טבלה 2 – languages: הטבלה שומרת על השפות הקיימות במערכת. והמפתח הראשי הוא ID.

שם השדה	טיפוס השדה	תיאור	מאפיינים	דוגמאות
ID	INTEGER	המספר המזהה של השפה	PRIMARY KEY AUTOINCREMENT	4
LANGUAGE	TEXT	השפה	NOT NULL UNIQUE	"Hebrew"

טבלה 3 – user_and_languages: הטבלה מתעדכנת כל פעם את השפות שכל משתמש יודע ומסדרת אותה כך שכל שורה יש בה את המספר המזהה של המשתמש והמספר המזהה של השפה שהוא יודע, וכל שורה זו שפה אחרת שהמשתמש יודע. המפתח הראשי הוא גם user_id וגם language_id ושניהם גם foreign key מתוך המספרים המזהים של הטבלאות האחרות.

שם השדה	טיפוס השדה	תיאור	מאפיינים	דוגמאות
user_id	INTEGER	המספר המזהה של המשתמש	FOREIGN KEY REFERENCES "users"("ID") PRIMARY KEY	2
language_id	INTEGER	המספר המזהה של השפה	FOREIGN KEY REFERENCES "languages"("ID") PRIMARY KEY	2

סקירת חולשות ואיומים

שכבת האפליקציה

- SQL injection – מפני שמסד הנתונים מבוסס על שפה SQL הוא חשוף להזרקת SQL. הזרקת SQL מתרחשת כאשר משתמש מזין בשדה ההזנה הצהרת SQL שהשרת יפעיל על מסד הנתונים. וכך קיים סיכון שהמשתמש ישבש/ישנה או אפילו יהרוס את מסד הנתונים. כדי למזער את הסיכויים לSQL injection, השתמשתי באפשרות שכל פעם שמתבצעת קליטת נתונים מהמשתמש, או מידע אחר, תתבצע בדיקה שאין בקלט קוד שיכול להיקרא על ידי SQLite3, ותוודא שאין תווים שיכולים להוות סיכון.

- תהליך ההתחברות למערכת – כדי לוודא שאנשים שאינם מורשים להתחבר לא יתחברו, יש לוודא שהמידע שמוכנס נכון ונמצא במסד הנתונים בנוסף לכך וידאתי שהמידע עבר בצורה מוצפנת בין המשתמש לשרת בעזרת RSA, כמו כן בהכנסת הסיסמה למסד הנתונים השתמשתי בhashing (sha256), גיבוב סיסמא זה מתוכנן להיות בלתי הפיך, כך שלא ניתן לפענח סיסמא שהוצפנה בצורה זו וגם כאשר מצפינים שתי סיסמאות בעלות תו אחד שונה הסיסמאות המגובבות יהיו שונות לגמרי זו מזו.

הסבר של hashing(sha256):

באלגוריתם הצפנה כל סיסמה תהיה מגובבת לאורך של 256 ביטים.

את התהליך ניתן לחלק לכמה שלבים:

1. ריפוד הסיסמה: מוסיפים ביטים כך שהיא תהיה בדיוק 64 ביטים פחות מכפולה של 512. בהוספה, הבית הראשון הוא 1 והאחרים הם 0.
2. הוספת ייצוג בגודל 64 ביטים של אורך ההודעה המקורית כך שכעת אורך הסיסמא הוא כפולה של 512.
3. חלוקת ההודעה לחלקים של 512 ביטים כל אחד.
4. מחלקים את החלקים של 512 ביטים ל16 חלקים של 32 ביטים כל אחד
5. מעבדים את כל החלקים של 32 ביטים 64 פעמים כל אחד
6. בסוף משרשרים חזרה את כל החלקים המעובדים להודעה מוצפנת

- MITM (man in the middle) – כדי למנוע אפשרות שבה אדם המנסה לצוטט לפאקטות ולגנוב מידע בעת העברתו וכך לוקח מידע שיכול להיות רגיש, השתמשתי בהצפנת RSA כך שהאדם באמצע יוכל להזין רק להודעות מוצפנות שאינן ברורות וכך יהיה לו מאוד קשה לפענח את ההודעות ולגשת למידע רגיש.

- DDoS - כדי למנוע הצפה של המערכת בבקשות אשר יגרמו לקריסתה או לעיכוב שירותיה יש לוודא שמספר הלקוחות אינו גדול מדי על מנת למנוע שליחה של מספר גדול מדי שלהודעות.

שכבת התעבורה

- פרוטוקול TCP ולחיצת יד משולשת: פרוטוקול זה מבטיח תקשורת אמינה בעזרת לחיצת היד המשולשת: דרך תקשורת לפיה הקשר מתנהל בשלושה שלבים:
 1. שליחת הודעה לפתיחת הקשר
 2. שליחת הודעה לאישור הקשר
 3. שליחת הודעה לאישור סופי
 **מפורט יותר למעלה באזור של תיאור פרוטוקול התקשורת.

מימוש הפרויקט

מודלים/מחלקות מיובאים

שם המודול	תפקיד המודול
Socket	תקשורת בין שרת ללקוח
Select	התמודדות עם מספר לקוחות דרך השרת. וגם בלקוח בשיחה כדי לעזור לקבלת כמה הודעות במקביל
Threading	משמש ליצירה של threads כדי שכמה קטעי קוד יוכלו לרוץ בו זמנית.
Json	טעינה וקריאה של נתונים יותר מסובכים
Tkinter	יצירת ממשק משתמש גרפי אך בפרויקט שלי אני משתמש רק בmessage box כדי להקפיץ הודעות
Customtkinter	יצירת ממשק משתמש גרפי. (GUI)

Sqlite3	הרצת פקודות SQL לבסיס הנתונים מסוג SQLite.
hashlib	גיבוב מידע עם פונקציות hash.
Time	משמש לעיכוב thread כדי למנוע עומס של בדיקות והודעות
(PIL) Pillow	הצגת תמונות – GIF
random	בוחר מספר ראשוני רנדומלי מקובץ של הרבה מספרים ראשוניים
Os	עוזר לשמירת הנתונים ההתחלתית של האדמין

מודלים/מחלקות שאני מפתח

מחלקה – server_comm

תפקיד המחלקה: לספק פעולות וסדר בתקשורת של השרת על הלקוחות

שם התכונה	תפקיד
server	יצירת השרת מסוג socket
clients	רשימה ששומרת את הלקוחות שכרגע מחוברים לשרת
outgoing_msg	רשימה של ההודעות שהשרת צריך לשלוח, מוסיף לרשימה כל פעם הודעה חדשה בהתאם להודעות שצריך לשלוח
incoming_msg	רשימה של כל ההודעות השרת קיבל מהלקוחות, לטיפול
public	המפתח הציבורי של השרת בשביל לשלוח ללקוחות להצפנה
private	המפתח הפרטי של השרת בשביל לשמור להצפנה
keys	מילון של כל לקוח עם המפתח הציבורי שלו להצפנה

שם הפעולה	טענת כניסה	טענת יציאה
build_and_send_message	לקוח, פקודה, ומידע	הפונקציה בונה הודעה לפי הפרוטוקול תקשורת ולפי הפעולה והמידע. ולאחר הבנייה היא שולחת את ההודעה ללקוח המתאים.
recv_message_and_parse	לקוח	הפונקציה מקבלת הודעה מהלקוח ומפרקת אותו לחלקיו לפי פרוטוקול התקשורת, ומחזירה את הפעולה והמידע.

print_client_sockets	אין	הפונקציה מדפיסה את כל הלקוחות המחוברים כרגע
run	אין	הפונקציה מריצה לולאה אינסופית שרצה עם תהליכון (thread) על מנת לקבל הודעות מלקוחות ולטפל בהן.

מחלקה – client_comm

תפקיד המחלקה: לספק פעולות וסדר בתקשורת של הלקוח לשרת

שם התכונה	תפקיד
client	יצירת הלקוח מסוג socket
status	משתנה שנועד לשמור מידע על האם הלקוח התחבר בהצלחה לשרת ולא עבר את סך המשתתפים המקסימלי, עוזר להגנה מפני DDoS
private	שומר על הלקוח הפרטי של הלקוח להצפנה
public	שומר על המפתח הציבורי של הלקוח להצפנה וכדי לשלוח לשרת
server_public	שומר על המפתח הציבורי של השרת להצפנה

שם הפעולה	טענת כניסה	טענת יציאה
error_and_exit	הודעת שגיאה	מציג הודעת שגיאה של הלקוח ועוזב
build_and_send_message	פקודה ומידע	הפונקציה בונה הודעה לפי הפרוטוקול תקשורת ולפי הפעולה והמידע. ולאחר הבנייה היא שולחת את ההודעה לשרת.
recv_message_and_parse	אין	הפונקציה מקבלת הודעה מהשרת ומפרקת אותו לחלקיו לפי פרוטוקול התקשורת, ומחזירה את הפעולה והמידע.
exchange_keys	אין	הפונקציה פעולת ישר לאחר החיבור של הלקוח לשרת ומחליפה עם השרת את המפתחות הציבוריים להצפנה.

מחלקה – ChatGUI

תפקיד המחלקה: ליצור ממשק גרפי ללקוח ולעזור ביצירות הודעות ופעולות

שם התכונה	תפקיד
ID	לשמור על המספר המזהה של אותו לקוח שקיבל מהשרת
chat_area	עוזר לשמור ולעדכן את המסך של השיחה בזמן השיחה
outgoing_msg	שומר את ההודעה שהלקוח מעוניין לשלוח לשרת
entry_username	עוזר לשמור על המידע של תיבת ההקלדה של שם המשתמש
entry_password	עוזר לשמור על המידע של תיבת ההקלדה של שם הסיסמה
entry_message	עוזר לשמור על המידע של תיבת ההקלדה של ההודעה בשיחה
language_vars	שומר על השפות של התכנית ועדכון, בנוסף גם עוזר להצגה הגרפית
is_animating	משתנה בוליאני, עוזר למנוע שגיאות הקשורות לאנימציה של gif
title	עוזר להצגה של הכותרת של כל מסך
geometry	עוזר להצגת גודל של המסך לכל מסך

שם הפעולה	טענת כניסה	טענת יציאה
clear_widgets	אין	הפעולה מוחקת את כל החלקים של הממשק הגרפי
create_start_interface	אין	הפעולה יוצרת את מסך הכניסה
show_signup_interface	אין	הפעולה יוצרת את מסך ההרשמה
show_login_interface	אין	הפעולה יוצרת את מסך ההתחברות
login	אין	הפעולה יוצרת את הודעת ההתחברות שהלקוח רוצה לשלוח לשרת

login_ans	פקודה ומידע	הפעולה מציגה על המסך את התשובה שהלקוח קיבל מהשרת על בקשת ההתחברות
signup	אין	הפעולה יוצרת את הודעה ההרשמה שהלקוח רוצה לשלוח לשרת
signup_ans	פקודה ומידע	הפעולה מציגה על המסך את התשובה שהלקוח קיבל מהשרת על בקשת ההרשמה
define_languages	מחרוזת של רשימות	הפעולה הופכת את המחרוזות למילון להצגה נוחה יותר של השפות על המסך
show_language_interface	אין	הפעולה יוצרת את מסך בחירת השפות
validate_languages	אין	הפעולה בודקת אם נבחרה כמות נכונה של שפות ושולחת הודעה ללקוח או לשרת בהתאם
languages_ans	פקודה	הפונקציה מציגה ללקוח את התשובה לבקשתו לעדכן את השפות
create_home_page	אין	הפונקציה יוצרת את מסך הבית
setting_screen	אין	הפונקציה יוצרת את מסך ההגדרות
create_chat_interface	אין	הפונקציה יוצרת את מסך השיחה
end_talking_and_go_home	אין	הפעולה יוצרת את הודעת ההתנתקות מהשיחה שהלקוח רוצה לשלוח לשרת
show_and_return_message	אין	הפעולה מציגה על מסך השיחה את ההודעה שהלקוח רוצה לשלוח ללקוח האחר בשיחה. בנוסף לכל היא יוצרת את ההודעה שהיא רוצה לשלוח לשרת כדי שיעביר ללקוח האחר
display_message	הודעה	מציגה את ההודעה על המסך בזמן השיחה

הפעולה מציגה הודעה בחלון נפתח על מנת להסב את תשומת הלב של הלקוח להודעה	הודעה	pop_message
הפעולה יוצרת את הודעת החלפת לקוח בשיחה, שהלקוח רוצה לשלוח לשרת	אין	change_random_person
הפונקציה מציגה את התשובה של לבקשה של הלקוח להחליף משתמש לשוחח איתו	פקודה	change_person_ans
הפונקציה מראה את מסך ההמתנה	אין	show_loading_screen
הפונקציה עוזרת ליצירת האנימציה לgif במסך ההמתנה	מיקום באנימציה (ברירת מחדל 0)	animate
הפונקציה מראה את מסך הבית של האדמין	אין	show_admin_screen
הפעולה יוצרת את בקשת האדמין לראות מי המשתמשים המחוברים	אין	show_online_people_request
הפעולה מציגה את התשובה לבקשה של האדמין להראות מי המשתמשים המחוברים	פקודה ומידע	show_online_people_ans
הפעולה יוצרת את בקשת האדמין לראות מי המשתמשים שבשיחה כרגע	אין	show_talking_people_request
הפעולה מציגה את התשובה לבקשה של האדמין לראות מי המשתמשים שבשיחה כרגע	פקודה ומידע	show_talking_people_ans
הפעולה יוצרת את בקשת האדמין לראות את סכום המדברים של כל שפה מהשפות של המשתמשים המחוברים	אין	show_languages_in_online_users_request
הפעולה מציגה את התשובה לבקשה של האדמין לראות את סכום המדברים של כל	פקודה ומידע	show_languages_in_online_users_ans

שפה מהשפות של המשתמשים המחוברים		
הפעולה מציגה את המסך בו היא מראה לאדמין את המשתמשים המחוברים	מידע	show_online_people_screen
הפעולה מציגה את המסך בו היא מראה לאדמין את המשתמשים שבשיחה כרגע	מידע	show_talking_people_screen
הפעולה מציגה את המסך בו היא מראה לאדמין את סכום המדברים של כל שפה מהשפות של המשתמשים המחוברים	מידע	show_languages_in_online_users_screen

מחלקה – ChatDatabase

תפקיד המחלקה: ליצור את הטבלאות ומסד הנתונים בשביל השימוש בפרויקט ושמירת הנתונים למחלקה יש תכונה אחת של שם מסד הנתונים אך אין לה פעולות והיא מהווה בסיס לשאר המחלקות היורשות ממנה

מחלקה – Activatedb

תפקיד המחלקה: לעדכן, לשנות ולהוסיף למסד הנתונים שקיים ממחלקת האב. למחלקה יש תכונה אחת שיורשת ממחלקת האב (שם מסד הנתונים) והיא בעלת פעולות.

שם הפעולה	טענת כניסה	טענת יציאה
insert_languages_and_admin	שפות	הפעולה מכניסה למסד הנתונים את השפות המתאימות ואת הנתונים של האדמין (אם אינם קיימים במסד הנתונים עדיין)
login	שם משתמש וסיסמה	הפעולה בודקת אם ניתן לאשר את ההתחברות של משתמש דרך מסד הנתונים
signup	שם משתמש וסיסמה	הפעולה בודקת אם ניתן להירשם עם נתונים אלו דרך מסד הנתונים ומוסיפה אותו למסד הנתונים

language_update	שפות ומספר מזהה	הפעולה מעדכנת את השפות שהמשתמש יודע במסד הנתונים לפי המספר המזהה
get_languages	מספר מזהה	הפעולה מחזירה את רשימת השפות שקיימות במסד הנתונים
check_user	מספר מזהה	הפעולה בודקת אם המשתמש תקין ואינו מהווה בעיה במסד הנתונים
remove_user	מספר מזהה	הפעולה מוציאה משתמש ממסד הנתונים אם הוא מהווה בעיה

מודול – protocol

תפקיד המודל: המודל בונה את ההודעה לפני שליחת לפני פרוטוקול התקשורת שלי, ובונה אותה נכון מבחינה תחבירית (syntax). המודל מכיל את כלל הפקודות שקיימות ללקוח, לשרת ולמשתמש אדמין

שם הפעולה	טענת כניסה	טענת יציאה
build_message	פקודה ומידע	הפעולה בונה הודעה בהתאם לתחביר את פרוטוקול התקשורת ומחזירה את ההודעה הבנויה
parse_message	הודעה	הפעולה מפרקת את ההודעה לחלקיה לפי התחביר את הפרוטוקול ומחזירה את חלקיה (הפקודה והמידע)
split_data	מידע ומספר חלקים צפויים	הפעולה מפרקת את המידע לפי מספר החלקים הצפויים שיהיו בתוך המידע ומחזירה רשימה של חלקיה

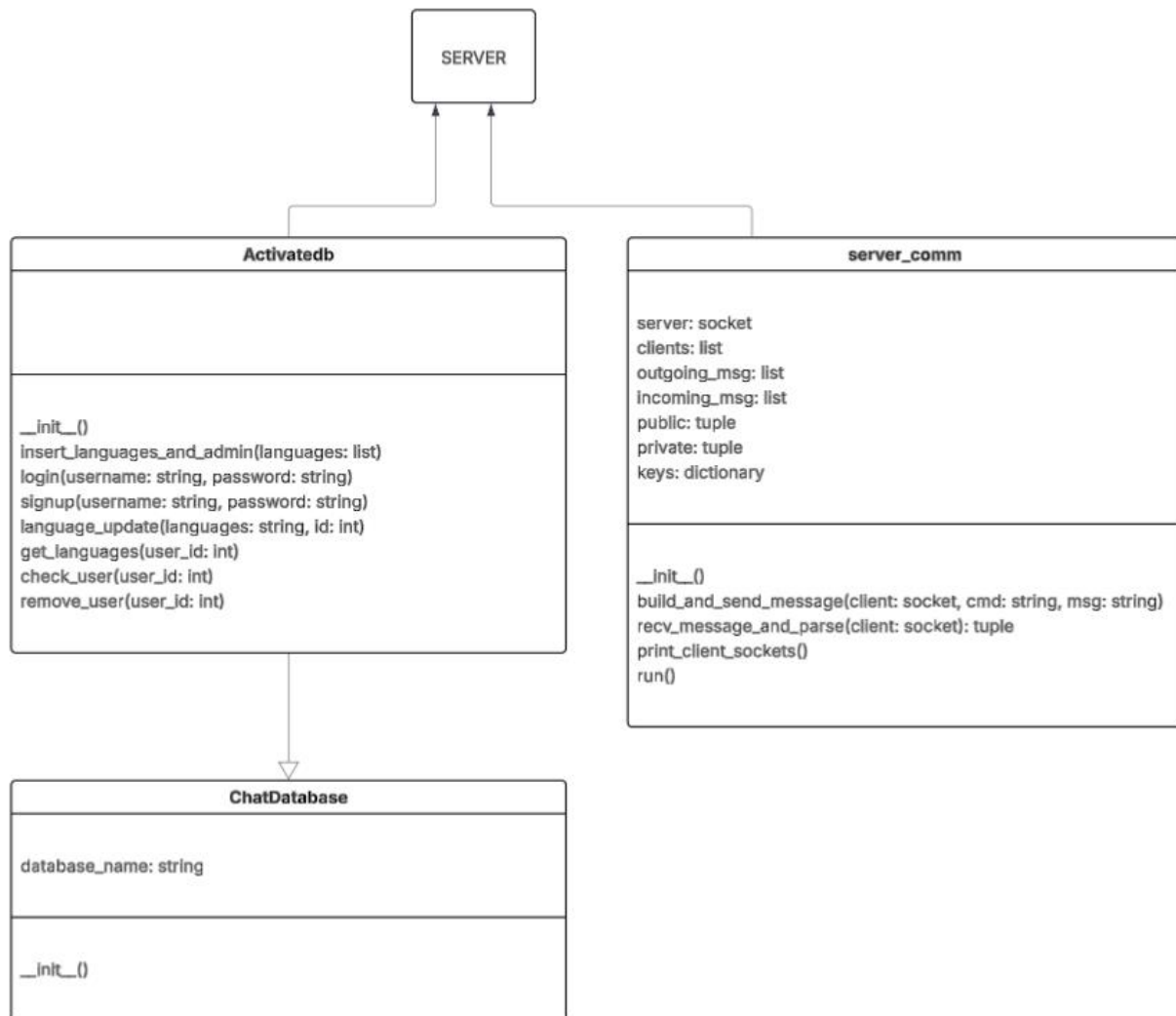
מודול – rsa

תפקיד המודל: המודל עוזר ובונה את המפתחות הפרטיים והציבוריים לשרת והלקוח לפי הצפנת RSA ונעזר במתמטיקה והחישובים הנדרשים.

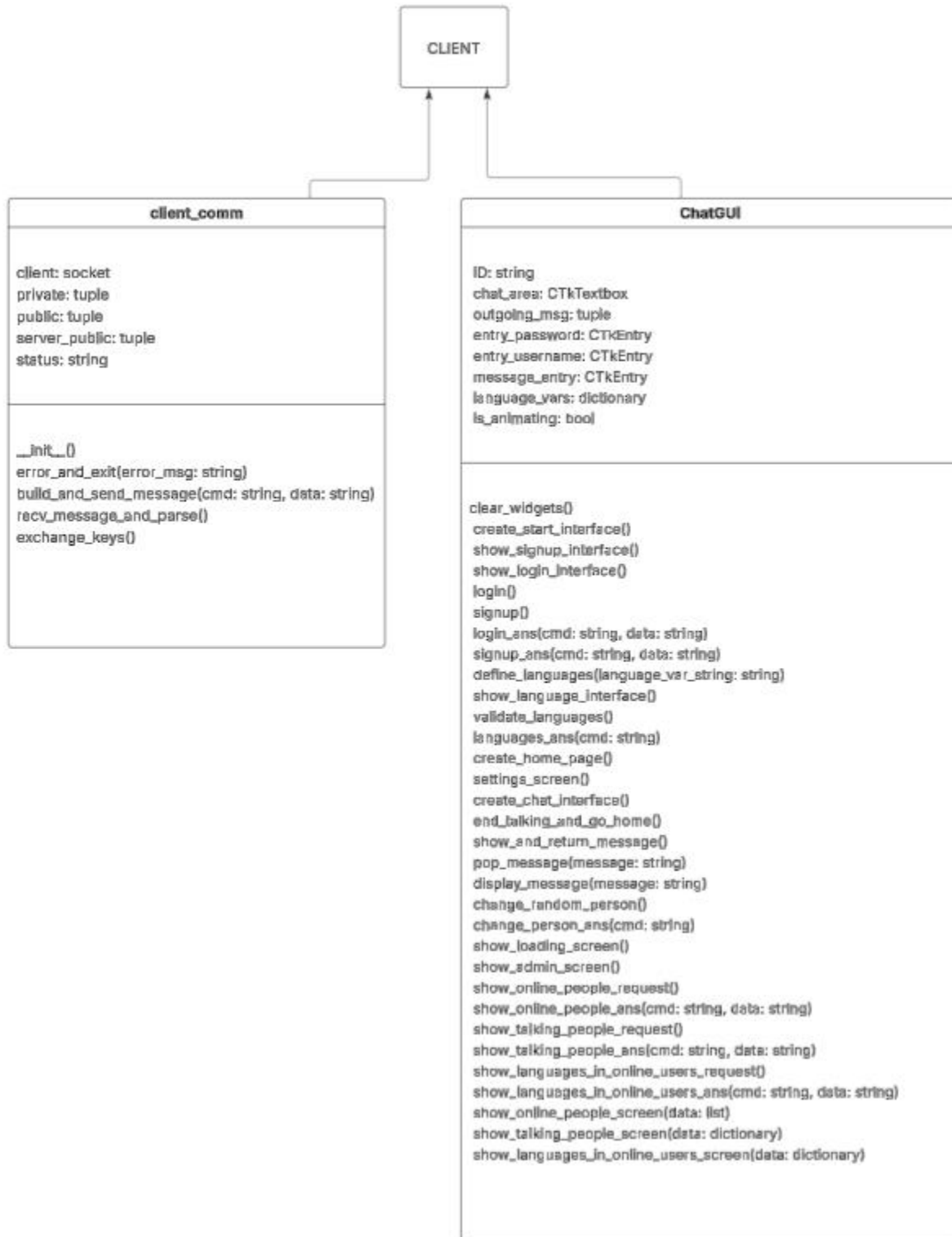
שם הפעולה	טענת כניסה	טענת יציאה
gcd (Greatest common divisor)	שני מספרים	הפעולה מוצאת את המספר המחלק המשותף הגדול ביותר לשני המספרים ומחזירה אותו
multiplicative_inverse	שני מספרים	הפעולה משתמש במפתח הציבורי על מנת ליצור את המפתח הפרטי ומחזירה אותו

generate_keypair	שני מספרים	הפעולה יוצרת 2 מפתחות, פרטי וציבורי ומחזירה אותן
encrypt	מפתח ציבורי, הודעה	הפעולה מצפינה את ההודעה לפי המפתח הציבורי ומחזירה את הצופן
decrypt	מפתח פרטי, קוד מוצפן	הפעולה מפענחת את הצופן לפי המפתח הפרטי ומחזירה את ההודעה לאחר הפענוח
generate_primes	אין	הפעולה בוחרת 2 מספרים רנדומליים מקובץ טקסט שמכיל כחצי מיליון מספרים ראשוניים ומחזירה אותן.

תרשים של המחלקות בצד השרת:



תרשים של המחלקות בצד הלקוח:



קטעי קוד האלגוריתמים המרכזיים

התחברות והרשמה למערכת

בהתחברות וההרשמה הקוד בצד של הלקוח אינו שומר את הנתונים אלא רק שולח לשרת (הודעה מוצפנת) והוא מכניס אותו למסד הנתונים לאחר שהצפין את הסיסמה. לאחר ההרשמה או ההתחברות, השרת מחזיר ללקוח הודעת הצלחה או שגיאה בהתאם ומחזיר לו את המספר המזהה שלו (שאותו כן יכול לשמור) אם קיבל הודעת הצלחה.

צד הלקוח:

```
#login with an existing account enter the program
def login():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get
    client_comm.build_and_send_message(cmd_send, data_send)

    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.login_ans(cmd_get, data_get)
    if gui_chat.ID and int(gui_chat.ID) == 1:
        gui_chat.show_admin_screen()

#signup with a new account enter the program
def signup():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get
    client_comm.build_and_send_message(cmd_send, data_send)

    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.signup_ans(cmd_get, data_get)
```

צד השרת:

התחברות:

```
def handle_login_message():
    global database, client, data, cmd, server_comm, logged_users_id, talking
    print("login: ",data)
    username, password = proto.split_data(data, 2)
    answer = database.login(username, password)
    if answer in logged_users_id.values():
        server_comm.outgoing_msg.append((client,proto.COMMAND_SERVER["login_failed"],
        "User already connected"))
    elif answer:
        talking[client] = None
        last_talked[client] = None
        logged_users_id[client] = answer
        print(logged_users_id.values())
        server_comm.outgoing_msg.append((client,
        proto.COMMAND_SERVER["login_ok"],
        f"{str(answer)}{proto.DATA_DELIMITER}{'.'.join(languages_online.keys())}"))
    else:
        server_comm.outgoing_msg.append((client,
        proto.COMMAND_SERVER["login_failed"], ""))
        cmd = ""
```

הרשמה:

```
def handle_signup_message():
    global database, client, data, cmd, server_comm, logged_users_id
    print("sign_in: ", data)
    username, password = proto.split_data(data, 2)
    answer = database.signup(username, password)
    if not username or not password or username == "" or password == "":
        server_comm.outgoing_msg.append((client,
        proto.COMMAND_SERVER["signup_failed"], ""))
        database.remove_user(answer)
    elif answer:
        server_comm.outgoing_msg.append((client,
        proto.COMMAND_SERVER["signup_ok"],
        f"{str(answer)}{proto.DATA_DELIMITER}{','.join(languages_online.keys())}"))
        logged_users_id[client] = answer
    else:
        server_comm.outgoing_msg.append((client,
        proto.COMMAND_SERVER["signup_failed"], ""))
```

עדכון מסד הנתונים:

```
def login(self, username, password):
    conn = sqlite3.connect('RandChat.db')
    c = conn.cursor()
    enc = hashlib.sha256(password.encode()).hexdigest()
    c.execute("SELECT ID FROM users WHERE username=? AND password=?",
    (username, enc))
    user_id = c.fetchone()
    print("the USER is: ", user_id)
    conn.close()
    if user_id:
        return user_id[0]
    else:
        return None

def signup(self, username, password):
    conn = sqlite3.connect('RandChat.db')
    c = conn.cursor()
    # Check if either the username or password already exists in the database
    enc = hashlib.sha256(password.encode()).hexdigest()
    c.execute("SELECT ID FROM users WHERE username=? OR password=?",
    (username, enc))
    existing = c.fetchone()
    if existing:
        # Duplicate found; do not create a new record.
        conn.close()
        return None
```

ניהול שיחה בין 2 לקוחות

כשהשרת משבץ זוג לקוחות לשיחה הוא מכניס אותם למילון שבו הוא מסמן את השיחות שמקיימות כרגע, וכך שולח הודעה לשני הלקוחות להפסיק לשחק והם נכנסים מסך השיחה, כאשר אחד מהם רוצה לשלוח הודעה ללקוח האחר השיחה, הוא שולח הודעה לשרת והשרת מעביר את ההודעה ללקוח המתאים בשיחה לפי מילון השיחות, וכך מתנהלת השיחה בין שני הלקוחות ובין כל השיחות.

וכדי שהלקוח יוכל גם לקבל הודעות וגם לשלוח הודעות באותו זמן, יש 2 תהליכים, אחד שרץ על הפעולות שהלקוח רוצה לעשות והאחר משתמש בselect כדי לקבל הודעות בזמן שיחה, מפני שכך הפעולה אינה חוסמת ומאפשרת לפעולות לקרות בו זמנים.

תהליכון שמקשיב ומציג הודעות על המסך: (לדעתי זו דרך שונה ויחסית מיוחדת לבעיה)

```
#a thread that runs while the client is talking and waits for messages from
the other client
def listen_to_friend():
    global client_comm, gui_chat, cmd_send, data_send, is_listening, cmd_get,
    data_get
    # Use a non-blocking approach to check for incoming messages
    while True:
        try:
            if is_listening:
                # Check for incoming messages
                r, _, _ = select.select([client_comm.client], [], [])
                if r:
                    print("im listening here")
                    cmd_get, data_get = client_comm.recv_message_and_parse()
                    print(f"listen command: {cmd_get} listen datas:
{data_get}")

                    if cmd_get is not None: # If a command is received
                        if cmd_get == proto.COMMAND_SERVER["forward_talk"]:
                            gui_chat.display_message(f"Friend: {data_get}")
                            cmd_get, data_get = None, None
                        elif cmd_get == proto.COMMAND_SERVER["error"] or
cmd_get == proto.COMMAND_SERVER["friend_left"]:

                            gui_chat.pop_message(data_get)
                            gui_chat.create_home_page() # Return to start
interface

                            is_listening = False
                            time.sleep(0.1) # Add a small delay to prevent busy waiting
                        except Exception as e:
                            print(f"Error in listen_to_friend: {e}")
```

תהליכון לכלל הפעולות של הלקוח:

```
# thread that check if the client wants to send something or do a command
def comm_thread():
    global cmd_send, data_send, client_comm, l, gui_chat
    while True:
        time.sleep(0.1)
        l.acquire()
        if gui_chat.outgoing_msg:
            cmd_send, data_send = gui_chat.outgoing_msg
            print(f"Processing outgoing message: cmd_send={cmd_send},
data_send={data_send}")
            handle_request_message()
        l.release()
```

שיבוץ הלקוחות לשיחות של אחד על אחד

לאחר שהמשתמש הודיע לשרת שהוא רוצה להיכנס לשיחה ושלח לו הודעה, הלקוח נכנס לרשימת מחכים והשרת מטפל בכל המשתמשים שמחכים באמצעות תהליכון שעובר על הרשימה ומצוות זוגות של משתמשים שמתאימים, בכך שיש להם שפה משותפת והם לא דיברו פעם אחרונה.

```
#this function is run with a thread to check if it can assign a client to
another client based languages and last talked conditions
def process_waiting_clients():
    global client
    while True:
        if len(waiting_clients) >= 2:
            for i in range(len(waiting_clients) - 1):
                for j in range(i + 1, len(waiting_clients)):
                    client1 = waiting_clients[i]
                    client2 = waiting_clients[j]

                    if (client1 in last_talked and last_talked[client1] ==
client2) or (client2 in last_talked and last_talked[client2] == client1):
                        continue # Skip pairing if they just talked or don't
have a matching language

                    languages1 =
database.get_languages(logged_users_id[client1])
                    languages2 =
database.get_languages(logged_users_id[client2])
                    common_languages = [item for item in languages1 if item
in languages2]

                    if not common_languages:
                        continue

                    server_comm.outgoing_msg.append((client1,
proto.COMMAND_SERVER["waiting_stop"], ""))
                    server_comm.outgoing_msg.append((client2,
proto.COMMAND_SERVER["waiting_stop"], ""))

                    talking[client1] = client2
                    talking[client2] = client1
```

```

        waiting_clients.remove(client1)
        waiting_clients.remove(client2)

        break

time.sleep(0.1)

```

העברת מידע בין השרת והלקוחות בצורה מאובטחת

לכל הודעה לאחר החלפת מפתחות הציבוריים והפרטיים, מוצפנת לפי RSA ונשלחת מוצפנת ורק לאחר קבלתה היא מפוענחת וניתן להשתמש בנתונים.

לקוח:

```

# this function gets the data to build an encrypted message to send to the
server
def build_and_send_message(self, cmd, data):
    try:
        full_msg = proto.build_message(cmd, data)
        encrypted_msg = rsa.encrypt(self.server_public, full_msg)
        # Convert the list of encrypted integers to a string
        encrypted_msg_str = ','.join(map(str, encrypted_msg))
        self.client.send(encrypted_msg_str.encode()) # Send the string
        representation of the encrypted message
    except Exception as e:
        self.error_and_exit(f"Failed to build or send message: {e}")

# this function receives the message and decrypts it
def recv_message_and_parse(self):
    try:
        full_msg = self.client.recv(BUFFER_SIZE).decode()
        split_msg = [int(a) for a in full_msg.split(',')]
        full_msg = rsa.decrypt(self.private, split_msg)
        cmd, data = proto.parse_message(full_msg)

        return cmd, data
    except Exception as e:
        self.error_and_exit(f"Failed to receive or parse message: {e}")

```

שרת:

```

# this function gets the data to build an encrypted message to send to the
client
def build_and_send_message(self, client, cmd, msg): #ENCRYPT
    try:
        full_msg = proto.build_message(cmd, msg)
        encrypted_msg = rsa.encrypt(self.keys[client], full_msg)

```



```

        encrypted_msg_str = ','.join(map(str, encrypted_msg))

        client.send(encrypted_msg_str.encode())
    except Exception as e:
        print(f"Failed to build or send message: {e}")

# this function receives the message and decrypts it
def recv_message_and_parse(self, client): #DECRYPT
    try:
        full_msg = client.recv(BUFFER_SIZE).decode()
        if not full_msg:
            # Handle client disconnection
            print(f"Connection closed by {client.getpeername()}")
            return None, None

        if client in self.keys:
            split_msg = [int(a) for a in full_msg.split(',')]
            full_msg = rsa.decrypt(self.private, split_msg)

            print("[CLIENT] ", full_msg) # Debug print
            cmd, data = proto.parse_message(full_msg)
            return cmd, data
    except Exception as e:
        print(f"Failed to receive or parse message: {e}")
        return None, None

```

פרוטוקול התקשורת (תחביר):

```

#this function builds a message based on the protocol
def build_message(cmd, data):
    if not isinstance(cmd, str) or not isinstance(data, str) or len(cmd) >=
CMD_FIELD_LENGTH:
        return None
    try:
        message = f"{cmd.ljust(4)}{DELIMITER}{data}"
        return message
    except Exception as e:
        print(f"Error constructing the message: {e}")
        return None

#this function parses a message based on the protocol
def parse_message(message):
    parts = message.split(DELIMITER)
    if len(parts) != 2:
        return None, None

    cmd = parts[0].strip()
    data = parts[1]

    return cmd, data

```

מסמך בדיקות מלא:

בדיקה 1: האם המידע עובר כמו שצריך בין צדדי התקשורת?

מטרת הבדיקה	מה בוצע בפועל	מה היו תוצאות הבדיקה	בעיות שהתגלו וכיצד נפתרו
לוודא שהמידע עובר במלואו וללא שגיאות בין צדדי התקשורת	הדפסתי את המידע בצד השולח ובצד המקבל ווידאתי שתוכן שתי ההדפסות זהה	בסוף התוצאות היו נכונות ומדויקות אך היו קצת בעיות בדרך.	בחלק מאוד קטן של ההודעות שנשלחות יכול להיגרם מצב שנשלחת הודעה הגדולה מ-1024 ביטים (והיא לא נשלחת) ומכיוון שהיא לא גדולה בהרבה מ-1024 ואינה יכולה לגדול עוד הרבה פשוט הגדלתי את המקסימום ל-2048 ביטים במקום 1024. בנוסף היה גם בעיה שבחלק ממקרי הקצה הלקוח קרס, אך לאחר הוספה של exceptions נפתרה הבעיה

בדיקה 2: תקינות מנגנון ההצפנה והפענוח

מטרת הבדיקה	מה בוצע בפועל	מה היו תוצאות הבדיקה	בעיות שהתגלו וכיצד נפתרו
לוודא שההצפנה והפענוח תקינים	הדפסתי את המידע לפני ההצפנה, והדפסתי את המידע לאחר ההצפנה ופענוח.	המידע היה תואם	בהתחלה הייתה לי בעיה שניסיתי להצפין עם ערך שאינו מחרוזת כמו מספר, רשימה או מילון. אך פתרתי בכך שהמרתי את הנתונים לערך של מחרוזת להצפנה וחזור בפענוח.

בדיקה 3: תפקוד מסד הנתונים

מטרת הבדיקה	מה בוצע בפועל	מה היו תוצאות הבדיקה	בעיות שהתגלו וכיצד נפתרו
לוודא שהשרת ומסד הנתונים מסונכרנים זה עם זה.	לאחר כל הכנסה או שינוי של מסד הנתונים פתחתי את הטבלאות במסד הנתונים באמצעות התוכנה DB Browser SQLite	הנתונים התעדכנו במסד הנתונים בהתאם למידע שניתן לו	לא היו בעיות

בדיקה 4: בדיקת הצגת המידע במסכי המשתמשים

מטרת הבדיקה	מה בוצע בפועל	מה היו תוצאות הבדיקה	בעיות שהתגלו וכיצד נפתרו
לוודא שכל המידע הרלוונטי ורק המידע הרלוונטי מוצג בממשק המשתמש הגרפי	בדקתי את המידע שנשלח או הגיע ללקוח והשוואתי בין מה שהלקוח קיבל לבין מה שהממשק הגרפי הציג	המידע שהוצג היה כולו רלוונטי וכולו הגיע אך היו קצת בעיות קטנות בהצגה של המידע על הממשק הגרפי.	בהצגת חלק מן הנתונים של האדמין כמו הרשימה של השפות וסך כל המשתתפים המחוברים שיודעים כל שפה, נתקלתי קצת בבעיות להציג את המידע על המסך אך פתרתי זאת כשהרחבתי והגדלתי את המסך וכשניקיתי את המילון שנשלח מהשרת וסידרתי אותו בהתאם

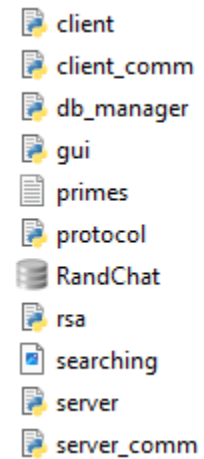
בדיקה 5: בדיקת מקרי התנתקות של המשתמש מהשרת וההפך

מטרת הבדיקה	מה בוצע בפועל	מה היו תוצאות הבדיקה	בעיות שהתגלו וכיצד נפתרו
לוודא שניתוק או קריסה של אחד המשתמשים או השרת לא תשפיע על שאר המשתמשים המערכת. ולוודא שכאשר משתמש מתנתק גם הנתונים של שברשת ובמסד הנתונים מתעדכנים. וכאשר	השוואתי בין הנתונים שהיו במסד הנתונים ובנתונים בשרת לפני ואחרי ההתנתקות של המשתמש ובדקתי אם כאשר ניתקתי אחד מהמשתמשים כל שאר המשתמשים והשרת ממשיכים לעבוד כרגיל, והוספתי בדיקה אם	הנתונים התעדכנו בשרת ובמסד הנתונים והמשתמשים האחרים המשיכו לפעול כרגיל ללא בעיה. וכאשר הייתה בעיה בבניית ההודעה ושליחתה אז זה ניתק את המשתמשים.	רוב הבעיות היו קטנות ופשוט הוספתי עוד exceptions ובשינוי של מסד הנתונים והנתונים בשרת לא היו בעיות.

		התקשורת של השרת עדיין קיימת עם הודעה ping	השרת נסגר או קרס זה שולח הודעה ומעיף את המשתתפים המחוברים
--	--	---	---

מדריך למשתמש

עץ קבצים



התקנת במערכת

פירוט הסביבה הנדרשת

על מנת לעבוד על הקוד ולהריצו דרוש מחשב שיש בו פייתון של 3.11 ומעלה, חיבור לרשת, CMD (Command Prompt), וכלל הספריות:

- Socket – גם בשרת וגם בלקוח
- Select – גם בשרת וגם בלקוח
- Sqlite3 – רק בשרת
- Threading – גם בשרת וגם בלקוח
- Customtkinter – רק בלקוח
- Json – רק שרת
- Pillow (PIL) – רק לקוח
- os – רק בשרת

פירוט הכלים הנדרשים

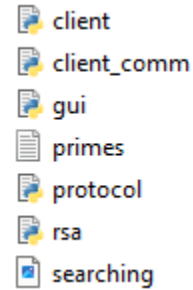
על מנת לעבוד על הקוד או להריץ אותו, מומלץ להשתמש ב-PYCHARM על מנת לקרוא את המידע במסד הנתונים, מומלץ להשתמש ב-DB Browser SQLite.

להתקנת המודלים והספריות המיובאים :

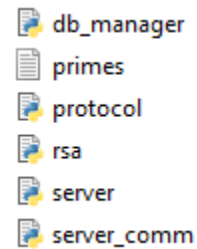
- להתקין את PIP (מנהל הספריות)
- להוריד את הספריות המצויינות לעיל בפתיחת CMD והרצת הפקודה: `PIP install [module_name]`

מיקומי הקבצים

בצד הלקוח/אדמין/כלל המשתמשים:

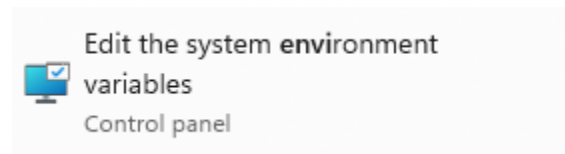


בצד השרת:

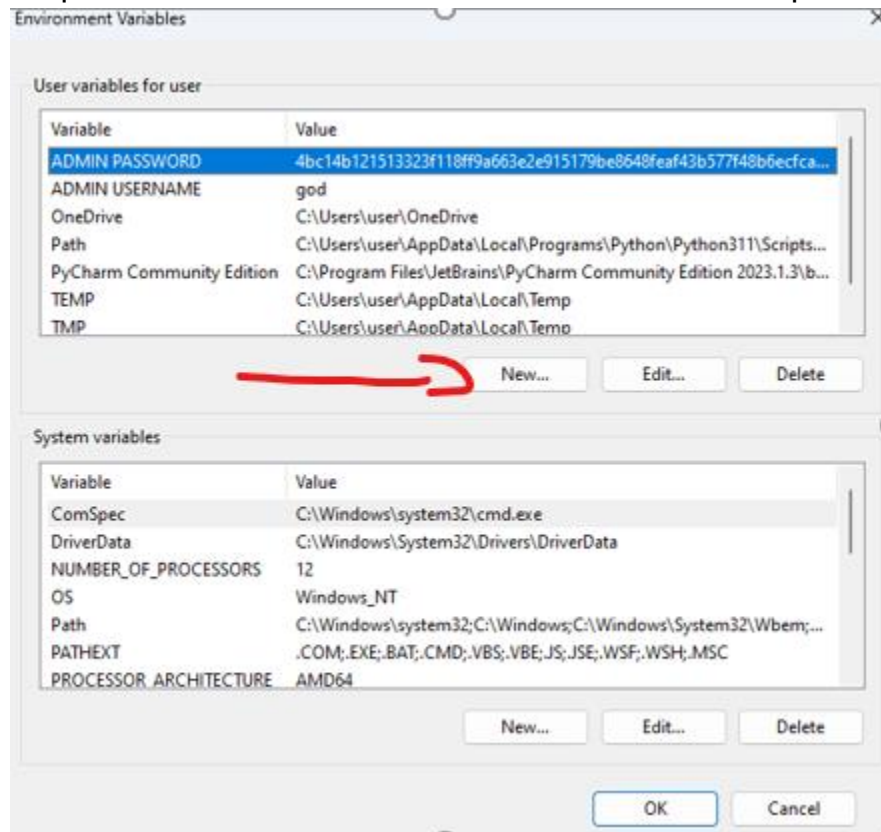


נתונים התחלתיים

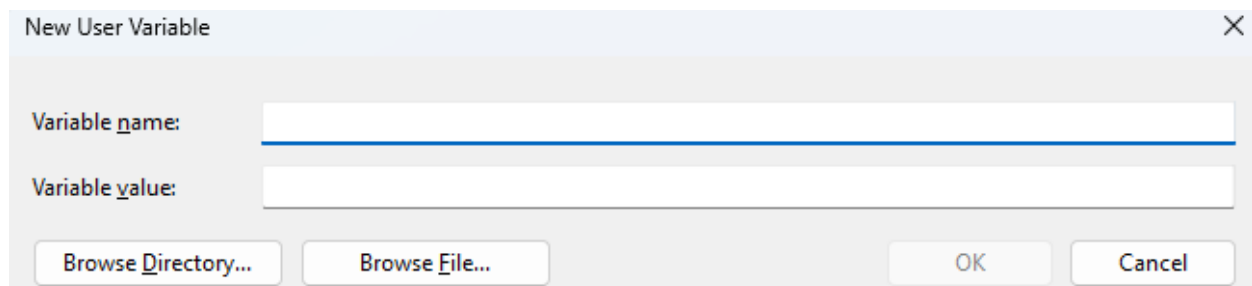
בקוד של השרת קיימים נתונים התחלתיים לגבי השפות וניתן לשנות אותם שם. אין צורך להיכנס למסד הנתונים ולשנות אותו. ולגבי שם המשתמש והסיסמה של האדמין לפני ההרצה של הפרויקט ללחוץ על הכפתור חיפוש של windows בשמאל למטה ולכתוב environment variables ולהיכנס במקום שבו ניתן לשנות את נתונים הללו:



לאחר לחיצה עליו יפתח מסך ויש ללחוץ על הכפתור environment variables. לאחר לחיצה אליו יפתח עוד מסך שבו יש לשים את הנתונים. כדי לשים את הנתונים יש ללחוץ על new



ויפתח מסך חדש



במקום של variable name יש לרשום ADMIN PASSWORD ובvalue יש לרשום 4bc14b121513323f118ff9a663e2e915179be8648feaf43b577f48b6ecfcac2c4 על מנת לשמור על הצפנה של הסיסמה

ועכשיו יש להוסיף עוד נתון, לכן יש להגיע לאותו מקום וללחוץ new שוב פעם.

ובמקום של variable name יש לרשום ADMIN USERNAME ובמקום של variable value יש לרשום god וזהו אפשר לסגור את זה.

רשת

על המחשבים להיות מחוברים לאותה רשת ומכיוון שהמערכת משתמשת בספריית socket, אשר דורשת כתובת IP על מנת ליצור חיבור, אז לפני הרצת המערכת יש להכניס לקוד הלקוח את כתובת ה IP המדויקת של השרת וגם לקוד השרת.

ארכיטקטורה מינימלית

בצד השרת:

- מעבד מודרני
- זיכרון (8GB לפחות, זה תלוי במספר המשתתפים בשרת)
- אחסון – תלוי במספר הנתונים במסד הנתונים אך 100 מגה בייט צריך להיות הרבה יותר ממספיק
- חיבור לאינטרנט
- מערכת הפעלה עדכנית (כמו windows 10 אך אפשר אחרות)

בצד הלקוח:

- מעבד מודרני
- זיכרון (4GB אמור להספיק)
- אחסון – לא צריך הרבה כלל, הרוב זה הקובץ עם המספרים הראשוניים (10 מגה מספיק אבל 100 מגה בייט ליתר ביטחון מספיק להרבה)
- חיבור לאינטרנט
- מערכת הפעלה עדכנית

משתמשי המערכת

לפני הרצת הפרויקט יש לוודא (ולשנות אם צריך) את המקום בserver_comm ובclient_comm שבו רשום את ה IP (HOST). כדי שהמערכת תעבוד חובה שהערך יהיה כתובת ה IP של המחשב שבו ממוקם השרת. ניתן למצוא את ה IP בפתיחת CMD וכתובת ipconfig לאחר מכן יופיעו על מיני מספרים, ויש לבחור את ארבעת המספרים שמחוברים עם נקודה ביניהן, שהמספר הראשון שלו הוא 10.

מדריך למנהל המערכת (צד השרת)

כדי להתחיל ולהריץ את המערכת:

1. יש להוריד את הספריות הנדרשות שאינן קיימות כרגע המוזכרות בצד השרת למעלה
2. להריץ רק את הקובץ server.py ובכך שלוחצים על החץ הימני למעלה בתוכנת pycharm או בלחיצת מקש ימינה ולחיצה על run server.py.

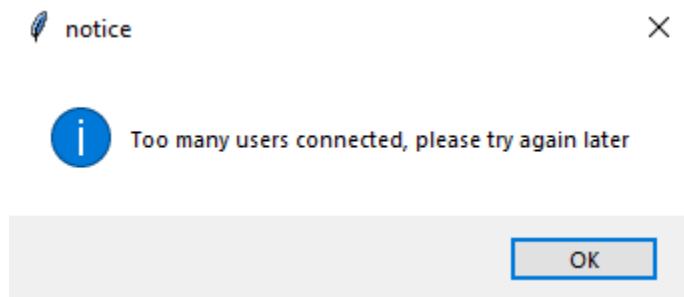
מדריך למשתמש המערכת

כדי להתחיל ולהריץ את המערכת:

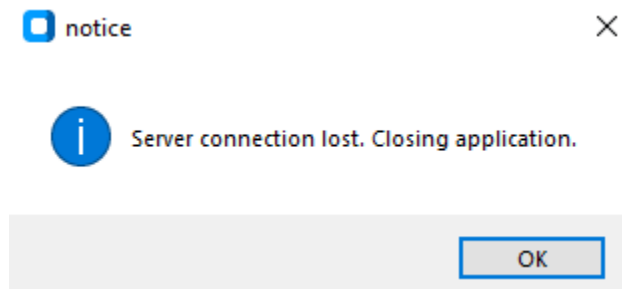
1. יש להוריד את הספריות הנדרשות שאינן קיימות כרגע המוזכרות בצד הלקוח למעלה

2. להריץ רק את הקובץ client.py ובכך שלוחצים על החץ הירוק בימין למעלה בתוכנת pycharm או בלחיצת מקש ימינה ולחיצה על client.py.run.

אם יש הרבה משתמשים מחוברים, יכול להיווצר עומס על השרת ולכן כדי למנוע DDoS יש מגבלה של משתמשים שיכולים להיות מחוברים, לכן אם משתמש מנסה להתחבר מעל המגבלה הוא יקבל את ההודעה הבאה ותיסגר התכנית:

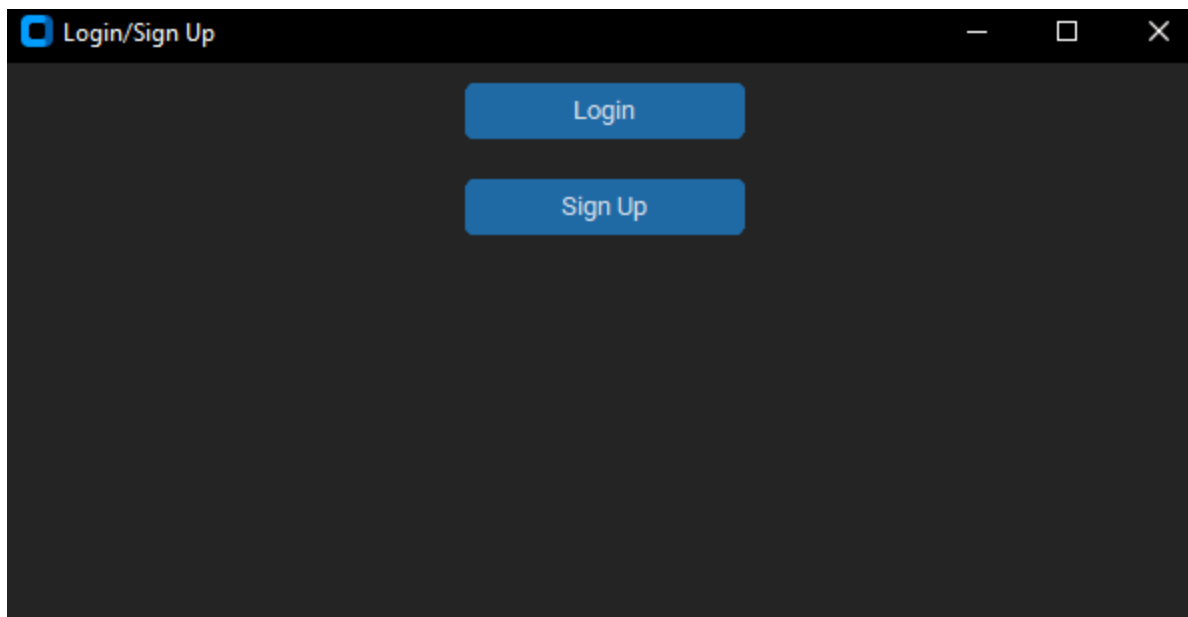


בנוסף אם השרת נסגר או קרס נשלחת ההודעה והתכנית תיסגר:



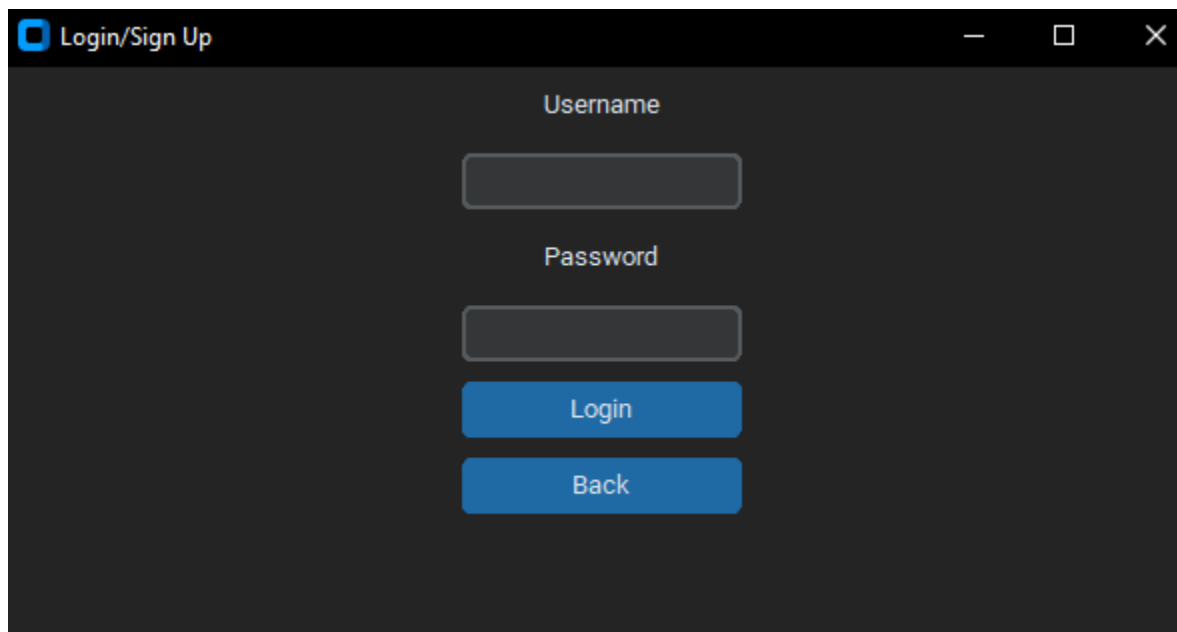
משתמש רגיל (לא אדמין)

לאחר הרצת הקובץ, יפתח הממשק הגרפי של ההתחברות והרשמה.



אם רוצים להתחבר למשתמש קיים יש ללחוץ על login ולהכניס את שם המשתמש והסיסמה. ואם רוצים להירשם עם משתמש חדש יש ללחוץ על signup ולהכניס את שם המשתמש והסיסמה החדשים, חובה להכניס סיסמה ראויה ומתאימה לפי ההוראות שרשומות על המסך ולא ניתן להשתמש בשם משתמש שכבר קיים.

התחברות:

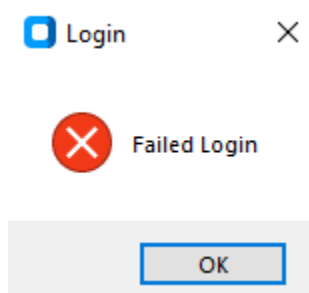


לאחר הכנסת הנתונים ולחיצה על login המשתמש יקבל הודעה.(ניתן גם ללחוץ על back על מנת לחזור למסך הקודם להרשמה).

אם ההתחברות מתאימה, תוצג ההודעה והמשתמש יעבור למסך הבית:



אם היא אינה מתאימה תוצג ההודעה והמשתמש ישאר במסך ההתחברות(יש אפשרות גם לחזור לאחור ולהירשם):



הודעה זו יכולה להיגרם אם שם המשתמש או הסיסמה שגויים, או שהמשתמש לא קיים, או שהמשתמש כבר מחובר למערכת ואין אפשרות להיכנס אליו בזמן שהוא מחובר.

הרשמה:

לאחר רשימת הנתונים המתאימים להרשמה ולחיצה על כפתור sign up תישלח הודעה.

אם הנתונים נכונים להרשמה יוצג ההודעה:

 Sign Up ×

 Sign up successful

OK

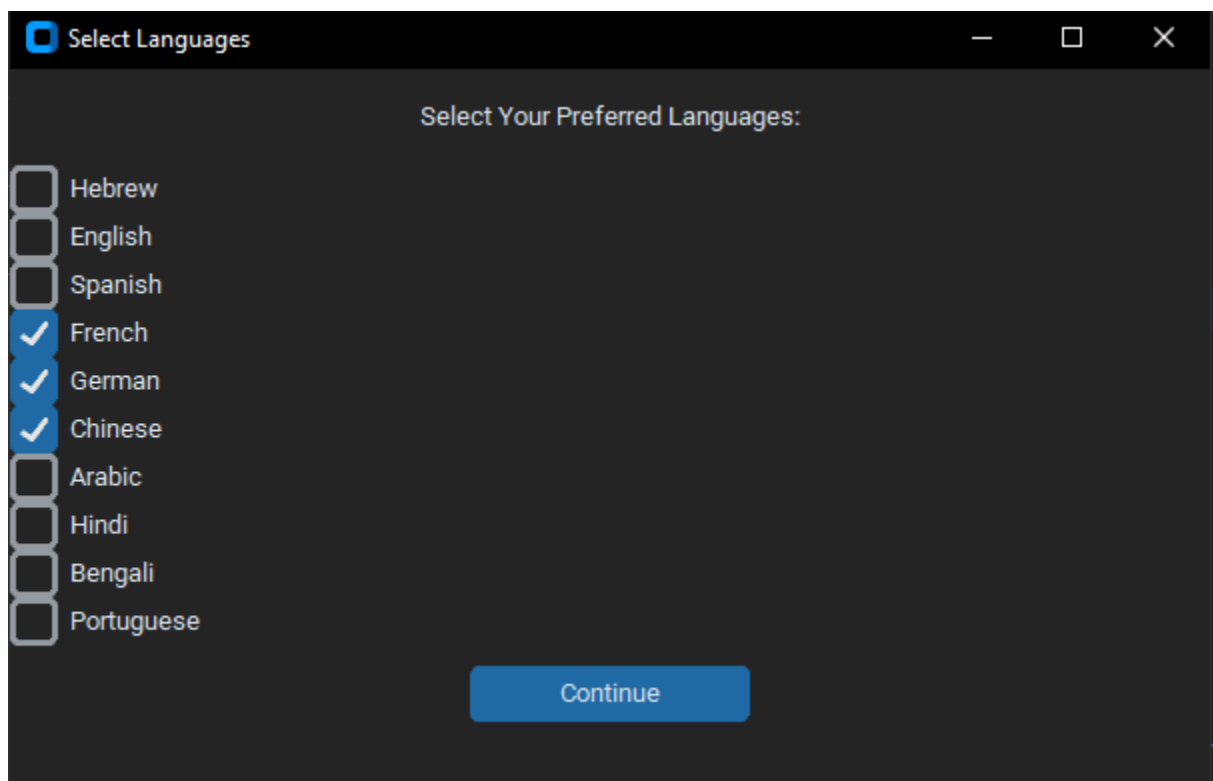
אם הנתונים אינם מתאימים תוצג ההודעה:

 Sign Up ×

 Username already exists or you need to use at least one character in the username and password

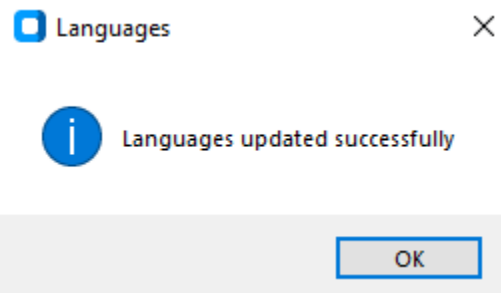
OK

אם הנתונים מתאימים יפתח המסך של הכנסת השפות. בו יש צורך לבחור לפחות שפה אחת שהשתמש יודע/רוצה לדבר איתה.

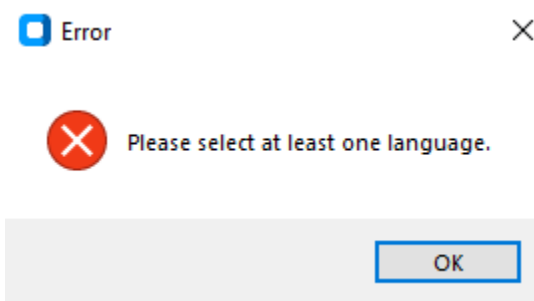


לאחר בחירת השפות ולחיצה על continue יוצג הודעה:

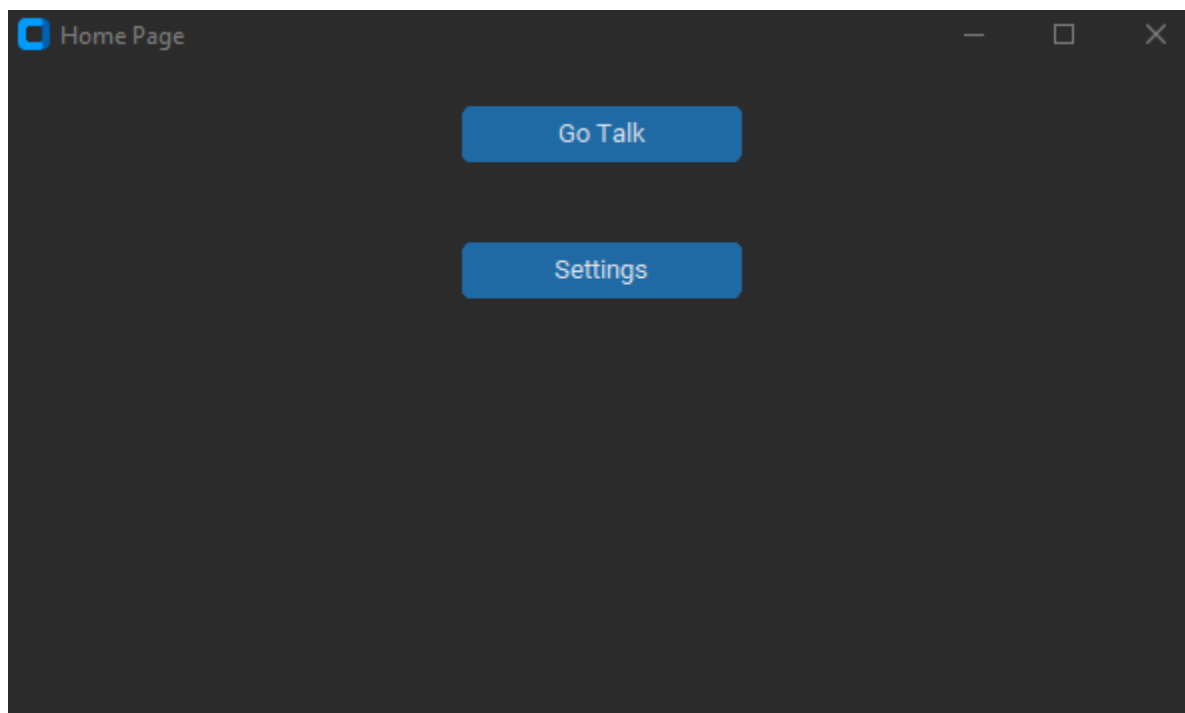
אם הבחירה נכונה המשתמש יעבור למסך הבית ויוצג ההודעה:



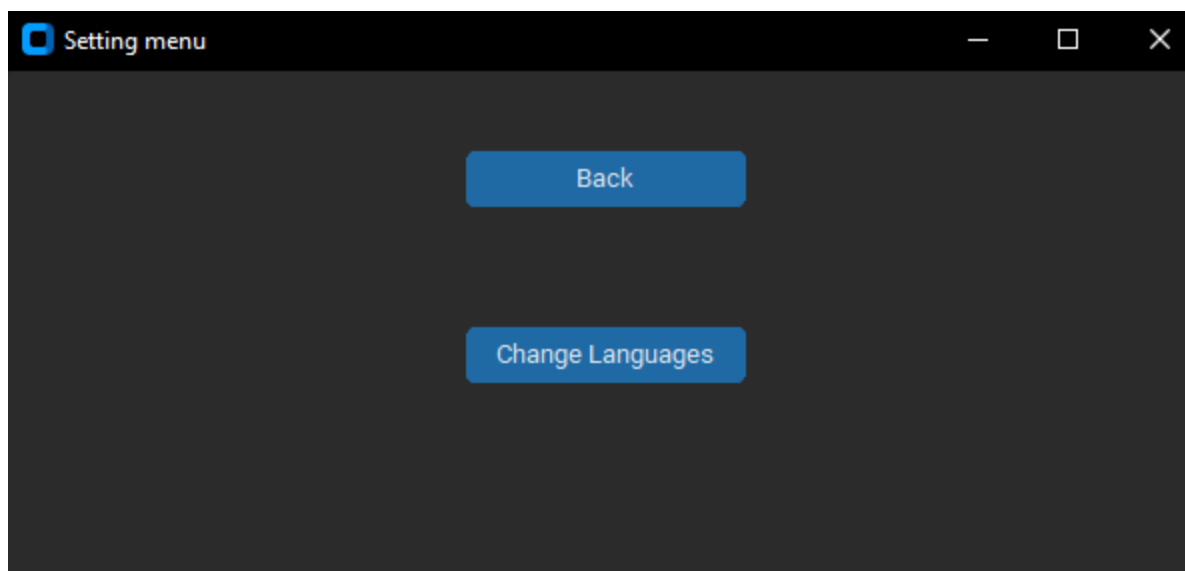
אם לא נחברה שפה המשתמש יקבל את ההודעה הבאה וישאר במסך השפות עד שיבחר שפה וימשיך למסך הבית:



מסך הבית:

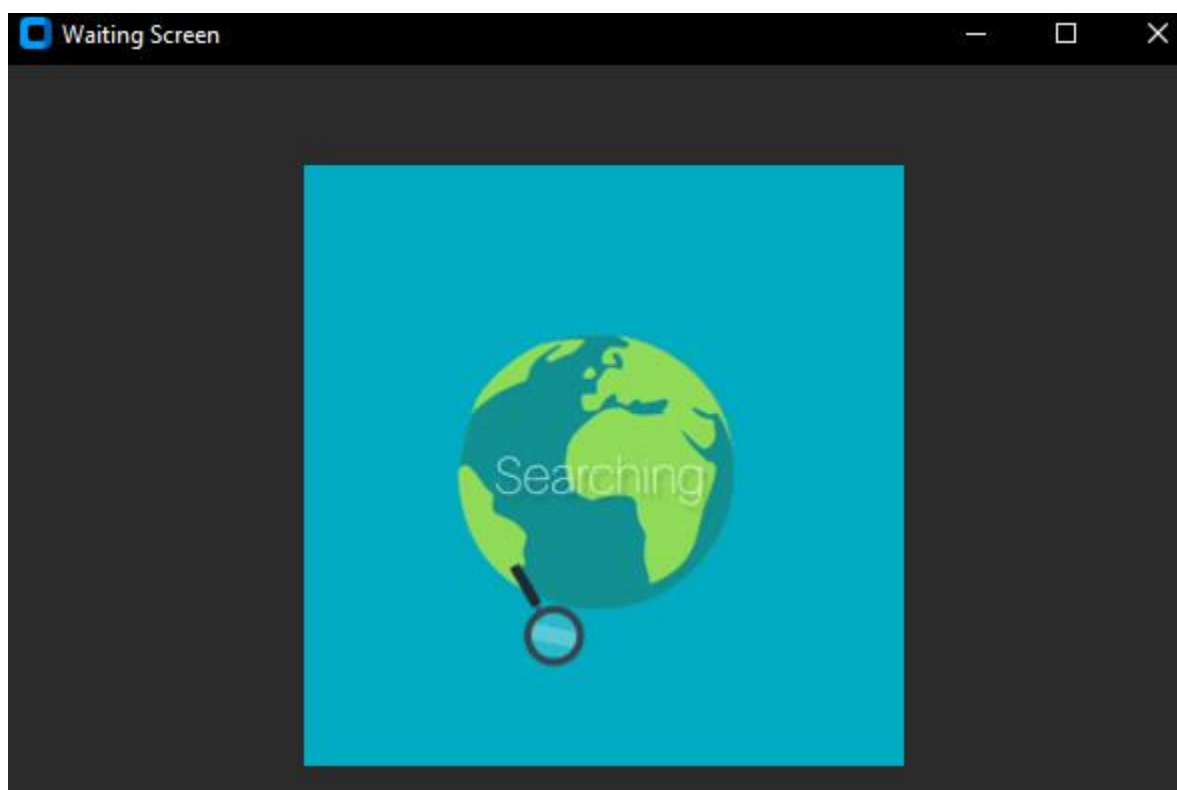


במסך הבית יש אופציה ללחוץ על settings ששם יש אפשרות לשנות את השפות שהמשתמש יודע/רוצה לדבר איתן.

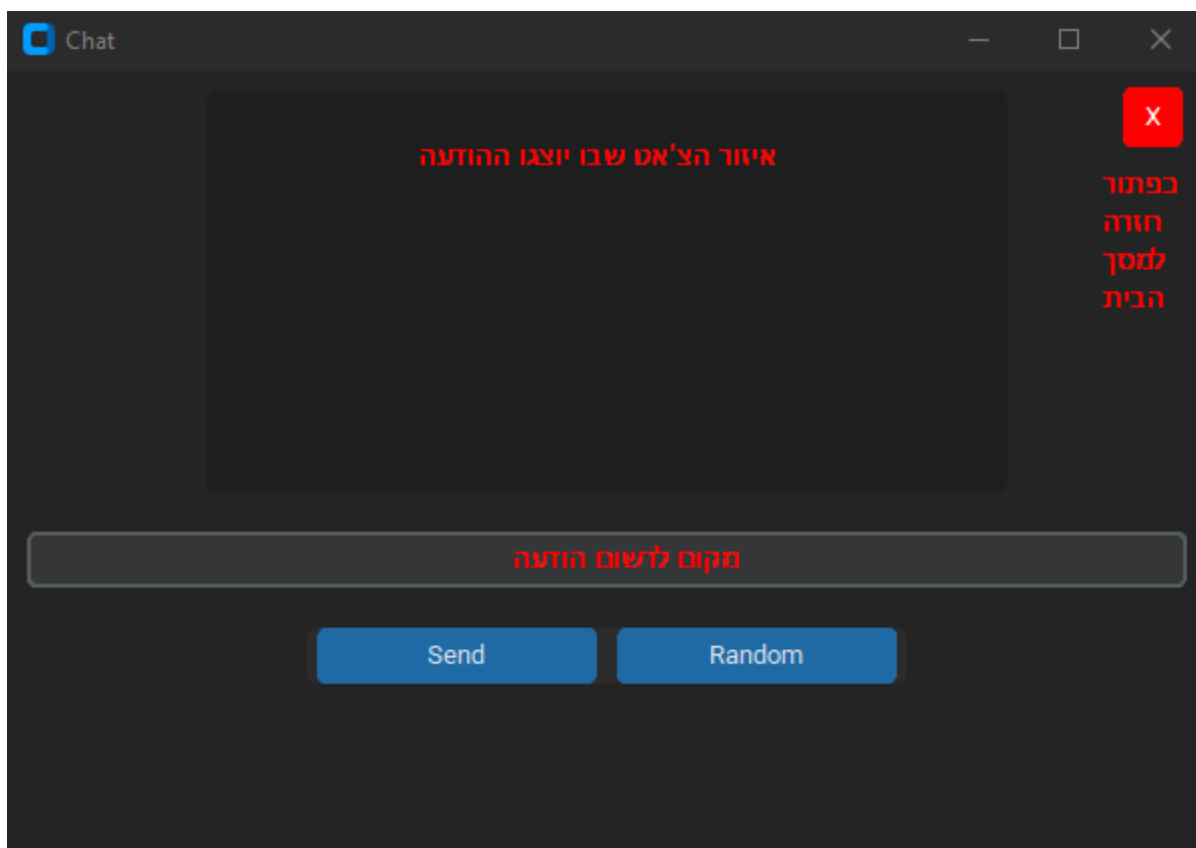


בלחיצה על change languages למשתמש יפתח אותו מסך כמו בתהליך ההרשמה שבו ניתן למלא את השפות ובסיומו יחזור למסך הבית.

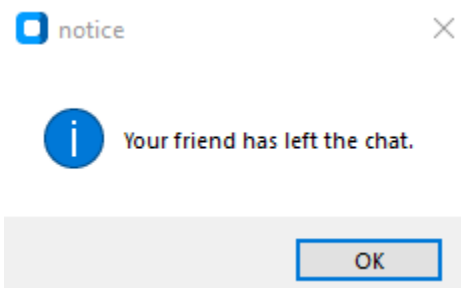
בלחיצה על כפתור go talk למשתמש יפתח מסך ההמתנה, בו המשתמש יחכה עד שהשרת ימצא לו משתמש ויפתח מסך הצ'אט.



במסך הצ'אט יש אזור של השיחה, ויש מקום שאפשר לרשום בו הודעה, לאחר רשימה של הודעה במקום שאפשר לרשום ולחיצה על כפתור send ההודעה תוצג על המסך ותישלח למשתמש האחר שנמצאים איתו בשיחה.

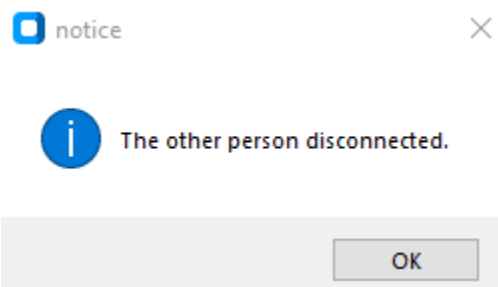


לאחר לחיצה על כפתור הX האדום בימין למעלה המשתמש יצא מהשיחה ויחזור למסך הבית. לאחר לחיצה על Random המשתמש יחזור למסך ההמתנה בו יחכה עד שהשרת יצוות לו מישהו חדש לשיחה. הכפתור בעצם מחליף את השיחה עם המשתמש הקיים למשתמש אחר אם קיים. אם המשתמש האחר התנתק מהשיחה או החליף שיחה תוצג ההודעה הבאה:



והמשתמש יחזור למסך הבית.

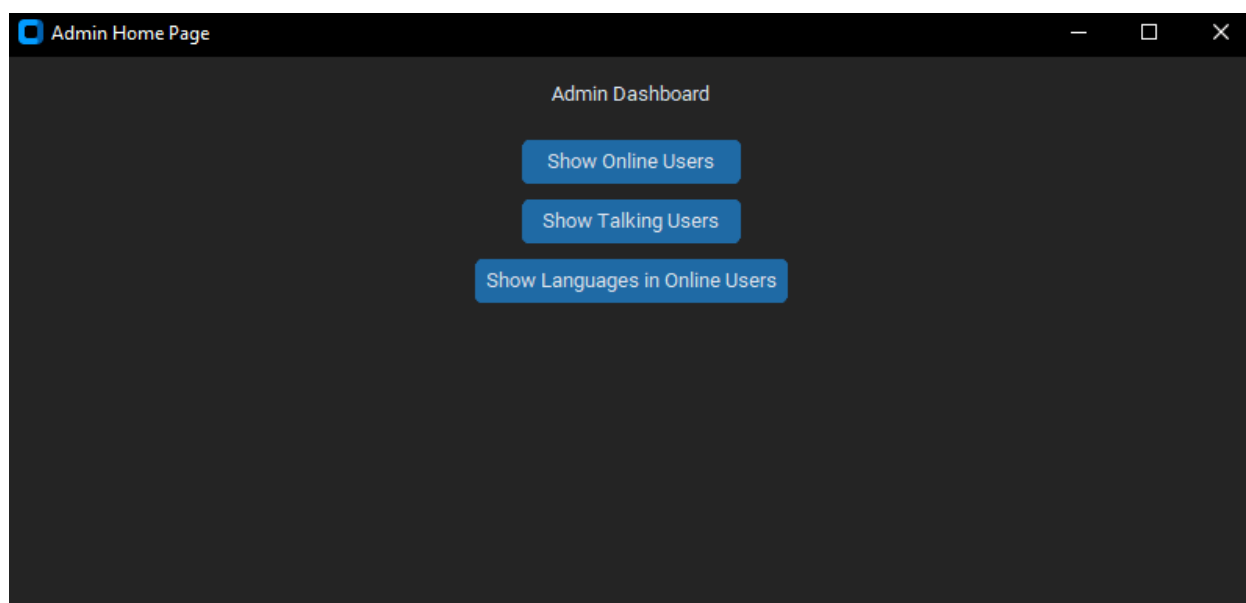
אם המשתמש האחר יצא מהמערכת בזמן השיחה תוצג ההודעה הבאה:



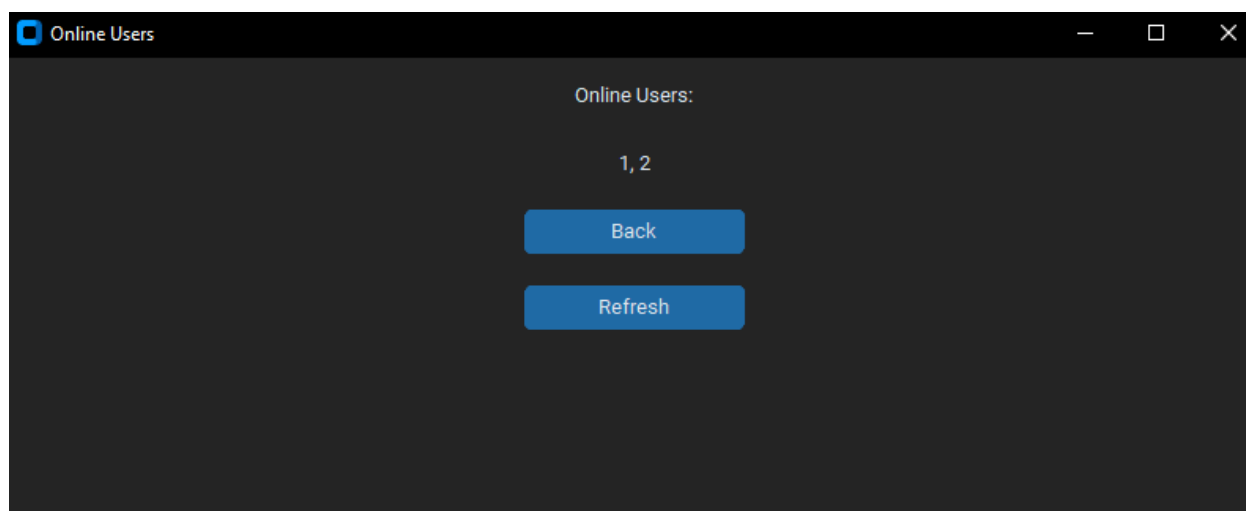
והמשתמש יחזור למסך הבית.

משתמש אדמין:

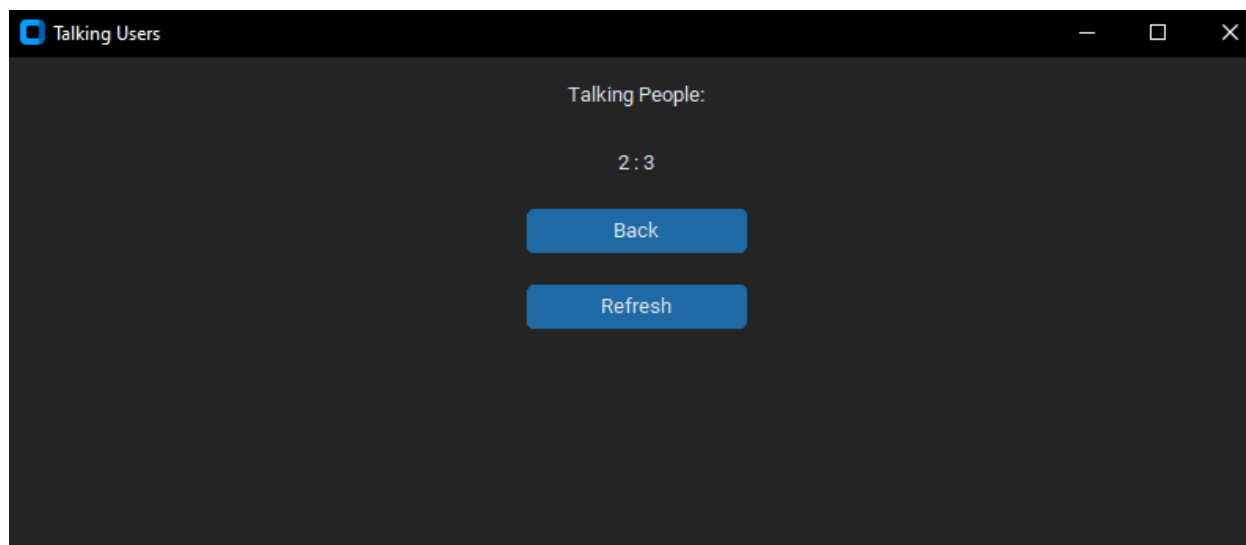
כדי להתחבר למשתמש אדמין חובה לעשות את השלב הראשון של התחברות כמו משתמש רגיל. לאחר הכנסת הנתונים של האדמין (שרק הצד של השרת מכיר והם נמצאים במסד הנתונים) תוצג הודעה של התחברות מוצלחת או שנכשלה בדיוק כמו בהתחברות של משתמש רגיל, ואז יפתח מסך הבית של האדמין.



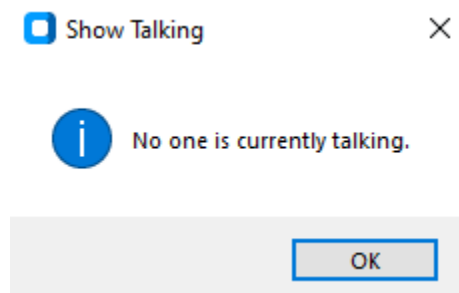
במסך הבית יש שלושה כפתורים, לאחר לחיצה על Show Online Users יפתח מסך בו יוצג המספר המזהה של כל משתמש מחובר (ID). המספר 1 תמיד יהיה מוצג מכיוון שזה המספר המזהה של האדמין.



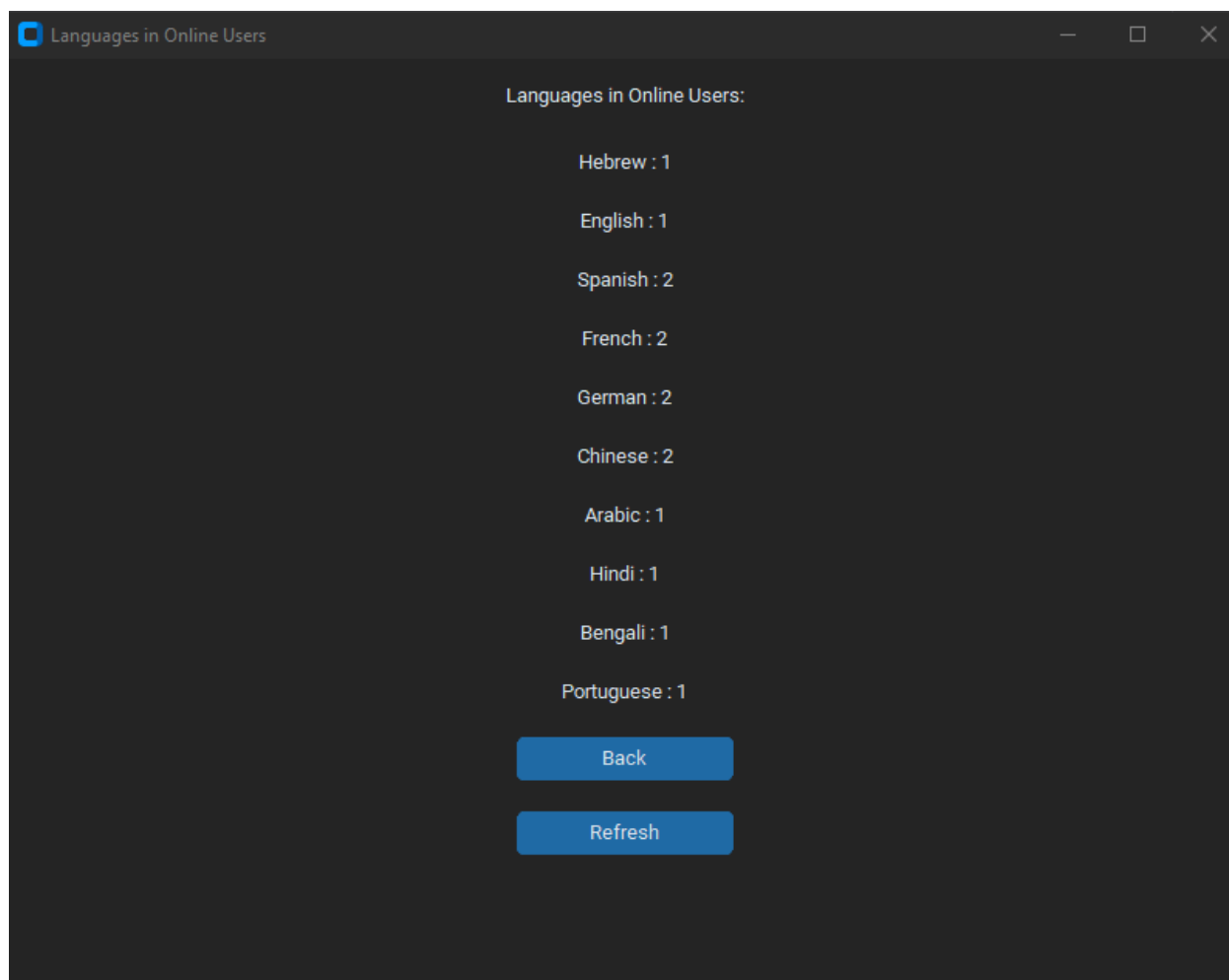
לאחר לחיצה על Show Talking Users האדמין יראה על המסך את המספרים המזהים של המשתמשים שכרגע בשיחה.



אם אין משתמשים שמדברים כרגע תוצג ההודעה:



לאחר לחיצה על Show Languages in Online Users יפתח מסך בו יהיה מוצג כמה משתמשים מחוברים יודעים כל שפה, לדוגמה: אם יש 2 משתמשים מחוברים, משתמש אחד שיועד את כל השפות ומשתמש אחר שיועד רק ספרדית, צרפתית, גרמנית וסינית, יוצג המסך הבא:



בנוסף בכל מסכי האדמין יש כפתור refresh שלאחר לחיצה עליו, מעדכן את המסך לפי הנתונים המתאימים אם משתמש התנתק או משתמש התחבר, או משתמשים נכנסו לשיחה או יצאו משיחה ועוד.

רפלקציה אישית

העבודה על הפרויקט הייתה קשה ומאתגרת מכמה סיבות, ראשית, סידור הזמנים בשילוב עם בית ספר, בגרויות נוספות ובשבילי גם פסיכומטרי שניגשתי במועד אפריל, ולכן היה עמוס מאוד וקשה לסדר את

הזמנים ולשלב בין הדברים. שנית, היה קושי בתחילת הפרויקט, מכיוון ששנה שעברה לא למדנו את כל החומר כמו שצריך, נאלצתי ללמוד קצת בחופש ואף חצי מהשנה הזאת נאלצנו ללמוד ולהשלים את רוב החומר שנצרך לחומר והיו הרבה מושגים וחומרים שלמדתי לבד בשל סיבה זאת כמו הצפנות, חלק מהתקשורת, חלק מהממשק הגרפי. בנוסף בגלל תחילת העבודה על הפרויקט המאוחרת, היה קושי נוסף בסידור הזמנים ובבחירת הנושא לפרויקט. אך לאחר סיום הפרויקט והעמידה בזמנים הייתה מאוד מתגמלת וגרמה לתחושת גאווה. כמו כן, הפרויקט הזה הוא פרויקט מאוד גדול יחסית לכל מה שעשינו עד עכשיו וזו הייתה קפיצה משמעותית בלמידה ובאתגר, הלמידה של איך לשלב את כל החלקים לפרויקט אחד וכל פעם להתקדם צעד צעד ולראות את ההתקדמות היא תחושה מדהימה שעזרה לי להתקדם. נעזרתי לא מעט בגוגל, קצת ai וסרטונים youtube. הדבר הכי חשוב שהייתי לוקח איתי להמשך הוא באמת כל הזמן להמשיך לנסות, ללמוד ולמצוא פתרון ולא להתחמק או לוותר, ולא להאמין לכל מה ש ai אומר ותמיד לבדוק ולעבור שוב ושוב. וכמובן שמאוד עזרו לי המנחים והמורים וגם השיחות עם חבריי למגמה שתרמו להתייעצויות רבות ולצפייה בסיטואציה מצד שונה משלי. בראייה לאחור היו כמה חלקים שהייתי משנה בפרויקט, דבר ראשון הייתי משנה את הדרך בה חיברתי זוג משתמשים לשיחה, במקום לעשות שהשרת יהיה באמצע ויעביר הודעות, הוא רק יצוות בין 2 משתמשים והם יוכלו לתקשר ישירות ביניהן, חיבור זה הוא חיבור P2P (peer to peer) וכך אפשר לשנות רק את נתוני התקשורת (IP, PORT) וכך לחבר 2 משמשים אחד לשני. ולאחר שאחד מהם מתנתק האחר היה חוזר לתקשורת עם השרת עד שימצא לו מישהו אחר, פתרון זה הרבה יותר יעיל וחוסך עומס רב של הודעות על השרת. בנוסף הייתי משנה ומוסיף עוד מחלקות ומסדר את הקבצים שונה. אם היו עוד משאבים וזמן לעבודה על הפרויקט הייתי מוסיף אפשרות לדרג את האפליקציה ואת השיחות, ולדווח עד משתמשים בעייתיים, מוסיף עוד נתונים שהאדמין יוכל לראות, ואולי אפילו לתת לו הרשאות כמו להעיק משתמשים וגם עושה עוד סטטיסטיקות לאפליקציה כמו מספר הכולל של משתמשים שהתחברו אי פעם, או המשתמש שהתחבר הכי הרבה פעם, הזמן של השיחה הכי ארוכה ועוד. כמו כן, הייתי רוצה להפוך את הפרויקט לאפליקציה שאפשר להשתמש בה בטלפון, מכיוון שאני מרגיש שהפרויקט הרבה יותר מותאם לטלפון. אני רוצה להודות לחבריי שעזרו לי לבדוק את הפרויקט והפכו את העבודה על הפרויקט להרבה יותר מהנה. להודות לכלל המנחים, עידן פלדברג, רום רפסון, ליאת סגל, עודי רז ויעקב שוצמן שעזרו תמיד, תמכו ונשארו שעות רבות בבית ספר ואחרי בית ספר לעזור ולענות על השאלות.

ביבליוגרפיה

[ספר מערכות הפעלה](#)

[סרטון על מסד הנתונים](#)

[קורס על יסודות וכלים לשליפת נתונים](#)

[פעולות ולמידה על שפת SQL](#)

[קורס next.py](#)

[קורס על תקשורת](#)

[למידה על customtkinter פעולות ואפשרויות](#)

[הסבר על sha256](#)

נספחים

protocol code:

```
# Protocol Constants

CMD_FIELD_LENGTH = 16
LENGTH_FIELD_LENGTH = 4
MAX_DATA_LENGTH = 10 ** LENGTH_FIELD_LENGTH - 1
MSG_HEADER_LENGTH = CMD_FIELD_LENGTH + 1 + LENGTH_FIELD_LENGTH + 1
MAX_MSG_LENGTH = MSG_HEADER_LENGTH + MAX_DATA_LENGTH
DELIMITER = chr(29)
DATA_DELIMITER = chr(31)

COMMAND_CLIENT = {
    "login": "0000",
    "logout": "0001",
    "logged": "0002",
    "talk": "0003",
    "signup": "0004",
    "select_languages": "0005",
    "change_person": "0006",
    "waiting": "0007",
    "done_talking": "0008",
    "give_key": "0009",
    "ping": "0010"
}

COMMAND_CLIENT_ADMIN = {
    "show_online" : "5000",
    "show_talking" : "5001",
    "show_languages_online" : "5002"
}

COMMAND_SERVER = {
    "login_ok": "1000",
    "login_failed": "1001",
    "logout_ok": "1002",
    "signup_ok": "1003",
    "signup_failed": "1004",
    "logged_ok": "1005",
    "languages_ok": "1006",
    "languages_error": "1007",
    "talk_ok": "1008",
    "forward_talk": "1009",
    "change_person_ok": "1010",
    "change_person_error": "1011",
    "waiting_stop": "1012",
    "keep_waiting": "1013",
    "friend_left": "1014",
    "give_key": "1015",
    "client_left_server": "1016",
    "show_online_ok": "1017",
    "show_talking_ok": "1018",
    "show_languages_online_ok": "1019",
    "done_talking_ok": "1020",
```

```

        "error": "9999"
    }

ERROR_RETURN = None

#this function builds a message based on the protocol
def build_message(cmd, data):
    if not isinstance(cmd, str) or not isinstance(data, str) or len(cmd) >=
CMD_FIELD_LENGTH:
        return None
    try:
        message = f"{cmd.ljust(4)}{DELIMITER}{data}"
        return message
    except Exception as e:
        print(f"Error constructing the message: {e}")
        return None

#this function parses a message based on the protocol
def parse_message(message):
    parts = message.split(DELIMITER)
    if len(parts) != 2:
        return None, None

    cmd = parts[0].strip()
    data = parts[1]

    return cmd, data

def split_data(data, expected_fields):
    fields = data.split(DATA_DELIMITER)
    if len(fields) != expected_fields:
        return None
    return fields

```

rsa code:

```

import random

# Euclid's algorithm for determining the greatest common divisor
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

# Euclid's extended algorithm for finding the multiplicative inverse of two
numbers
# function for extended Euclidean Algorithm
def multiplicative_inverse(e, phi):
    d = 0
    x1 = 0

```

```

x2 = 1
y1 = 1
temp_phi = phi

while e > 0:
    temp1 = temp_phi//e
    temp2 = temp_phi - temp1 * e
    temp_phi = e
    e = temp2

    x = x2- temp1* x1
    y = d - temp1 * y1

    x2 = x1
    x1 = x
    d = y1
    y1 = y

if temp_phi == 1:
    return d + phi

def generate_keypair(p, q):
    n = p * q

    #Phi is the totient of n
    phi = (p-1) * (q-1)

    #Choose an integer e such that e and phi(n) are coprime
    e = random.randrange(2, phi)

    #Use Euclid's Algorithm to verify that e and phi(n) are coprime
    g = gcd(e, phi)
    while g != 1:
        e = random.randrange(2, phi)
        g = gcd(e, phi)
    #Use Extended Euclid's Algorithm to generate the private key
    d = multiplicative_inverse(e, phi)

    #Return public and private keypair
    #Public key is (e, n) and private key is (d, n)
    return ((e, n), (d, n))

def encrypt(pk, plaintext):
    key, n = pk
    #Convert each letter in the plaintext to numbers based on the character
    using a^b mod m
    cipher = [pow(ord(char), key, n) for char in plaintext]
    #Return the array of bytes
    return cipher

def decrypt(pk, ciphertext):
    #Unpack the key into its components
    key, n = pk
    #Generate the plaintext based on the ciphertext and key using a^b mod m
    plain = [chr(pow(char, key, n)) for char in ciphertext]

```

```
#Return the array of bytes as a string
return ''.join(plain)

def generate_primes():
    f = open(r"primes.txt", "r")
    primes = f.read()
    primes_list = primes.split(",")
    n1 = random.randrange(10000, 30000)
    # 664579
    prime1 = primes_list[n1]
    n2 = random.randrange(10000, 30000)
    while n1 == n2:
        n2 = random.randrange(10000, 30000)
    prime2 = primes_list[n2]
    return int(prime1), int(prime2)
```

db_manager code:

```
import sqlite3
import hashlib
import os

class ChatDatabase:
    database_name = 'RandChat.db'
    def __init__(self):
        conn = sqlite3.connect(self.database_name)
        c = conn.cursor()
        c.execute('''CREATE TABLE IF NOT EXISTS "users" ("username" TEXT NOT
NULL UNIQUE, "password" TEXT NOT NULL , "languages" TEXT,
"ID" INTEGER NOT NULL , PRIMARY KEY("ID"));''')
        c.execute('''CREATE TABLE IF NOT EXISTS "languages" ("ID" INTEGER,
"language" TEXT NOT NULL UNIQUE,
PRIMARY KEY("ID" AUTOINCREMENT));''')
        c.execute('''CREATE TABLE IF NOT EXISTS "user_and_languages"
("user_id" INTEGER,"language_id" INTEGER,
FOREIGN KEY("language_id") REFERENCES "languages"("ID"),FOREIGN
KEY("user_id") REFERENCES "users"("ID"),
PRIMARY KEY("user_id","language_id"));''')

        conn.commit()
        conn.close()

class Activatedb(ChatDatabase):
    def __init__(self):
        super().__init__()

    def insert_languages_and_admin(self, languages):
```

```

        conn = sqlite3.connect(self.database_name)
        c = conn.cursor()
        for language in languages:
            try:
                c.execute("INSERT INTO languages (language) VALUES (?)",
(language,))
            except sqlite3.IntegrityError:
                # Language already exists, skip it
                continue
            try:
                username = os.getenv("ADMIN USERNAME")
                password = os.getenv("ADMIN PASSWORD")
                c.execute("INSERT INTO users (username, password) VALUES (?, ?)",
(username, password))
            except sqlite3.IntegrityError:
                pass
                # problem inserting the admin or admin already exists
        conn.commit()
        conn.close()

    def login(self, username, password):
        conn = sqlite3.connect(self.database_name)
        c = conn.cursor()
        enc = hashlib.sha256(password.encode()).hexdigest()
        c.execute("SELECT ID FROM users WHERE username=? AND password=?",
(username, enc))
        user_id = c.fetchone()
        conn.close()
        if user_id:
            return user_id[0]
        else:
            return None

    def signup(self, username, password):
        conn = sqlite3.connect(self.database_name)
        c = conn.cursor()
        # Check if either the username or password already exists in the
database
        enc = hashlib.sha256(password.encode()).hexdigest()
        c.execute("SELECT ID FROM users WHERE username=? OR password=?",
(username, enc))
        existing = c.fetchone()
        if existing:
            # Duplicate found; do not create a new record.
            conn.close()
            return None

        try:
            enc = hashlib.sha256(password.encode()).hexdigest()
            c.execute("INSERT INTO users (username, password) VALUES (?, ?)",
(username, enc))
            conn.commit()
            user_id = c.lastrowid
        except sqlite3.IntegrityError:
            user_id = None
        conn.close()
        return user_id

```



```

def language_update(self, languages, id):
    conn = sqlite3.connect(self.database_name)
    c = conn.cursor()

    try:
        c.execute("UPDATE users SET languages=? WHERE ID=?", (languages,
id))

        conn.commit()

        languages = languages.strip("[]").replace("'", "").split(",")
        selected_languages = [language.strip() for language in languages]

        c.execute("SELECT language_id FROM user_and_languages WHERE
user_id=?", (id,))
        current_languages = c.fetchall()
        current_language_ids = [lang[0] for lang in current_languages]

        # Identify languages to delete
        languages_to_delete = set(current_language_ids) -
set(selected_languages)
        for language_id in languages_to_delete:
            c.execute("DELETE FROM user_and_languages WHERE user_id=? AND
language_id=?", (id, language_id))

        for language in selected_languages:
            # Retrieve the language ID from the languages table
            c.execute("SELECT ID FROM languages WHERE language=?",
(language,))

            language_id = c.fetchone()

            if language_id:
                # Check if the entry already exists
                c.execute("SELECT * FROM user_and_languages WHERE
user_id=? AND language_id=?", (id, language_id[0]))
                existing_entry = c.fetchone()
                if existing_entry:
                    continue # Skip insertion if it already exists

            try:
                c.execute("INSERT INTO user_and_languages (user_id,
language_id) VALUES (?, ?)", (id, language_id[0]))
                conn.commit()
            except sqlite3.IntegrityError as e:
                print(f"IntegrityError: {e} - Could not insert
user_id: {id}, language_id: {language_id[0]}")
            else:
                return False # Return false if any language is not found

        result = True
    except sqlite3.IntegrityError:
        result = False
    conn.close()
    return result

def get_languages(self, user_id):

```

```

        try:
            conn = sqlite3.connect(self.database_name)
            c = conn.cursor()

            c.execute("SELECT language_id FROM user_and_languages WHERE
user_id = ?", (user_id,))
            language_ids = c.fetchall()

            return [lang[0] for lang in language_ids]

        except sqlite3.Error as e:
            print(f"Database error: {e}")
            return []
        except Exception as e:
            print(f"Unexpected error: {e}")
            return []

    def check_user(self, user_id):
        conn = sqlite3.connect(self.database_name)
        c = conn.cursor()

        try:
            # Check if the user exists and retrieve their details
            c.execute("SELECT username, password, languages FROM users WHERE
ID=?", (user_id,))
            user_details = c.fetchone()

            if user_details:
                username, password, languages = user_details
                # Check if username, password, and languages are not None or
empty
                if username and password and languages or int(user_id) == 1:
                    return True
                return False
            except sqlite3.Error as e:
                print(f"Database error: {e}")
                return False
            finally:
                conn.close()

    def remove_user(self, user_id):
        conn = sqlite3.connect(self.database_name)
        c = conn.cursor()

        try:
            # Delete the user from the users table
            c.execute("DELETE FROM users WHERE ID=?", (user_id,))
            conn.commit()

            # Check if any rows were affected (i.e., user was deleted)
            if c.rowcount > 0:
                return True
            else:
                # No user found with ID
                return False
        except sqlite3.Error as e:
            print(f"Database error: {e}")

```

```
        return False
    finally:
        conn.close()
```

client code:

```
from gui import ChatGUI
import protocol as proto
from client_comm import Comm
import threading
import time
import select

cmd_get, data_get = "", ""
cmd_send, data_send = "", ""
client_comm = Comm()
gui_chat = ChatGUI()
if client_comm.status != "Connected":
    gui_chat.pop_message("Too many users connected, please try again later")
    client_comm.client.close()
is_listening = False

def monitor_server():
    global client_comm, gui_chat
    while True:
        # Attempt to send a simple message to check server status
        client_comm.build_and_send_message(proto.COMMAND_CLIENT["ping"], "")
        if client_comm.status == "Denied":
            client_comm.client.close()
            gui_chat.pop_message("Server connection lost. Closing application.")
            gui_chat.destroy() # Close the GUI
            time.sleep(5) # Check every 5 seconds

def handle_request_message():
    global cmd_send, data_send, gui_chat
    if cmd_send == proto.COMMAND_CLIENT["login"]:
        login()
    elif cmd_send == proto.COMMAND_CLIENT["signup"]:
        signup()
    elif cmd_send == proto.COMMAND_CLIENT["select_languages"]:
        change_languages()
    elif cmd_send == proto.COMMAND_CLIENT["talk"]:
        send_message()
    elif cmd_send == proto.COMMAND_CLIENT["change_person"]:
```

```

        change_person()
    elif cmd_send == proto.COMMAND_CLIENT["waiting"]:
        waiting()
    elif cmd_send == proto.COMMAND_CLIENT["done_talking"]:
        back_home()
    elif cmd_send == proto.COMMAND_CLIENT_ADMIN["show_online"]:
        show_online()
    elif cmd_send == proto.COMMAND_CLIENT_ADMIN["show_talking"]:
        show_talking()
    elif cmd_send == proto.COMMAND_CLIENT_ADMIN["show_languages_online"]:
        show_languages_online()
    else:
        gui_chat.pop_message("Unknown command")

def show_online():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get
    client_comm.build_and_send_message(cmd_send, data_send)

    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.show_online_people_ans(cmd_get, data_get)

def show_talking():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get
    client_comm.build_and_send_message(cmd_send, data_send)

    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.show_talking_people_ans(cmd_get, data_get)

def show_languages_online():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get
    client_comm.build_and_send_message(cmd_send, data_send)

    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.show_languages_in_online_users_ans(cmd_get, data_get)

#login with an existing account enter the program
def login():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get
    client_comm.build_and_send_message(cmd_send, data_send)

    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.login_ans(cmd_get, data_get)
    if gui_chat.ID and int(gui_chat.ID) == 1:
        gui_chat.show_admin_screen()

#signup with a new account enter the program
def signup():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get
    client_comm.build_and_send_message(cmd_send, data_send)

    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.signup_ans(cmd_get, data_get)

def change_languages():
    global cmd_send, data_send, client_comm, gui_chat, cmd_get, data_get

```

```

client_comm.build_and_send_message(cmd_send, data_send)

cmd_get, data_get = client_comm.recv_message_and_parse()
gui_chat.languages_ans(cmd_get)

def send_message():
    global cmd_send, data_send, gui_chat, client_comm
    gui_chat.show_and_return_message()
    client_comm.build_and_send_message(cmd_send, data_send)
    gui_chat.outgoing_msg = None

def back_home():
    global cmd_send, data_send, gui_chat, client_comm, is_listening
    is_listening = False
    client_comm.build_and_send_message(cmd_send, data_send)
    gui_chat.outgoing_msg = None

def change_person():
    global cmd_send, data_send, gui_chat, client_comm, is_listening, cmd_get,
    data_get
    is_listening = False
    client_comm.build_and_send_message(cmd_send, data_send)
    cmd_get, data_get = client_comm.recv_message_and_parse()
    gui_chat.change_person_ans(cmd_get)

def waiting():
    global cmd_send, data_send, client_comm, gui_chat, is_listening, cmd_get,
    data_get
    client_comm.build_and_send_message(cmd_send, data_send)
    cmd_get, data_get = client_comm.recv_message_and_parse()
    if cmd_get == proto.COMMAND_SERVER["waiting_stop"]:
        gui_chat.is_animating = False
        gui_chat.outgoing_msg = None
        gui_chat.create_chat_interface() # Switch to chat interface
        gui_chat.is_chat_window_open = True
        is_listening = True

#a thread that runs while the client is talking and waits for messages from
the other client
def listen_to_friend():
    global client_comm, gui_chat, cmd_send, data_send, is_listening, cmd_get,
    data_get
    # Use a non-blocking approach to check for incoming messages
    while True:
        try:
            if is_listening and client_comm.status == "Connected":
                # Check for incoming messages
                r, _, _ = select.select([client_comm.client], [], [])
                if r:
                    cmd_get, data_get = client_comm.recv_message_and_parse()
                    if cmd_get is not None: # If a command is received
                        if cmd_get == proto.COMMAND_SERVER["forward_talk"]:

```

```

        gui_chat.display_message(f"Friend: {data_get}")
        cmd_get, data_get = None, None
    elif cmd_get == proto.COMMAND_SERVER["error"] or
cmd_get == proto.COMMAND_SERVER["friend_left"]:

        gui_chat.pop_message(data_get)
        gui_chat.create_home_page() # Return to start

interface

        is_listening = False
        time.sleep(0.1) # Add a small delay to prevent busy waiting
    except Exception as e:
        print(f"Error in listen_to_friend: {e}")

# thread that check if the client wants to send something or do a command
def comm_thread():
    global cmd_send, data_send, client_comm, gui_chat
    while True:
        time.sleep(0.1)
        if gui_chat.outgoing_msg:
            cmd_send, data_send = gui_chat.outgoing_msg
            handle_request_message()

if __name__ == "__main__":
    threading.Thread(target=comm_thread, daemon=True).start()
    threading.Thread(target=listen_to_friend, daemon=True).start()
    threading.Thread(target=monitor_server, daemon=True).start()
    try:
        if client_comm.status == "Connected":
            gui_chat.mainloop() # Start the GUI application
    except KeyboardInterrupt:
        client_comm.client.close()
    except Exception as e:
        client_comm.client.close()

```

server code:

```

import time
from db_manager import Activatedb
from server_comm import Comm
import protocol as proto
import threading
import json

talking = {}
waiting_clients = []
logged_users_id = {}
last_talked = {}

client, cmd, data = None, "", ""
l = threading.Lock()

```

```

languages_online = {
    "Hebrew" : 0,
    "English" : 0,
    "Spanish" : 0,
    "French" : 0,
    "German" : 0,
    "Chinese" : 0,
    "Arabic" : 0,
    "Hindi" : 0,
    "Bengali" : 0,
    "Portuguese" : 0
}

database = Activatedb()
database.insert_languages_and_admin(languages_online.keys())
server_comm = Comm()

def handle_client_message():
    global client, cmd, data
    if cmd == proto.COMMAND_CLIENT["login"]:
        handle_login_message()
    elif cmd == proto.COMMAND_CLIENT["signup"]:
        handle_signup_message()
    elif cmd == proto.COMMAND_CLIENT["select_languages"]:
        handle_languages_message()
    elif cmd == proto.COMMAND_CLIENT["talk"]:
        handle_talk_message()
    elif cmd == proto.COMMAND_CLIENT["change_person"]:
        handle_change_person_message()
    elif cmd == proto.COMMAND_CLIENT["waiting"]:
        handle_waiting_message()
    elif cmd == proto.COMMAND_CLIENT["done_talking"]:
        handle_stop_talking()
    elif cmd == proto.COMMAND_SERVER["client_left_server"]:
        handle_client_disconnection()
    elif cmd == proto.COMMAND_CLIENT_ADMIN["show_online"]:
        handle_show_online()
    elif cmd == proto.COMMAND_CLIENT_ADMIN["show_talking"]:
        handle_talking_online()
    elif cmd == proto.COMMAND_CLIENT_ADMIN["show_languages_online"]:
        handle_languages_online()
    elif cmd == proto.COMMAND_CLIENT["ping"]:
        pass
    else:
        handle_error_message()

#ADMIN FUNCTIONS
def handle_show_online():
    global client, logged_users_id, server_comm
    server_comm.outgoing_msg.append((client,
    proto.COMMAND_SERVER["show_online_ok"], ','.join(map(str,
list(logged_users_id.values()))).replace(",","")))

def handle_talking_online():

```

```

global client, talking, logged_users_id, server_comm
talking_id = {}
for client1 in talking:
    client2 = talking[client1]
    if client2 is not None and logged_users_id[client1] <
logged_users_id[client2]:
        talking_id[logged_users_id[client1]] = logged_users_id[client2]
    if talking_id: # Ensure talking_id is not empty
        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["show_talking_ok"], json.dumps(talking_id)))
    else:
        # No talking IDs to send
        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["show_talking_ok"], ""))

def handle_languages_online():
    global client, languages_online, server_comm
    update_languages_online()
    server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["show_languages_online_ok"],
json.dumps(languages_online)))

def update_languages_online():
    global languages_online, logged_users_id, database

    # Reset the counts for each language to zero
    for language in languages_online.keys():
        languages_online[language] = 0
    # Iterate through each logged user
    for user_id in logged_users_id.values():
        # Get the languages known by the user
        user_languages = database.get_languages(user_id)

        # Increment the count for each language the user knows
        for lang_id in user_languages:
            # Get the language name using the lang_id as the index
            language_name = list(languages_online.keys())[lang_id-1] #
Convert keys to a list and access by index
            if language_name in languages_online:
                languages_online[language_name] += 1

# USER FUNCTIONS
def handle_error_message():
    global client
    server_comm.outgoing_msg.append((client, proto.COMMAND_SERVER["error"],
"Unknown command"))

def handle_client_disconnection():
    # Notify the other client that this client has disconnected
    global client, server_comm, database ,talking, waiting_clients,
logged_users_id, last_talked
    if client not in server_comm.clients:

        if client in logged_users_id.values() and not
database.check_user(logged_users_id[client]):
            database.remove_user(logged_users_id[client])

```



```

        if client in talking.values() and talking[client] is not None:
            talking[talking[client]] = None
            server_comm.build_and_send_message(talking[client],
proto.COMMAND_SERVER["error"], "The other person disconnected.")
            talking.pop(client)
            last_talked.pop(client)

        if client in waiting_clients:
            waiting_clients.remove(client)

        if client in logged_users_id.keys():
            logged_users_id.pop(client)

def handle_login_message():
    global database, client, data, cmd, server_comm, logged_users_id, talking
    username, password = proto.split_data(data, 2)
    answer = database.login(username, password)
    if answer in logged_users_id.values():

server_comm.outgoing_msg.append((client, proto.COMMAND_SERVER["login_failed"],
"User already connected"))
        elif answer:
            talking[client] = None
            last_talked[client] = None
            logged_users_id[client] = answer
            server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["login_ok"],
f"{str(answer)}{proto.DATA_DELIMITER}{','.join(languages_online.keys())}")
            else:
                server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["login_failed"], ""))
                cmd = ""

def handle_signup_message():
    global database, client, data, cmd, server_comm, logged_users_id
    username, password = proto.split_data(data, 2)
    answer = database.signup(username, password)
    if not username or not password or username == "" or password == "":
        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["signup_failed"], ""))
        database.remove_user(answer)
    elif answer:
        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["signup_ok"],
f"{str(answer)}{proto.DATA_DELIMITER}{','.join(languages_online.keys())}")
        logged_users_id[client] = answer
    else:
        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["signup_failed"], ""))

def handle_languages_message():
    global database, client, data
    id, languages = proto.split_data(data, 2)
    if database.language_update(languages, id):

```

```

        talking[client] = None
        last_talked[client] = None
        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["languages_ok"], ""))
    else:
        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["languages_error"], ""))

def handle_waiting_message():
    global client, cmd, waiting_clients
    waiting_clients.append(client)

#this function is run with a thread to check if it can assign a client to
another client based languages and last talked conditions
def process_waiting_clients():
    global client
    while True:
        if len(waiting_clients) >= 2:
            for i in range(len(waiting_clients) - 1):
                for j in range(i + 1, len(waiting_clients)):
                    client1 = waiting_clients[i]
                    client2 = waiting_clients[j]

                    if (client1 in last_talked and last_talked[client1] ==
client2) or (client2 in last_talked and last_talked[client2] == client1):
                        continue # Skip pairing if they just talked or don't
have a matching language

                    languages1 =
database.get_languages(logged_users_id[client1])
                    languages2 =
database.get_languages(logged_users_id[client2])
                    common_languages = [item for item in languages1 if item
in languages2]

                    if not common_languages:
                        continue

                    server_comm.outgoing_msg.append((client1,
proto.COMMAND_SERVER["waiting_stop"], ""))
                    server_comm.outgoing_msg.append((client2,
proto.COMMAND_SERVER["waiting_stop"], ""))

                    talking[client1] = client2
                    talking[client2] = client1

                    waiting_clients.remove(client1)
                    waiting_clients.remove(client2)

                    break

            time.sleep(0.1)

def handle_talk_message():

```

```

global client, data
# Send the message to the other client
if talking[client]: # Check if the client is connected to another client
    server_comm.outgoing_msg.append((talking[client],
proto.COMMAND_SERVER["forward_talk"], data))
else:
    pass
    # No connected client to send the message

def handle_stop_talking():
    global client
    # Notify the other client that this client has left
    if talking[client]: # Check if the client is connected to another client
        other_client = talking[client]
        server_comm.outgoing_msg.append((other_client,
proto.COMMAND_SERVER["friend_left"], "Your friend has left the chat.))

        # make sure he isn't talking twice in a row
        last_talked[client] = other_client
        last_talked[other_client] = client

        # make sure they aren't talking right now
        talking[other_client] = None
        talking[client] = None

def handle_change_person_message():
    global client
    if talking[client]: # Check if the client is connected to another client
        other_client = talking[client]

        server_comm.outgoing_msg.append((other_client,
proto.COMMAND_SERVER["friend_left"], "Your friend has left the chat.))

        # make sure he isn't talking twice in a row
        last_talked[client] = other_client
        last_talked[other_client] = client

        # make sure they aren't talking right now
        talking[other_client] = None
        talking[client] = None

        server_comm.outgoing_msg.append((client,
proto.COMMAND_SERVER["change_person_ok"], "putting you in wait list"))

def comm_thread():
    global client, cmd, data, l, server_comm
    server_comm.run()

def separate_incoming():
    global client, cmd, data, l, server_comm
    while True:
        l.acquire()

```

```

        for message in server_comm.incoming_msg:
            client, cmd, data = message
            server_comm.incoming_msg.remove(message)
            handle_client_message()
            l.release()

if __name__ == "__main__":
    threading.Thread(target=comm_thread).start()
    threading.Thread(target=separate_incoming).start()
    threading.Thread(target=process_waiting_clients).start()

```

server_comm code:

```

import socket
import select
import protocol as proto
import threading
import rsa

# Server configuration
HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 2048
CLIENT_LIMIT = 20
l = threading.Lock()

class Comm:
    def __init__(self):
        self.server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.server.bind((HOST, PORT))
        self.server.listen()
        self.clients = []
        self.outgoing_msg = []
        self.incoming_msg = []
        prime1, prime2 = rsa.generate_primes()
        self.public, self.private = rsa.generate_keypair(prime1, prime2)
        self.keys = {}

    # this function gets the data to build an encrypted message to send to
    the client
    def build_and_send_message(self, client, cmd, msg): #ENCRYPT
        try:
            full_msg = proto.build_message(cmd, msg)
            encrypted_msg = rsa.encrypt(self.keys[client], full_msg)
            encrypted_msg_str = ','.join(map(str, encrypted_msg))

            client.send(encrypted_msg_str.encode())
        except Exception as e:
            print(f"Failed to build or send message: {e}")

```

```
# this function receives the message and decrypts it
def recv_message_and_parse(self, client): #DECRYPT
    try:
        full_msg = client.recv(BUFFER_SIZE).decode()
        if not full_msg:
            # Handle client disconnection
            print(f"Connection closed by {client.getpeername()}")
            return None, None

        if client in self.keys:
            split_msg = [int(a) for a in full_msg.split(',')]
            full_msg = rsa.decrypt(self.private, split_msg)

        cmd, data = proto.parse_message(full_msg)
        return cmd, data
    except Exception as e:
        print(f"Failed to receive or parse message: {e}")
        return None, None

def print_client_sockets(self):
    for current_socket in self.clients:
        address = current_socket.getpeername()
        print(f"Client: {address[0]}:{address[1]}")

def run(self):
    while True:
        read_list, write_list, error_list = select.select([self.server] +
self.clients, self.clients, [])
        for current_socket in read_list:
            if current_socket is self.server:
                if len(self.clients) < CLIENT_LIMIT:
                    client, address = self.server.accept()
                    client.send("Connected".encode()) # הודעת שליחת
הצלחה

                    print(f"New connection from {address}")
                    self.clients.append(client)
                    # self.print_client_sockets()
                else:
                    client, client_address = self.server.accept()
                    client.send("Denied".encode()) # כישלון הודעת שליחת
                    client.close()
            else:
                try:
                    result = self.recv_message_and_parse(current_socket)
                    if result is None:
                        print(f"Connection closed by
{current_socket.getpeername()}")
                        self.incoming_msg.append((current_socket,
proto.COMMAND_SERVER["client_left_server"], ""))
                        self.clients.remove(current_socket)
                        current_socket.close()
                        continue

                    cmd, data = result
```

```

        if cmd is None:
            print(f"Error parsing message from
{current_socket.getpeername()}")
            self.incoming_msg.append((current_socket,
proto.COMMAND_SERVER["client_left_server"], ""))
            self.clients.remove(current_socket)
            current_socket.close()
            continue

        elif cmd == proto.COMMAND_CLIENT["give_key"]:
            self.keys[current_socket] = tuple([int(x) for x
in data.split(",")])
            current_socket.send((",").join(map(str,
self.public))).encode()

        else:
            self.incoming_msg.append((current_socket, cmd,
data))

    except Exception as e:
        print(f"Error handling client
{current_socket.getpeername()}: {e}")
        self.incoming_msg.append((current_socket,
proto.COMMAND_SERVER["client_left_server"], ""))
        self.clients.remove(current_socket)
        current_socket.close()

    for message in self.outgoing_msg:
        client, cmd, msg = message
        if client in write_list:
            try:
                self.build_and_send_message(client, cmd, msg)
                self.outgoing_msg.remove(message)
            except Exception as e:
                print(f"Error sending message to
{client.getpeername()}: {e}")
                self.clients.remove(client)
                client.close()

```

client_comm code:

```

import socket
import protocol as proto
import rsa
# Client Configuration
HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 2048

```

```

class Comm:
    def __init__(self):
        self.client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.client.connect((HOST, PORT))
        self.private = None
        self.public = None
        self.server_public = None
        self.status = self.client.recv(BUFFER_SIZE).decode()
        if self.status == "Connected":
            self.exchange_keys()

    def error_and_exit(self, error_msg):
        print("Error: " + error_msg)
        self.status = "Denied"

    # this function gets the data to build an encrypted message to send to
    the server
    def build_and_send_message(self, cmd, data):
        try:
            full_msg = proto.build_message(cmd, data)
            encrypted_msg = rsa.encrypt(self.server_public, full_msg)
            # Convert the list of encrypted integers to a string
            encrypted_msg_str = ','.join(map(str, encrypted_msg))
            self.client.send(encrypted_msg_str.encode()) # Send the string
            representation of the encrypted message
        except Exception as e:
            if "10054" in str(e):
                self.error_and_exit(f"Failed to build or send message: {e}")

    # this function receives the message and decrypts it
    def recv_message_and_parse(self):
        try:
            full_msg = self.client.recv(BUFFER_SIZE).decode()
            split_msg = [int(a) for a in full_msg.split(',')]
            full_msg = rsa.decrypt(self.private, split_msg)
            cmd, data = proto.parse_message(full_msg)

            return cmd, data
        except Exception as e:
            self.error_and_exit(f"Failed to receive or parse message: {e}")

    def exchange_keys(self):
        prime1, prime2 = rsa.generate_primes()
        self.public, self.private = rsa.generate_keypair(prime1, prime2)

self.client.send((proto.build_message(proto.COMMAND_CLIENT["give_key"],
", ".join(map(str, self.public)))).encode())
self.server_public = tuple([int(x) for x in
self.client.recv(BUFFER_SIZE).decode().split(",")])

```

gui code:

```
import customtkinter as ctk
from tkinter import messagebox
import protocol as proto
from PIL import Image
import json

class ChatGUI(ctk.CTk):
    def __init__(self):
        super().__init__()
        self.ID = None
        self.chat_area = None
        self.outgoing_msg = None
        self.entry_password = None
        self.entry_username = None
        self.message_entry = None
        self.language_vars = {}

        self.is_animating = None
        # self.home_frame = None
        # Configure the main window
        self.title("Login/Sign Up")
        self.geometry("600x400") # Set default size of the window
        ctk.set_appearance_mode("dark")
        ctk.set_default_color_theme("blue")

        # Display the main interface
        self.create_start_interface()

    def clear_widgets(self):
        for widget in self.wininfo_children():
            widget.destroy()

    def create_start_interface(self):
        # Clear existing widgets
        self.clear_widgets()

        # Add Login and Sign Up buttons
        login_button = ctk.CTkButton(self, text="Login",
command=self.show_login_interface)
        login_button.pack(pady=10) # Use pack to ensure widgets are
displayed

        signup_button = ctk.CTkButton(self, text="Sign Up",
command=self.show_signup_interface)
        signup_button.pack(pady=10)
    def show_signup_interface(self):
        # Clear existing widgets
        self.clear_widgets()

        # Create Sign Up form
        ctk.CTkLabel(self, text="Username").pack(pady=5)
```



```

self.entry_username = ctk.CTkEntry(self)
self.entry_username.pack(pady=5)

ctk.CTkLabel(self, text="Password").pack(pady=5)
self.entry_password = ctk.CTkEntry(self, show="*")
self.entry_password.pack(pady=5)

# Add Sign Up and Back buttons
ctk.CTkButton(self, text="Sign Up", command=self.signup).pack(pady=5)
ctk.CTkButton(self, text="Back",
command=self.create_start_interface).pack(pady=5)

def show_login_interface(self):
    # Clear existing widgets
    self.clear_widgets()

    # Create Login form
    ctk.CTkLabel(self, text="Username").pack(pady=5)
    self.entry_username = ctk.CTkEntry(self)
    self.entry_username.pack(pady=5)

    ctk.CTkLabel(self, text="Password").pack(pady=5)
    self.entry_password = ctk.CTkEntry(self, show="*")
    self.entry_password.pack(pady=5)

    # Add Login and Back buttons
    ctk.CTkButton(self, text="Login", command=self.login).pack(pady=5)
    ctk.CTkButton(self, text="Back",
command=self.create_start_interface).pack(pady=5)

def login(self):
    self.outgoing_msg = (proto.COMMAND_CLIENT["login"],
f"{self.entry_username.get()} {proto.DATA_DELIMITER} {self.entry_password.get()}")

def signup(self):
    self.outgoing_msg = (proto.COMMAND_CLIENT["signup"],
f"{self.entry_username.get()} {proto.DATA_DELIMITER} {self.entry_password.get()}")

def login_ans(self, cmd, data):
    if cmd == proto.COMMAND_SERVER["login_ok"]:
        self.ID, language_var_string = proto.split_data(data, 2)
        self.define_languages(language_var_string)

        messagebox.showinfo("Login", "Login successful")
        self.outgoing_msg = None
        if int(self.ID) > 1:
            self.create_home_page()
    else:
        messagebox.showerror("Login", "Failed Login")
        self.outgoing_msg = None

def signup_ans(self, cmd, data):
    if cmd == proto.COMMAND_SERVER["signup_ok"]:
        self.ID, language_var_string = proto.split_data(data, 2)
        self.define_languages(language_var_string)

```

```

        messagebox.showinfo("Sign Up", "Sign up successful")
        self.show_language_interface()
        # self.incoming_msg = None
        self.outgoing_msg = None

    else:
        messagebox.showerror("Sign Up", "Username already exists or you
need to use at least one character in the username and password")
        # self.incoming_msg = None
        self.outgoing_msg = None

    def define_languages(self, language_var_string):

        language_var_string = language_var_string.strip("[ ]") # Convert the
string to a list
        language_var_string = language_var_string.replace("'", "")
        lang_var = language_var_string.split(",")

        for lang in lang_var:
            lang = lang.strip()
            var = ctk.StringVar()
            self.language_vars[lang] = var

    def show_language_interface(self):
        # Clear existing widgets
        self.clear_widgets()
        self.title("Select Languages")

        # Create a label to prompt for language selection
        ctk.CTkLabel(self, text="Select Your Preferred
Languages:").pack(pady=10)

        for lang, var in self.language_vars.items():
            ctk.CTkCheckBox(self, text=lang, variable=var, onvalue=lang,
offvalue="").pack(anchor="w")

        # Add a button to proceed to the chat interface
        ctk.CTkButton(self, text="Continue",
command=self.validate_languages).pack(pady=10)

    def validate_languages(self):
        # Get selected languages
        selected_languages = [lang for lang, var in
self.language_vars.items() if var.get() == lang]
        if not selected_languages:
            # Show an error if no languages are selected
            messagebox.showerror("Error", "Please select at least one
language.")
        else:
            languages_selected = ",".join(selected_languages)
            # Proceed with sign-in using selected languages
            self.outgoing_msg = (proto.COMMAND_CLIENT["select_languages"],
f"{self.ID}{proto.DATA_DELIMITER}{languages_selected}")
            # messagebox.showinfo("Languages", f"Selected Languages: {'",

```

```

'.join(selected_languages)}")

def languages_ans(self, cmd):
    if cmd == proto.COMMAND_SERVER["languages_ok"]:
        messagebox.showinfo("Languages", "Languages updated
successfully")
        self.create_home_page()
        # self.incoming_msg = None
        self.outgoing_msg = None
    else:
        messagebox.showerror("Languages", "Error in updating languages")
        # self.incoming_msg = None
        self.outgoing_msg = None

def create_home_page(self):
    self.clear_widgets()
    self.title("Home Page")
    # Define home_frame as a local variable
    home_frame = ctk.CTkFrame(self)
    home_frame.pack(fill="both", expand=True)
    go_play_button = ctk.CTkButton(home_frame, text="Go Talk",
command=self.show_loading_screen)
    go_play_button.pack(pady=20)
    settings_button = ctk.CTkButton(home_frame, text="Settings",
command=self.settings_screen)
    settings_button.pack(pady=20)

def settings_screen(self):
    # Clear existing widgets
    self.clear_widgets()
    self.title("Setting menu")
    # Create a new settings frame as a local variable
    settings_frame = ctk.CTkFrame(self)
    settings_frame.pack(fill="both", expand=True)
    back_button = ctk.CTkButton(settings_frame, text="Back",
command=self.create_home_page)
    back_button.pack(pady=40)
    language_button = ctk.CTkButton(settings_frame, text="Change
Languages", command=self.show_language_interface)
    language_button.pack(pady=20)

def create_chat_interface(self):
    self.is_animating = False # Stop any ongoing animation
    # Clear existing widgets
    self.clear_widgets()
    self.title("Chat")

    exit_button = ctk.CTkButton(self,
text="X",width=30,height=30,fg_color="red",text_color="white",command=self.en
d_talking_and_go_home)
    # Place the exit button in the top right corner
    exit_button.place(relx=0.98, rely=0.02, anchor="ne")

    # Create chat interface
    self.chat_area = ctk.CTkTextbox(self, state='disabled', width=400,
height=200)
    self.chat_area.pack(padx=10, pady=10)

```

```

self.message_entry = ctk.CTkEntry(self)
self.message_entry.pack(padx=10, pady=10, fill=ctk.X)
self.message_entry.bind("<Return>", self.show_and_return_message)

button_frame = ctk.CTkFrame(self)
button_frame.pack(padx=10, pady=10)

ctk.CTkButton(button_frame, text="Send",
command=self.show_and_return_message).grid(row=0, column=0, padx=5)
ctk.CTkButton(button_frame, text="Random",
command=self.change_random_person).grid(row=0, column=1, padx=5)

def end_talking_and_go_home(self):
    self.outgoing_msg = (proto.COMMAND_CLIENT["done_talking"], "")
    self.create_home_page()

def show_and_return_message(self):
    message = self.message_entry.get()
    # looks like ctk error this message is called twice upon button press
    if message == "":
        return
    self.message_entry.delete(0, ctk.END)
    self.display_message(f"You: {message}")
    self.outgoing_msg = (proto.COMMAND_CLIENT["talk"], message)

def pop_message(self, message):
    messagebox.showinfo("notice", message)

def display_message(self, message):
    self.chat_area.configure(state='normal')
    self.chat_area.insert(ctk.END, message + '\n')
    self.chat_area.configure(state='disabled')
    self.chat_area.yview(ctk.END)

def change_random_person(self):
    self.outgoing_msg = (proto.COMMAND_CLIENT["change_person"], "")

def change_person_ans(self, cmd):
    if cmd == proto.COMMAND_SERVER["change_person_ok"]:
        self.outgoing_msg = None
        self.show_loading_screen()
    else:
        messagebox.showinfo("Random", "error trying to change person")
        self.outgoing_msg = None

def show_loading_screen(self):
    # Clear the window
    self.clear_widgets()
    self.title("Waiting Screen")

    # Create a new frame for the loading screen
    loading_frame = ctk.CTkFrame(self)
    loading_frame.pack(expand=True, fill="both")

```

```

self.outgoing_msg = (proto.COMMAND_CLIENT["waiting"], "")

# Load the GIF using Pillow
try:
    gif_image = Image.open("searching.gif") # Replace with your GIF
file path
except Exception as e:
    print("Error loading GIF:", e)
    return

# Load all frames from the gif and convert them to CTKImage objects.
frames = []
try:
    while True:
        frame = gif_image.copy().resize((300, 300)) # Resize frame
as needed
        ctk_image = ctk.CTkImage(light_image=frame, dark_image=frame,
size=(300, 300))
        frames.append(ctk_image)
        gif_image.seek(len(frames)) # Move to the next frame
except EOFError:
    pass # End of sequence

if not frames:
    # No frames loaded
    return

# Create a label to display the animated gif.
gif_label = ctk.CTkLabel(loading_frame, image=frames[0], text="")
gif_label.pack(expand=True)

# Start the animation in a separate thread
def animate(frame_index=0):
    if self.is_animating and gif_label.winfo_exists():
        gif_label.configure(image=frames[frame_index])
        next_frame = (frame_index + 1) % len(frames)
        self.after(100, animate, next_frame)
    else:
        self.is_animating = False # Stop the animation if gif_label
no longer exists

self.is_animating = True
animate()

def show_admin_screen(self):
    # Clear existing widgets
    self.clear_widgets()
    self.title("Admin Home Page")
    self.geometry("800x600")
    # Create a label for the admin screen
    ctk.CTkLabel(self, text="Admin Dashboard").pack(pady=10)

    # Button to show online people
    online_button = ctk.CTkButton(self, text="Show Online Users",

```

```

command=self.show_online_people_request)
online_button.pack(pady=5)

# Button to show talking people
talking_button = ctk.CTkButton(self, text="Show Talking Users",
command=self.show_talking_people_request)
talking_button.pack(pady=5)

# Button to show languages in online users
languages_button = ctk.CTkButton(self, text="Show Languages in Online
Users", command=self.show_languages_in_online_users_request)
languages_button.pack(pady=5)

def show_online_people_request(self):
    self.outgoing_msg = (proto.COMMAND_CLIENT_ADMIN["show_online"], "")

def show_online_people_ans(self, cmd, data):
    if cmd == proto.COMMAND_SERVER["show_online_ok"]:
        self.show_online_people_screen(data)
        self.outgoing_msg = None
    else:
        messagebox.showerror("Show online", "Failed to show screen")
        self.outgoing_msg = None

def show_talking_people_request(self):
    self.outgoing_msg = (proto.COMMAND_CLIENT_ADMIN["show_talking"], "")

def show_talking_people_ans(self, cmd, data):
    if cmd == proto.COMMAND_SERVER["show_talking_ok"]:
        if data: # Check if data is not empty
            talking_id_dict = json.loads(data) # Convert JSON string
back to dictionary
            self.show_talking_people_screen(talking_id_dict) # Implement
this method to display the talking IDs
            self.outgoing_msg = None
        else:
            messagebox.showinfo("Show Talking", "No one is currently
talking.")
            self.show_admin_screen()
            self.outgoing_msg = None
    else:
        messagebox.showerror("Show talking", "Failed to show screen")
        self.outgoing_msg = None

def show_languages_in_online_users_request(self):
    self.outgoing_msg =
(proto.COMMAND_CLIENT_ADMIN["show_languages_online"], "")

def show_languages_in_online_users_ans(self, cmd, data):
    if cmd == proto.COMMAND_SERVER["show_languages_online_ok"]:
        if data:
            languages_dict = json.loads(data)
            self.show_languages_in_online_users_screen(languages_dict)
            self.outgoing_msg = None
        else:
            messagebox.showinfo("Show online languages", "Failed to
receive languages")

```

```

        else:
            messagebox.showerror("Show languages online", "Failed to show
screen")
            self.outgoing_msg = None

# ADD REFRESH MECHANISM TO ALL THE SCREENS AND A BACK OPTION

def show_online_people_screen(self, data):
    # Clear existing widgets
    self.clear_widgets()
    self.title("Online Users")

    # Create a label for the online users screen
    ctk.CTkLabel(self, text="Online Users:").pack(pady=10)

    # Display the list of online user IDs in a single line
    if data:
        online_users = ', '.join(data) # Join user IDs with a comma and
space
        online_users_label = ctk.CTkLabel(self, text=online_users) #
Create a single label for all users
        online_users_label.pack(pady=5)
    else:
        ctk.CTkLabel(self, text="No users online").pack(pady=5)

    # Add Back button to return to the admin screen
    back_button = ctk.CTkButton(self, text="Back",
command=self.show_admin_screen)
    back_button.pack(pady=10)

    # Add Refresh button (functionality not implemented yet)
    refresh_button = ctk.CTkButton(self, text="Refresh",
command=self.show_online_people_request) # Placeholder for refresh
functionality
    refresh_button.pack(pady=10)

def show_talking_people_screen(self, data):
    # Clear existing widgets
    self.clear_widgets()
    self.title("Talking Users")

    # Create a label for the talking people screen
    ctk.CTkLabel(self, text="Talking People:").pack(pady=10)

    # Check if data is not empty
    if data:
        # Display each pair of IDs with a colon between them
        for key, value in data.items():
            talking_pair_label = ctk.CTkLabel(self, text=f"{key} :
{value}")
            talking_pair_label.pack(pady=5)
    else:
        ctk.CTkLabel(self, text="No one is currently
talking.").pack(pady=5)

    # Add Back button to return to the admin screen

```

```

        back_button = ctk.CTkButton(self, text="Back",
command=self.show_admin_screen)
        back_button.pack(pady=10)

        # Add Refresh button to refresh the screen
        refresh_button = ctk.CTkButton(self, text="Refresh",
command=self.show_talking_people_request)
        refresh_button.pack(pady=10)

    def show_languages_in_online_users_screen(self, data):
        # Clear existing widgets
        self.clear_widgets()
        self.title("Languages in Online Users")

        # Create a label for the languages in online users screen
        ctk.CTkLabel(self, text="Languages in Online Users:").pack(pady=10)

        # Check if data is not empty
        if data:
            # Display each language with the number of online people who know
it
            for language, count in data.items():
                language_label = ctk.CTkLabel(self, text=f"{language} :
{count}")
                language_label.pack(pady=5)
            else:
                ctk.CTkLabel(self, text="No languages available.").pack(pady=5)

        # Add Back button to return to the admin screen
        back_button = ctk.CTkButton(self, text="Back",
command=self.show_admin_screen)
        back_button.pack(pady=10)

        # Add Refresh button to refresh the screen
        refresh_button = ctk.CTkButton(self, text="Refresh",
command=self.show_languages_in_online_users_request)
        refresh_button.pack(pady=10)

```